TÉCNICO LISBOA

# Path Following and Control for Autonomous Driving of a Formula Student Car

**Alexandre Pereira de Athayde do Passo**

Thesis to obtain the Master of Science Degree in

## Mechanical Engineering

Supervisors: Prof. José Raul Carreira Azinheira
Prof.ª Alexandra Bento Moutinho

### Examination Committee

Chairperson: Prof. Carlos Baptista Cardeira
Supervisor: Prof. José Raul Carreira Azinheira
Member of the Committee: Prof. Miguel Afonso Dias de Ayala Botto

**January 2021**

Pelo que nunca chegaste a ver.

# Acknowledgments

I would like to thank firstly and foremost to the supervising professors in their constant help throughout the development of this dissertation, always providing me with answers, or, more importantly, with the means to obtain them.

I also thank my mom for her incredible patience and tireless help during the eight months of the development of this thesis, many of which in lock-down.

Kind thoughts go for my friends that also suffered from these months, from unanswered messages or calls to my daydreams of how to build an autonomous vehicle.

Lastly, I would like to thank Associação dos Estudantes do Instituto Superior Técnico (AEIST) for the immense contribution to my personal growth during my academic years, undoubtedly marked by the time I was part of it.

# Resumo

Esta dissertação foi desenvolvida com o objetivo de implementar vários algoritmos de cálculo de erros de caminho e diferentes estratégias de controlo lateral para o guiamento de um veículo Formula Student (FST) autónomo. Adicionalmente, a velocidade foi controlada incorporando a curvatura do caminho.
Foi desenvolvida uma simulação dum veículo FST para que fossem implementadas e comparadas as diferentes estratégias de seguimento de caminho. O cálculo dos erros foi feito no referencial do veículo, explorando os efeitos duma antecipação relativa a pontos mais à frente no caminho. A regulação da velocidade foi feita também com esta antecipação, obtendo a curvatura num ponto futuro, e adequando a velocidade para a compensar. Em termos de estratégias de controlo, optou-se por usar não lineares, denominadas Stanley e L1 neste texto, e três consistindo em aplicações de métodos de Controlo Ótimo (LQR/LQG) a variações do modelo bicicleta com 2 estados, 3 estados e 4 estados, recorrendo ao escalonamento de ganhos.
Por último, as diversas estratégias foram implementadas e testadas com a simulação desenvolvida do modelo real. No geral, todos os controladores apresentaram resultados satisfatórios no seguimento do caminho, sendo que o controlador LQG com 4 estados e o controlador L1 se apresentaram como soluções mais vantajosas, tendo desvios menores em relação a caminho e menor oscilação do guiamento. A aplicação da antecipação demonstrou ser uma vantagem para a regulação da velocidade e para o cálculo dos erros, contribuindo para um seguimento mais fiável e com menos oscilação da direção.

**Palavras-chave:** formula student, veículos autónomos, controlo LQR, seguimento de trajetória, guiamento lateral, controlo não-linear

# Abstract

This dissertation was developed with the objective of designing different ways of obtaining path-following errors, as well as the implementing several lateral control algorithms, with the aim of achieving autonomous guidance of a driverless Formula Student (FST) vehicle. Additionally, the velocity was regulated according to the curvature of the path.

A simulation of a FST vehicle was developed with the aim of implementing and comparing these path-following strategies. The error computation was made in the frame of the vehicle, also exploring the usage of a lookahead distance for future waypoints. The regulation of the speed considered this anticipation as well, obtaining the curvature of the path ahead of the car, adjusting the velocity as to maintain a correct tracking of the path. Regarding the control strategies, two nonlinear were used, being adapted versions of the Stanley and L1 controllers, and three consisting on applications of Optimal Control (LQR/LQG) methods with gain scheduling to different bicycle models: with 2 states, 3 states and 4 states.

These different strategies were then implemented and tested with the simulation of the real model. All of them provided with satisfactory results in the tracking of the path, with the L1 controller and the 4 states LQG presenting less deviation in relation to the path and smaller oscillation of the steering. The lookahead application presented itself as a significant advantage, either for the velocity regulation or the error computation, allowing the vehicle to achieve an overall better path-following.



**Keywords:** formula student, autonomous vehicles, LQR control, path-following, lateral guidance, nonlinear control

# Contents


Acknowledgments . . . . . v
Resumo . . . . . vii
Abstract . . . . . ix
Contents . . . . . xi
List of Tables . . . . . xv
List of Figures . . . . . xvii
Nomenclature . . . . . xxi
Glossary . . . . . xxvii

**1 Introduction** **1**
1.1 Motivation . . . . . 1
1.2 Topic Overview . . . . . 1
1.3 Objectives . . . . . 4
1.4 Thesis Outline . . . . . 5

**2 Theoretical Background** **6**
2.1 Linear Control Design . . . . . 6
2.1.1 Linearization . . . . . 8
2.1.2 Properties of linear systems . . . . . 9
2.1.3 Feedback control . . . . . 10
2.1.4 Observer design . . . . . 15
2.1.5 Separation theorem and LQG control . . . . . 16
2.2 Nonlinear Control Guidance Strategies . . . . . 17
2.2.1 Stanley controller . . . . . 17
2.2.2 L1 controller . . . . . 18

**3 Vehicle Models** **19**
3.1 Formula Student Vehicle . . . . . 19
3.1.1 Steering . . . . . 21
3.1.2 Tyres . . . . . 22
3.1.3 Propulsion . . . . . 23
3.1.4 Resistive forces . . . . . 24

3.1.5 Model of the vehicle . . . 24
3.2 Models for Lateral Control . . . 27
3.2.1 Bicycle kinematics model . . . 27
3.2.2 Bicycle kinematics model with sideslip . . . 29
3.2.3 Bicycle dynamics model . . . 30
3.3 Models for Velocity Control . . . 32

**4 Path Following** **34**
4.1 Global and Local Coordinate Systems . . . 34
4.2 Path Following Errors and Curvature Computation . . . 36
4.3 Lateral Control . . . 44
4.3.1 Stanley Controller . . . 44
4.3.2 L1 Controller . . . 46
4.3.3 2 state LQR controller . . . 48
4.3.4 3 state LQG controller . . . 50
4.3.5 4 state LQG controller . . . 52
4.4 Velocity Control . . . 55
4.4.1 References for velocity regulation . . . 55
4.4.2 LQT controller . . . 56
4.5 Path-Following Architecture . . . 58

**5 Simulation Environment** **61**
5.1 Tracks for Simulation and Testing . . . 61
5.2 Evaluation Metrics . . . 64
5.3 Simulator Description . . . 66

**6 Results** **68**
6.1 Performance Evaluation at Constant Velocity . . . 69
6.2 Performance Evaluation at Varying Velocity . . . 71
6.3 Discussion . . . 73
6.3.1 Discussion of results at constant velocity . . . 73
6.3.2 Discussion of results at varying velocity . . . 75
6.3.3 Final considerations . . . 76

**7 Conclusions and Future Work** **79**
7.1 Achievements . . . 79
7.2 Future Work . . . 80

**Bibliography** **80**

**A Validation of the controllers** **84**

**B Results at varying velocity** **86**

B.1 Results for Track 1 . . . . . 86

B.2 Results for Track 2 . . . . . 91

B.3 Results for Track 3 . . . . . 96

# List of Tables


3.1 Constants and parameters used in simulation . . . . . 26

4.1 Validation of the Stanley controller . . . . . 45
4.2 Validation of the L1 controller . . . . . 48
4.3 Validation of the LQR controller with 2 states . . . . . 49
4.4 Estimation of lateral velocity with a 3 state Kalman filter . . . . . 51
4.5 Validation of the LQG controller with 3 states . . . . . 52
4.6 Estimation of lateral and angular velocities with a 4 state Kalman filter . . . . . 53
4.7 Validation of the LQG controller with 4 states . . . . . 54

5.1 Evaluation metrics used for tests at constant velocity. . . . . 66
5.2 Evaluation metrics used for tests at varying velocity. . . . . 66

6.1 Performance evaluation at a constant velocity of 5 metres per second . . . . . 69
6.2 Performance evaluation at a constant velocity of 10 metres per second . . . . . 70
6.3 Performance evaluation for Track 1 . . . . . 71
6.4 Performance evaluation for Track 2 . . . . . 72
6.5 Performance evaluation for Track 3 . . . . . 72

# List of Figures


1.1 Layout of the dissertation . . . . . 5

2.1 State-space representation of a linear system . . . . . 7
2.2 State-space representation of a regulator . . . . . 11
2.3 State-space representation of a servo system . . . . . 12
2.4 State-space representation of a servo system with the integral of the error feedback . . . 12
2.5 State-space representation of an observer . . . . . 15
2.6 Path tracking logic for the Stanley controller . . . . . 18
2.7 Path tracking logic for the L1 controller . . . . . 18

3.1 Formula Student Vehicle FST 09e . . . . . 20
3.2 Forces acting on the FST vehicle . . . . . 21
3.3 Schematic of the bicycle kinematics model . . . . . 28
3.4 Schematic of the bicycle dynamics model with sideslip . . . . . 31

4.1 Local and global coordinate systems . . . . . 35
4.2 Path-following errors for a line segment reference . . . . . 37
4.3 Path-following errors for a arc of circumference reference . . . . . 38
4.4 Arc of circumference with positive and negative radii, and path-following errors . . . . . 38
4.5 Path-following errors using a polynomial function as reference . . . . . 39
4.6 Path-following errors using a lookahead distance . . . . . 42
4.7 Validation of the Stanley controller using a line segment as reference path . . . . . 46
4.8 Validation of the Stanley controller using a circumference as reference path . . . . . 46
4.9 Validation of the L1 controller using a line segment as reference path . . . . . 47
4.10 Validation of the L1 controller using a circumference as reference path . . . . . 48
4.11 Estimation of lateral velocity with a Kalman filter using 3 states . . . . . 51
4.12 Estimation of lateral and angular velocities with a Kalman filter using 4 states . . . . . 53
4.13 Velocity profile applied in function of the path curvature . . . . . 56
4.14 Validation of the LQR controller for the velocity control . . . . . 58
4.15 Path-following architecture . . . . . 59
4.16 Schematic of path-following with two lookahead distances . . . . . 60

5.1 Original waypoints of Track 1 and curvatures . . . 62
5.2 Smoothed reference path for Track 1 . . . 62
5.3 Curvature and side limits of Track 1 . . . 63
5.4 Track 2: curvature and distances to the borders of the track . . . 63
5.5 Track 3: curvature and distances to the borders of the track . . . 64
5.6 Simulink model used for simulations . . . 67

6.1 Example of a simulation in which the vehicle crosses the limits of the track . . . 77
6.2 Losses of the reference path . . . 77

A.1 Validation of the LQR controller with 2 states using a line segment as reference path . . . 84
A.2 Validation of the LQR controller with 2 states using a circumference as reference path . . . 84
A.3 Validation of the LQG controller with 3 states using a line segment as reference path . . . 85
A.4 Validation of the LQG controller with 3 states using a circumference as reference path . . . 85
A.5 Validation of the LQG controller with 4 states using a line segment as reference path . . . 85
A.6 Validation of the LQG controller with 4 states using a circumference as reference path . . . 85

B.1 Track 1: Stanley controller without lookahead distance . . . 86
B.2 Track 1: Stanley controller with 5 metre lookahead distance . . . 86
B.3 Track 1: Stanley controller with 10 metre lookahead distance . . . 87
B.4 Track 1: Stanley controller with $L_e$ automatically regulated . . . 87
B.5 Track 1: L1 controller with 5 metre lookahead distance . . . 87
B.6 Track 1: L1 controller with 10 metre lookahead distance . . . 87
B.7 Track 1: L1 controller with $L_e$ automatically regulated . . . 88
B.8 Track 1: 2-state LQR controller without lookahead distance . . . 88
B.9 Track 1: 2-state LQR controller with 5 metre lookahead distance . . . 88
B.10 Track 1: 2-state LQR controller with 10 metre lookahead distance . . . 88
B.11 Track 1: 2-state LQR controller with $L_e$ automatically regulated . . . 89
B.12 Track 1: 3-state LQG controller without lookahead distance . . . 89
B.13 Track 1: 3-state LQG controller with 5 metre lookahead distance . . . 89
B.14 Track 1: 3-state LQG controller with 10 metre lookahead distance . . . 89
B.15 Track 1: 3-state LQG controller with $L_e$ automatically regulated . . . 90
B.16 Track 1: 4-state LQG controller without lookahead distance . . . 90
B.17 Track 1: 4-state LQG controller with 5 metre lookahead distance . . . 90
B.18 Track 1: 4-state LQG controller with 10 metre lookahead distance . . . 90
B.19 Track 1: 4-state LQG controller with $L_e$ automatically regulated . . . 91
B.20 Track 2: Stanley controller without lookahead distance . . . 91
B.21 Track 2: Stanley controller with 5 metre lookahead distance . . . 91
B.22 Track 2: Stanley controller with 10 metre lookahead distance . . . 91
B.23 Track 2: Stanley controller with $L_e$ automatically regulated . . . 92

B.24 Track 2: L1 controller with 5 metre lookahead distance . . . . . 92
B.25 Track 2: L1 controller with 10 metre lookahead distance . . . . . 92
B.26 Track 2: L1 controller with $L_e$ automatically regulated . . . . . 92
B.27 Track 2: 2-state LQR controller without lookahead distance . . . . . 93
B.28 Track 2: 2-state LQR controller with 5 metre lookahead distance . . . . . 93
B.29 Track 2: 2-state LQR controller with 10 metre lookahead distance . . . . . 93
B.30 Track 2: 2-state LQR controller with $L_e$ automatically regulated . . . . . 93
B.31 Track 2: 3-state LQG controller without lookahead distance . . . . . 94
B.32 Track 2: 3-state LQG controller with 5 metre lookahead distance . . . . . 94
B.33 Track 2: 3-state LQG controller with 10 metre lookahead distance . . . . . 94
B.34 Track 2: 3-state LQG controller with $L_e$ automatically regulated . . . . . 94
B.35 Track 2: 4-state LQG controller without lookahead distance . . . . . 95
B.36 Track 2: 4-state LQG controller with 5 metre lookahead distance . . . . . 95
B.37 Track 2: 4-state LQG controller with 10 metre lookahead distance . . . . . 95
B.38 Track 2: 4-state LQG controller with $L_e$ automatically regulated . . . . . 95
B.39 Track 3: Stanley controller without lookahead distance . . . . . 96
B.40 Track 3: Stanley controller with 5 metre lookahead distance . . . . . 96
B.41 Track 3: Stanley controller with 10 metre lookahead distance . . . . . 96
B.42 Track 3: Stanley controller with $L_e$ automatically regulated . . . . . 96
B.43 Track 3: L1 controller with 5 metre lookahead distance . . . . . 97
B.44 Track 3: L1 controller with 10 metre lookahead distance . . . . . 97
B.45 Track 3: L1 controller with $L_e$ automatically regulated . . . . . 97
B.46 Track 3: 2-state LQR controller without lookahead distance . . . . . 97
B.47 Track 3: 2-state LQR controller with 5 metre lookahead distance . . . . . 98
B.48 Track 3: 2-state LQR controller with 10 metre lookahead distance . . . . . 98
B.49 Track 3: 2-state LQR controller with $L_e$ automatically regulated . . . . . 98
B.50 Track 3: 3-state LQG controller without lookahead distance . . . . . 98
B.51 Track 3: 3-state LQG controller with 5 metre lookahead distance . . . . . 99
B.52 Track 3: 3-state LQG controller with 10 metre lookahead distance . . . . . 99
B.53 Track 3: 3-state LQG controller with $L_e$ automatically regulated . . . . . 99
B.54 Track 3: 4-state LQG controller without lookahead distance . . . . . 99
B.55 Track 3: 4-state LQG controller with 5 metre lookahead distance . . . . . 100
B.56 Track 3: 4-state LQG controller with 10 metre lookahead distance . . . . . 100
B.57 Track 3: 4-state LQG controller with $L_e$ automatically regulated . . . . . 100

# Nomenclature

### Greek symbols

| | |
|---|---|
| $\alpha$ | Slip angle. |
| $\beta$ | Sideslip angle. |
| $\delta$ | Steering angle. |
| $\eta$ | Angle between the velocity of the car, $V$, and a given lookahead line. |
| $\eta_{motor}$ | Efficiency of the motor. |
| $\mu_{lat}$ | Lateral friction coefficient between the tyres of the car and the track. |
| $\mu_x$ | Mean value of a generic variable $x$. |
| $\omega_{wheel}$ | Angular velocity of the wheel. |
| $\psi$ | Heading of the vehicle, or orientation angle. |
| $\rho$ | Air density. |
| $\tau$ | Time variable used for the performance index $J$ over a finite time period. |

### Roman symbols

| | |
|---|---|
| $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$ | State-space matrices of a linear model. |
| $\mathbf{I}$ | Identity matrix. |
| $\mathbf{K}$ | Controller gain matrix. |
| $\mathbf{L}$ | Observer gain matrix. |
| $\mathbf{P}$, $\mathbf{P_0}$ | Solutions of the Algebraic Riccati Equation for the LQR controller and Kalman filter, respectively. |
| $\mathbf{Q_{Lyap}}$, $\mathbf{P_{Lyap}}$ | Matrices for the Lyapunov Equatiom. |
| $\mathbf{Q}$, $\mathbf{R}$ | Weighting matrices for the LQR controller. |

| | |
|---|---|
| $\mathbf{Q}_0$, $\mathbf{R}_0$ | Weighting matrices for the Kalman filter. |
| $\mathbf{R}_\psi$ | Bidimensional rotation matrix around the $z$ axis with an angle $\psi$. |
| $\mathcal{C}$ | Controllability matrix. |
| $\mathcal{O}$ | Observability matrix. |
| $\mathbf{e}$ | Error vector obtained between a reference vector $\mathbf{r}$ and the corresponding state vector $\mathbf{x}$. |
| $\mathbf{r}$ | Reference vector. |
| $\mathbf{u}$ | Input vector. |
| $\mathbf{x}$ | State vector. |
| $\mathbf{y}$ | Output vector. |
| $C_{circ} = [X_C, Y_C]$ | Center of a circumference arc. |
| $G = (X_G, Y_G)$ | Global coordinate system. |
| $P = [X, Y]$ | Position of the vehicle in the global coordinate system. |
| $V = [u, v]$ | Velocity of the vehicle in the local coordinate system. |
| $W = [X_W, Y_W]$ | Waypoint position expressed in the global coordinate system. |
| $A_F$ | Frontal area of the vehicle. |
| $B_{tyre}$ | Stiffness factor of the tyre. |
| $C_d$ | Aerodynamic drag coefficient. |
| $C_{roll}$ | Rolling resistance coefficient. |
| $C_{tyre}$ | Shape factor of the tyre. |
| $D_{i,j}$ | Displacement vector between two points $P_i$ and $P_j$, expressed in the global coordinate system. |
| $D_{tyre}$ | Peak-value of the lateral force for the tyre. |
| $F^{tyre}$ | Force acting on the tyre. |
| $F_{aero}$ | Arodynamic drag force. |
| $F_{prop}$ | Propulsion force. |
| $F_r$ | Rolling resistance force. |
| $GR$ | Gear radio of the transmission. |

| | |
|---|---|
| $I_z$ | Inertia moment of the vehicle around its $z$ azis. |
| $J$ | Performance index for the LQR controller. |
| $K$ | Curvature of path obtained in global coordinates. |
| $L$ | Wheelnbase of the vehicle. |
| $L_A$ | Lookahead distance. |
| $L_e$ | Lookahead distance for error computation. |
| $L_k$ | Lookahead distance for curvature computation. |
| $L_F$ | Distance of the CG of the car to the front axis. |
| $L_R$ | Distance of the CG of the car to the rear axis. |
| $L_W$ | Track width of the car. |
| $R_{circ}$ | Radius of a circumference arc. |
| $RP$ | Current reference point. |
| $RP_e$ | Reference point for error computation. |
| $RP_k$ | Reference point for curvature computation. |
| $T_{max}$ | Maximum motor torque. |
| $T_{sol}$ | Tangent of path at the solution point. |
| $T_s$ | Sample time. |
| $a_{lat}$ | Lateral acceleration. |
| $c_\delta$ | Time constant of the steering actuation. |
| $d_{cmd}$ | Longitudinal velocity command. |
| $e_\psi$ | Orientation error. |
| $e_r$ | Estimation error for the angular velocity $r$. |
| $e_v$ | Estimation error for the lateral velocity $v$. |
| $e_y$ | Crosstrack error. |
| $f_{pol}$ | Polynomial function obtained from the interpolation of waypoints in the local frame. |
| $g$ | Acceleration of gravity. |
| $k$ | Curvature of path obtained in local coordinates. |
| $m$ | Mass of the car. |

| | |
|---|---|
| $r$ | Angular velocity around the $z$ axis of the vehicle. |
| $r_{wheel}$ | Outer radius of the wheel. |
| $s$ | Complex variable. |
| $s_{path}$ | Path variable. |
| $t$ | Time. |
| $w = [x_W, y_W]$ | Waypoint position expressed in the vehicle frame. |
| $m^i{}_{eval}$ | Evaluation metric $i$. |
| $\tilde{x}$ | Estimation of state $x$ obtained from an observer. |
| $\hat{x}$ | Deviation of state $x$ from the equilibrium condition $x_{eq}$. |

**Subscripts**

| | |
|---|---|
| $\infty$ | Infinite time. |
| $aug$ | Augmented state. |
| $CG$ | Taken at the Center of Gravity of the vehicle. |
| $const$ | Indicates variable or parameter obtained at constant velocity. |
| $eq$ | Denotes equilibrium or constant conditions. |
| $eval$ | Specific for evaluation purposes. |
| $F, R$ | Denotes front or rear of the vehicle. |
| $FR, FL, RR, RL$ | Front-right, front-left, rear-right and rear-left tyres. |
| $lat$ | Referent to lateral control. |
| $local$ | Variables or parameters represented in the local coordinate system. |
| $max$ | Maximum value for the variable or parameter. |
| $min$ | Minimum value for the variable or parameter. |
| $ref$ | Reference value. |
| $RP$ | Taken at the current Reference Point. |
| $sol$ | Obtained at a solution point. |
| $tyre$ | Refers to a tyre ($FR, FL, RR, RL$). |
| $var$ | Indicates variable or parameter obtained at varying velocity. |

| | |
|---|---|
| $X, Y, Z$ | Cartesian components on a global referential. |
| $x, y, z$ | Cartesian components on a local referential. |

**Superscripts**

| | |
|---|---|
| $2S$ | Reffers to the LQR controller using two states. |
| $3S$ | Reffers to the LQG controller using three states. |
| $4S$ | Reffers to the LQG controller using four states. |
| $\perp$ | Orthogonal. |
| $L$ | Obtained using a lookahead distance. |
| $T$ | Transpose. |
| $vel$ | Reffers to the LQR controller for velocity control. |

# Glossary

| | |
|---|---|
| **AMZ** | Academic Motorsports Club Zurich |
| **ARE** | Algebraic Riccati Equation |
| **CG** | Center of Gravity |
| **DARPA** | Defense Advanced Research Projects Agency |
| **FST** | Formula Student |
| **GPS** | Global Positioning System |
| **LIDAR** | LIght Detection And Ranging |
| **LQG** | Linear Quadratic Gaussian |
| **LQR** | Linear Quadratic Regulator |
| **LTI** | Linear Time Invariant |
| **MIMO** | Multiple Inputs / Multiple Outputs |
| **MPC** | Model Predictive Control |
| **NED** | North-East-Down |
| **RMS** | Root Mean Square |
| **SISO** | Single Input / Single Outputs |
| **UAV** | Unmanned Aerial Vehicle |

# Chapter 1

# Introduction

In this first chapter, the motivation for this dissertation will be described, as well as an overview on the control of driverless vehicles. The objectives of this thesis will be explained as well, and the outline of this dissertation is shown, as to provide a comprehensive insight of the developed work.

## 1.1 Motivation

Autonomous driving is one of the central topics in the twenty-first century, with recent breakthroughs technology and academic-wise enabling for vehicles to steer and guide themselves. Within this main topic, Formula Student teams are creating the world of driverless competition, with complex algorithms and control strategies presenting a new scenario where autonomous driving plays a central role and providing a modern engineering field related with race-cars and self-driving systems as well. The Formula Student team from Instituto Superior Técnico - Universidade de Lisboa, FST Lisboa, is currently developing its first self-driven car, FST10d, with the objective of competing in driverless competitions.
With the referred goal of designing a driverless car, several theses have been in development, addressing interesting fields related to autonomous driving. This dissertation is inserted in such topic, namely on the application of different path-following algorithms and control methods, exploring the advantages and limitations of each one, and how these could benefit FST Lisboa in their objective of having a fully autonomous car. Self-driving is surely an area of relevant significance in different fields of engineering, as it requires interconnection between the dynamics of the vehicle, the sensors or cameras used to perceive the environment, the planning of the path the car should take, and the control strategies used. Overcoming the challenges from all these factors is the central motivation for this work, hoping to provide a fruitful contribution on this subject for FST Lisboa.

## 1.2 Topic Overview

Autonomous vehicles are a topic that has been demonstrating a continuous growth and investment in the last decades, with particular emphasis in recent years due to the permanent development of the re-

quired technology and processing power. As one may think, such autonomy is not restricted to car-like vehicles: it can range from small unguided robotic vehicles [1] that despite their size or simplicity, already have the necessary sensors and actuating mechanisms for performing complex tasks; to vehicles capable of underwater or aerial navigation, which the commonly named Unmanned Aerial Vehicles (UAVs) are a remarking example. Naturally, the development of every one of these areas is a world by itself, each one having their respective hindrances and challenges. Focusing on the case of driverless cars, it is commonly known that some automobile brands already have cars that are capable of driving themselves, but there are still numerous obstacles needed to overcome in order to achieve the widespread of such autonomy in real-driving scenarios, whether these are ethical issues, social challenges, related to the trust in driverless cars [2], or technological barriers, since there are still unacceptable error margins in many of the subsystems required for navigation [3], among many others. Therefore, driverless cars, along with the necessary technologies, still have a long course to cover before they can be part of a day-to-day reality, but progress is continuously being made for this goal.

Driverless cars and vehicles in general contain three fundamental parts to their autonomy: perception, planning and control. The perception, as the word itself indicates, allows the vehicle, or the running program, to take notice of the environment around the physical body of the car, in order to direct its movement. In general, perception algorithms focus on the usage of sensors [4], computer vision, GPS (Global Positioning System), or more commonly a given combination of two or more of these elements [1, 3, 5, 6], allowing these to complement themselves and even having some beneficial redundancy, with the goal of providing a more accurate positioning of the objects surrounding the vehicle. With the emergence of neural networks, machine learning and deep learning in day-to-day aspects, such knowledge has been contributing to the progress of perception algorithms as well [7], allowing for an almost automatic recognition of features in the environment, crucial to the desired driving behaviour. Regarding autonomous racing cars, one advantage manifests itself: since these vehicles are yet expected to compete in races by themselves, the degree of unpredictability of their surroundings is distinctly lower that the one a vehicle driving in a city-scenario would face, for example, which allows for simpler approaches regarding perception, relying frequently on the combination of LIDAR (Light Detection and Ranging) sensors and computer vision [5, 8]. On the other hand, the planning and control on these situations frequently requires more data than these systems can provide, having to account for ways of knowing accelerations, like a gyroscope or accelerometer, or even resorting to the estimation of such variables [9].

Path planning, the second referred component, is the key component that articulates what the vehicle perceives with how it should move in future instants. Similarly to the described perception-related systems, the complexity of the underlying path planning algorithms matches the scenario where the car is inserted. Such algorithms can range from simple lane keeping capacities [10] or generating a path avoiding certain obstacles [1, 11] to programs that allow collision avoidance from moving objects [12], even being able to be aware of the other road users [3]. When it comes to racing situations, the degrees of freedom are reduced, frequently with no other relevant actors beyond the vehicle and the track. In these cases, path-planning bases itself on the calculation of an optimal course, accounting for

the boundaries of the track [13], frequently resorting to Simultaneous Location and Mapping (SLAM) for further optimization in following laps [6]. Lastly, it is important to note that most of the path-planning methods rely on a path-following approach, with velocity regulation in parallel if required, instead of trajectory-tracking ones. The later requires the specification of the desired position for the vehicle, as well as some indication of the time it must move to such position and is more usual in mobile robotics [1], despite still being applicable in autonomous cars [14]. On the other hand, path-following strategies are based on the objective of describing a given course at a given velocity, constant or varying, and frequently consist of a decoupled-approach: one part of the algorithm guides the vehicle to the prescribed path [15, 16], and another regulates its velocity, and latter should account the car's actuation limitations as well as the track, in order not to compromise the following of the course.

The last component, which is the focus of this thesis, is the control of the car: from the prescribed course, the control algorithm will then give input signals to the car which are, accordingly to a decoupled approach, a steering command, aiming to drive the vehicle towards such path, and an acceleration or a braking signal, regulating its speed. The first is frequently referred as "Lateral Control", since it guides the vehicle with the steering command to a different lateral position, and the second as "Velocity Control", providing the car with the control input required to achieve a certain desired speed. Control strategies in general differ mainly in two aspects: the model used for obtaining the controller, and the type of control technique applied. One of the commonly used models for lateral control is the bicycle kinematics model, which represents the vehicle only by a set of two wheels connected by the wheelbase of the vehicle [16]. These simple models are more frequently applied when the velocities are low, and thus they can provide a relatively accurate approximation of the behaviour of the car, or for exploring the limits of such representation, comparing it to dynamic models [17]. This last type of models is more common in high velocity situations, as they account for the dynamics that result from it [18, 19], and they can be a bicycle-type interpretation of the vehicle, simplifying the front and rear wheels of a car to one in each axle [17], or a four-wheeled one, including the influences of all the wheels [19] and even complex steering geometries and tyre forces [18]. In the area of velocity control, the models tend to include dynamics as a rule, since the inertia of the car and the aerodynamic drag forces should always be accounted for, as well as possibly internal resistance forces of the vehicle and road inclination[18].

When it comes to strategies regarding the lateral control, the choice is made by balancing two aspects: on one side, the degree of exactitude required for the vehicle to follow the path; and on the other side, the available computational power and the accessible information from sensors. If low speeds are in order or the car should follow a course without special precision, geometrical path-following strategies like the "Pure Pursuit" [20, 21], proportional controllers [19], or fuzzy logic-based ones [22] frequently satisfy the desired project requirements, without demanding a surplus of processing power or complex sensors. On an intermediate balance between the stated aspects, nonlinear control strategies, which the "Stanley" controller is an example off [15, 16], as well as adaptive linear controllers [23] often achieve good results and are relatively easy to implement. Lastly, there is a requirement for high precision in following the course or large velocities are common as design requirements, Model Predictive Controllers are often the selected strategies, as they enable to account for the vehicle's behaviour in a given time horizon

[6]. Additionally, learning based approaches are increasingly more common, and can be combined with some of the previous referred control strategies to achieve better results. Naturally, these methods have high demands when it comes to processing power but are growing in popularity as the technologies involved in driverless vehicles evolves. In the velocity control area, the problem is frequently simplified by assuming that many of the dynamics of the drivetrain, motors and transmission is regulated by low-level controllers, resulting in a fairly simple system to control, which is done mainly with linear controllers [19] or combined with the lateral control for an appropriate MPC formulation, for example [6].
Regarding the Formula Student teams specifically, the driverless competition is recent and several universities all over the globe have been developing the first models race in such contests. One of such teams is the Academic Motorsports Club Zurich (AMZ [6]), from ETH Zurich (Swiss Federal Institute of Technology in Zurich), and contributions from this team have already been cited multiple times, namely in the MPC and learning based controllers, and will be done as well further in this work. In Portugal, FST Lisboa from Instituto Superior Técnico is currently developing its first driverless vehicle, FST10d, for which this thesis aims to contribute.

## 1.3 Objectives

This thesis consists in designing a path-following method for a Formula Student car in an unknown track, and developing appropriate control strategies that enable the vehicle to effectively follow such a track, taking into account the influence of sideslip in this tracking. With this main goal as a basis, a set of objectives can be drawn from it, and these are summarized now:

- Explore the dynamics of a race-car and design a model that approximates a Formula Student vehicle in the key aspects relevant for the lateral control when tracking a path;
- Develop an algorithm for the computation of path-following errors and curvature of the path, allowing the navigation to compute a course at each time instant, provided it receives information from sensors about the track;
- Design different control strategies that enable the vehicle to steer and accelerate accordingly to the planning, effectively driving autonomously;
- Validate such controllers and guidance methods with simplified models of the car;
- Test, evaluate and compare the different approaches with the model of the Formula Student vehicle on simulations of different real tracks, providing useful information about their respective performances and limitations.

## 1.4 Thesis Outline

This dissertation is divided into seven parts, which correspond to seven chapters, illustrated in the following diagram:

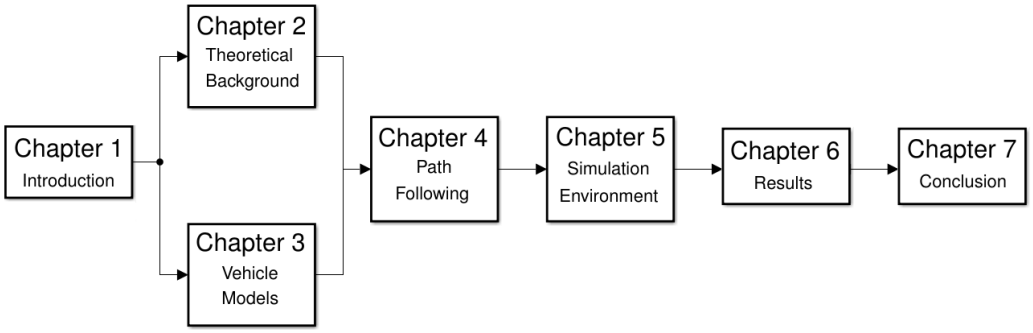


Figure 1.1: Structure of the dissertation, organized by its chapters, from left to right.

Besides the introduction, the key-concepts required for the design of linear control strategies, as well as the application of some nonlinear methods common in path-following problems, are depicted and explained in Chapter 2, which is the Theoretical Background. After this, the simulated car model is described in Chapter 3, as well as some simplified models of such vehicle. In Chapter 4, the theoretical concepts and the models from the two previous chapters are used for designing the controllers for lateral and velocity control of the vehicle, being validated with the simplified models. Then an overview of the simulation scenario is provided on Chapter 5, describing the tracks from real competitions, the metrics used for evaluate the performance of the developed strategies, and explaining the simulation environment. Finally, the multiple controllers and respective variations are used for controlling the model of the real vehicle in Chapter 6, initially at constant speed, and then varying it, evaluating and comparing performances. The conclusions of this dissertation are then present on its last chapter, Chapter 7, as well as some considerations about the achieved accomplishments and notes about work that may follow this dissertation.

# Chapter 2

# Theoretical Background

In this chapter, some key theoretical concepts are defined in order for them to be applied in later sections. As stated in Chapter 1, this thesis focuses on developing, implementing and testing in simulation different control strategies, in order to successfully implement a solution, or a set of solutions, with the objective for a Formula Student vehicle to drive autonomously, following a given path. Naturally, some solutions rely on linear systems theory as their respective bases, and it is crucial to present and explain the linearization process and some of the properties of linear systems as well. Therefore, control and observer design methods are described in section 2.1, with the goal of being applied to the path-following problem at hand. Additionally, two nonlinear strategies are depicted in section 2.2, consisting in chosen approaches applied in different situations of following a reference path. Lastly, in Chapter 4, these controllers are implemented and validated with the vehicle models that will be derived on Chapter 3.

## 2.1 Linear Control Design

A variety of physical processes can be approximated by linear dynamics, from dynamical and thermal systems, to hydraulic and electronic ones, and are a fundamental, if not basilar, part of the theory of Control Systems. These systems have the common property of being modelled through a linear differential equation, or a set of such equations, and the respective solution can be computed. However, some systems, despite not being inherently linear, can be approximated by a linear set of equations, or can be linearized around a given operating point, when certain conditions are met, in order to apply linear control strategies.

Starting from a general system, not knowing about its linearity or time-dependency, when it can be described by a set of equations that relate its $n$ state variables, $x_1, x_2, ..., x_n$, with its $m$ inputs, $u_1, u_2, ..., u_m$, and another set relating the state variables and the inputs with the $p$ outputs, $y_1, y_2, ..., y_p$, then such system can be expressed by the following equations:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}, \mathbf{u}, t) \tag{2.1a}$$

$$\mathbf{y}(t) = \mathbf{g}(\mathbf{x}, \mathbf{u}, t) \tag{2.1b}$$

where $\mathbf{x}$, also called the *state vector*, represents the $n$ state variables, $\mathbf{y}$ is the output vector, $\mathbf{u}$ is the input vector, and $t$ is the time variable. The vector functions $\mathbf{f}$ and $\mathbf{g}$ describe the system according to its inputs and state variables. In general, these can be either linear or nonlinear, and can be dependent of time, $t$. In the particular case of these functions being linear and independent of time, the system they characterize will be called a *linear time-invariant* (LTI) system, and the equations of (2.1) can be rewritten as:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \tag{2.2a}$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \tag{2.2b}$$

In (2.2), $\mathbf{A}$ matrix, also known as the *state matrix*, expresses how the system, namely its states, will evolve depending on the relations between these, and has $n \times n$ dimensions, being a square matrix. Matrix $\mathbf{A}$ also defines the *order* of a system, which will correspond to the length of the state vector $\mathbf{x}$, $n$. Matrix $\mathbf{B}$ represents the inputs and how these influence the system, and it has $n \times m$ dimensions. Similarly, matrix $\mathbf{C}$ will have $p \times n$ dimensions, and it will describe the relationship between the output and the state variables, usually being dependent on sensors available. Lastly, the $\mathbf{D}$ matrix represents direct input to output action, being $m \times p$. In case of Single-Inputs-Single-Output (SISO) models, $\mathbf{B}$ will be a vector of length $n$, $\mathbf{C}$ will be a line vector of length $n$, and $\mathbf{D}$ will be a scalar [24]. In figure 2.1 it can be seen a general state-space representation of a system, illustrating 2.2.

The poles of the system will correspond the eigenvalues of matrix $\mathbf{A}$, and can be obtained from the

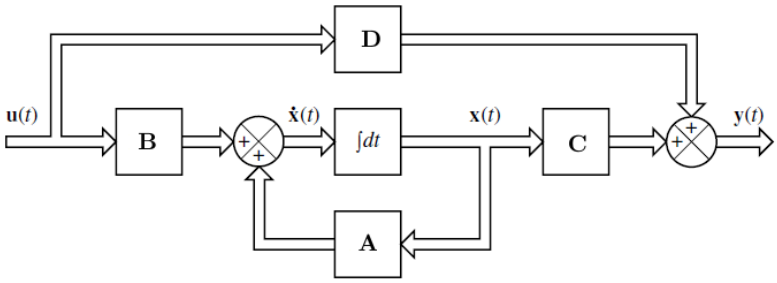


Figure 2.1: Block diagram for state-space representation of a system [24].

determinant of $(s\mathbf{I} - \mathbf{A})$, where $\mathbf{I}$ is the identity matrix of the same order of $\mathbf{A}$. The location of these poles in the complex planes provides crucial information about the behaviour of the system, and other fundamental properties like the stability, which will be explained in this section. Similarly, the zeros of the system are the roots of $(\mathbf{C}\,\mathrm{adj}(s\mathbf{I} - \mathbf{A})\mathbf{B})$, with $\mathrm{adj}(\cdot)$ representing the adjoint matrix.

Control system theory can be further read in [24] and [25], which present classical control theory as well. Specifically for state-space methods, [26] and [27] are excellent references.

### 2.1.1 Linearization

In the case of systems that have no dependence of time, or such independence may be achieved with an adequate approximation, the equations that express them will be somewhat an intermedium between the system described in (2.1) and (2.2), since these shall be nonlinear but not time-variant:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) \tag{2.3a}$$

$$\mathbf{y}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{u}(t)) \tag{2.3b}$$

meaning that both the state equation and the output one will be functions of the state $\mathbf{x}$ and input $\mathbf{u}$. In spite of their invariance regarding time, this set of equations remains nonlinear and, as so, linear control techniques cannot be applied directly as it is. There is a multitude of different nonlinear control methods available with varying complexities, but this section focuses explicitly on the design of linear controllers. With this in mind, a linearization method can be applied to a nonlinear system in order to obtain a model that can be used in the architecture of (2.2). There are different linearization methods available [28], but a local process, also known as a *linearization at an equilibrium point* will be described shortly. Naturally, one must first provide a definition on an equilibrium point: a given pair of state and input vectors $(\mathbf{x}, \mathbf{u})$, with their respective dimensions taken into account, is an equilibrium point of its system if a subset of its derivatives are equal to zero, $\dot{\mathbf{y}}_{\mathbf{eq}} = \mathbf{C}\dot{\mathbf{x}}_{\mathbf{eq}} = 0$, and, in this given point, the following holds [29]:

$$\mathbf{u} = \mathbf{u}_{eq}, \quad \mathbf{x} = \mathbf{x}_{eq}, \quad \mathbf{y} = \mathbf{y}_{eq} = \mathbf{g}(\mathbf{u}_{eq}, \mathbf{x}_{eq}) \tag{2.4}$$

In dynamical systems, such conditions generally represent an equilibrium of forces acting on the system as well, as its derivatives or a subset of these will be null [29]. Once the equilibrium point, or a set of these, has been determined, local linearization can take place. The successive steps differ in complexity depending on the order of the system and number of inputs and outputs it has. For the case of a MIMO system, the linearized matrices required for (2.2) can be obtained from the following expressions at an equilibrium point as stated in (2.4) [28]:

$$\begin{aligned}
\mathbf{A} &= \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathbf{x}=\mathbf{x}_{eq}, \mathbf{u}=\mathbf{u}_{eq}} \\
\mathbf{B} &= \left.\frac{\partial \mathbf{f}}{\partial \mathbf{u}}\right|_{\mathbf{x}=\mathbf{x}_{eq}, \mathbf{u}=\mathbf{u}_{eq}} \\
\mathbf{C} &= \left.\frac{\partial \mathbf{g}}{\partial \mathbf{x}}\right|_{\mathbf{x}=\mathbf{x}_{eq}, \mathbf{u}=\mathbf{u}_{eq}} \\
\mathbf{D} &= \left.\frac{\partial \mathbf{g}}{\partial \mathbf{u}}\right|_{\mathbf{x}=\mathbf{x}_{eq}, \mathbf{u}=\mathbf{u}_{eq}}
\end{aligned} \tag{2.5}$$

Undergoing such a process, the resulting linear system will approximate the original one for small variations of $\mathbf{x}$ and $\mathbf{u}$ around $(\mathbf{u}_{eq}, \mathbf{x}_{eq})$. If such conditions are met, the approximation should allow for the design of appropriate controllers, to be tested using the nonlinear system, or its simulation.

### 2.1.2 Properties of linear systems

Once the system being analysed has been linearized, the properties of its linear equivalent should be evaluated, in order to allow further insight into its behaviour and characteristics. Three different linear systems' properties are of particular relevance, since these will be used throughout this thesis: stability, controllability and observability. Brief definitions of such properties are presented next, as these will be evaluated with the vehicle models presented in Chapter 3.

- Stability:

  There are several definitions for this key concept of control system theory, whether being semantic or mathematical ones, and depending on the system, namely its linearity and time-dependency. The main idea behind some of these definitions is common: for systems with inputs and outputs, a given bounded input will result in a provided bounded output [27]. This is called the *BIBO stability*, representing *bounded in / bounded out*. As stated before, the stability is a vast concept and topic, and it will not be exposed in detail here, and two brief definitions will be provided instead.

  The first one is well-known from its simplicity and easiness to apply: if the poles of a given system have negative real part, then the system is asymptotically stable [24]. However, if at least one of these poles has a positive real part, the system will be unstable. Since the poles are also the eigenvalues of matrix $\mathbf{A}$, then these can be computed from the following determinant:

  $$|s\mathbf{I} - \mathbf{A}| = 0 \tag{2.6}$$

  where $I$ represents the identity matrix of dimensions equal to the order of the system. This definition allows for a sufficient stability analysis in most cases, and it is extendable to a linearized system at an equilibrium point [27]. Lastly, it is should be noted that if none of described pole location cases arise, further study about the system should be accomplished, as it may be marginally stable. This happens when a system has at least one pole at the origin, for example, which is an important observation for the later models.

  Another method that is commonly applied when state-space representations is the *Lyapunov Stability*. This states that a given system with state matrix $\mathbf{A}$ will be asymptotically stable if the following equation is fulfilled [28]:

  $$\mathbf{Q}_{Lyap} = -(\mathbf{A}^T\mathbf{P}_{Lyap} + \mathbf{P}_{Lyap}\mathbf{A}) \tag{2.7}$$

  and $\mathbf{Q}_{Lyap}$, $\mathbf{P}_{Lyap}$ are symmetric positive-definite matrices. The subscript "$Lyap$" has been added to matrices $\mathbf{Q}_{Lyap}$ and $\mathbf{P}_{Lyap}$ as to distinguish them from matrices that present similar nomenclature. (2.7) is called the *Lyapunov Equation*, and it allows for a stability analysis without explicit pole computation. However, if the matrix obtained, whether starting from $\mathbf{P}_{Lyap}$ to compute $\mathbf{Q}_{Lyap}$ or vice-versa, is not symmetric positive-definite, then the system may not be asymptotically stable, and explicit pole-computation may be required.

- Controllability:

  Similarly to the stability concepts, there are different ways to define or express the controllability

of linear systems. In this work, the key concept is explained, and then the condition that allows for the determination of the controllability of a system is shown.

The main idea is that if it is possible to transfer a system expressed by (2.2) from any initial state to any other state in a finite time period with a finite control action, then such system is said to be *controllable*, and its poles can be arbitrarily placed by designing an appropriate feedback law. The algebraic condition for controllability is the following [26]:

$$\text{rank}(\mathcal{C}) = \text{rank}[\mathbf{B} \quad \mathbf{AB} \quad ... \quad \mathbf{A}^{n-1}\mathbf{B}] = n \tag{2.8}$$

where $n$ is the order of the system and the dimension of the state vector, and matrix $\mathcal{C}$ is known as the *controllability matrix*. Such expression allows for almost direct evaluation of the controllability, since it can be easily programmed.

- Observability:

  The last of these properties to be presented is the observability of linear systems. Its definition can be expressed in an analogous way to the controllability: the idea behind this concept is that if the observation over a finite time period of the system output allows for the determination of any initial state, then the system is said to be *observable*. The parallel can be made in the algebraic condition for observability as well, as it is presented next [26]:

$$\text{rank}(\mathcal{O}) = \text{rank}[\mathbf{C} \quad \mathbf{CA} \quad ... \quad \mathbf{CA}^{n-1}]^T = n \tag{2.9}$$

  where $n$ remains as the order of the system and matrix $\mathcal{O}$ is the correspondent *observability matrix*. If a system is determined to be observable, then their states can be successfully estimated, even if not present at the output, through the design of an appropriate observer.

### 2.1.3 Feedback control

Feedback control of a system is the process of designing a feedback law with a predefined goal: it can be to simply to stabilize an unstable system; it can change its general behaviour, reducing the oscillatory response, for example; or even change the fastness of this response. In state feedback, the objective is to feed back not only the output of the system, but some or all its states as well, or respective estimates, in order to be able to redefine arbitrarily the poles of the closed-loop, which output feedback may be incapable. The condition for this is that the pair of matrices $(\mathbf{A}, \mathbf{B})$ must be controllable, through the satisfaction of the condition expressed by (2.8).

In the case of a regulator problem, which has the particularity of having a null reference, $\mathbf{r}(t) = 0$, is the starting point of the state feedback design. A schematic for this problem is presented on figure 2.2.

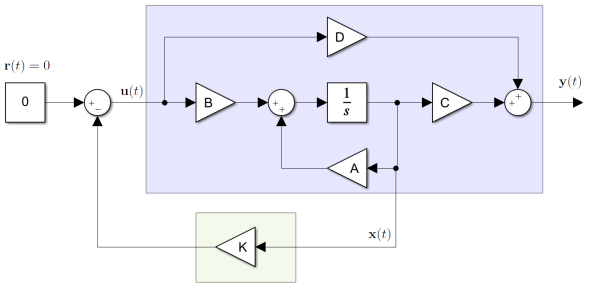


Figure 2.2: Block diagram a regulator problem.

In the above figure, the purple-shaded area represents the LTI system to be controlled, which has an input $\mathbf{u}(t)$ that only depends on the gain matrix $\mathbf{K}$, which is shown on the green-shaded area. The bold subscripts have been suppressed in the matrices of the above figure for simplicity, but these are to be interpreted as vectors and matrices as defined by (2.2).
The feedback law for such problem can then be computed by the following expression:

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t) = -k_1 x_1(t) - k_2 x_2(t) + ... - k_n x_n(t) \tag{2.10}$$

The controller $\mathbf{K}$ is a vector or matrix depending on the states that are being fed back, as well as the structure of the system, and its entries will have corresponding units in order to (2.10) to make sense to the physical aspect of the system. The closed-loop dynamics of the controlled system will then be expressed by [24, 26]:

$$\dot{\mathbf{x}}(t) = (\mathbf{A} - \mathbf{B}\mathbf{K})\mathbf{x}(t) \tag{2.11}$$

The above equation clearly indicates that the poles of the closed-loop are directly dependant on the gain matrix $\mathbf{K}$, and will be the same as the eigenvalues of $(\mathbf{A} - \mathbf{B}\mathbf{K})$.
The natural question that arises, once the above has been stated, is which gain matrix $\mathbf{K}$ should be chosen, and, related to this, how it can be computed knowing the state-space representation of the system. The first and more traditional method is through pole-placement: by determining the desired closed-loop poles, $\mathbf{K}$ can be computed in order to assure that the closed-loop system in (2.11) will have these same poles. However, despite of its intuitive simplicity and fairly easy methods to accomplish it, pole-placement frequently requires extensive knowledge about the system, which is not always readily available or obtained with ease, and can lack several other properties, like actuator requirements. Therefore, a different approach is chosen, having its basis on optimal control theory, which is the *Linear Quadratic Regulator* (LQR).

- Servo Systems:

  Before moving on to the LQR background and mathematical basis, two particular dispositions of feedback control should be shown, as these will have particular relevance throughout this thesis.
  If a given control system has states $\mathbf{x} = [x_1, x_2, x_3]^T$ but its output only has states $\mathbf{y} = [x_1, x_2]^T$, it

means that only the first and second state are able to be compared with the respective reference when they are fed back, assuming there are such references to be followed. In such cases, a tracking system with an outer feedback loop is developed for the states which have indeed a reference available, containing their correspondent gains, $K_1$ and $K_2$, and a inner feedback loop is designed in order to stabilize the third state, with the gain of the third state, $K_3$ [24]. The input to the LTI system with then be:

$$\mathbf{u} = -\begin{bmatrix} K_1 & K_2 \end{bmatrix} \begin{bmatrix} r_1 - x_1 \\ r_2 - x_2 \end{bmatrix} + K_3\, x_3 \tag{2.12}$$

For an adequate visualization of such scheme, figure 2.3 is presented, illustrating this control architecture with a block diagram, in which $\mathbf{r}(t) = [r_1, r_2]^T$ represents the desired values for $x_1$ and $x_2$ respectively, and assumed to be constant. It is important to note that $\mathbf{K}_{ref} = [K_1, K_2]$, as it was presented in the figure as a single matrix in order to simplify the diagram.

This notion of a servo system can be generalized for systems with more states and/or inputs. The

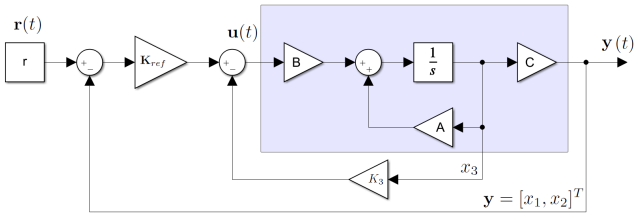


Figure 2.3: Block diagram for a servo system for reference tracking.

concept to be retained is that states, usually outputs, for which there is a reference for comparison can be used to define an *error* such that $\mathbf{e_y} = [\mathbf{y}_{ref} - \mathbf{y}]^T$ which is controlled by the gains in the outer loop, whilst the remaining states are stabilized with the aid of their correspondent gains as well in the inner loop.

The second disposition of a servo system can be obtained if one feeds back the integral of the error $\mathbf{e_y}$ of the state to track. This results in a null steady-state error, although at the cost of diminishing the marginal stability of the controlled system, but such concepts are not used or detailed in this work, and it is suggested to consult the appropriate references for further insight on this notions [26]. The block diagram of a servo system that feeds back the integral of the error is the shown on figure 2.4.

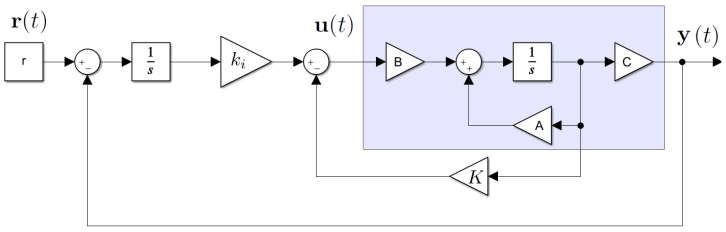


Figure 2.4: Block diagram for a servo system with feedback of the error integral.

A new variable $\mathbf{x_i}$ can then be defined as the integral of the difference between the reference $\mathbf{r(t)}$ and the measured output $\mathbf{y(t)}$, forming an extended state $\mathbf{x}_{aug} = [\mathbf{x}^T, \mathbf{x_i}^T]^T$. This will result in an augmented open loop of the system, described by the two following matrix equations:

$$\begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{x}}_\mathbf{i} \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ -\mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{x_i} \end{bmatrix} + \begin{bmatrix} \mathbf{B} \\ \mathbf{0} \end{bmatrix} \mathbf{u} + \begin{bmatrix} \mathbf{0} \\ \mathbf{1} \end{bmatrix} \mathbf{r} \tag{2.13a}$$

$$\begin{bmatrix} \dot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{x_i} \end{bmatrix} \tag{2.13b}$$

It should be noted that $\mathbf{0}$ and $\mathbf{1}$ respectively represent vectors of zeros or ones, in order to correspond to the dimensions of the remaining matrices and vectors. It can be proven, using the provided definitions, that this system does not present the property of observability, but will remain controllable if the original non-augmented one was as well. Hence, a gain matrix $\mathbf{K}_{aug} = [\mathbf{K}, \mathbf{k_i}]$ can be computed for this extended system applying different methods as one would do in a regulator case, in order to obtain the gains for the inner loop, $\mathbf{K}$, as well as the gain respective to the integral of the tracking error, $\mathbf{k_i}$.

- Linear Quadratic Regulator:

  Returning to the problem of computing the values of the gain matrix $\mathbf{K}$, the LQR approach consists on minimizing a given *quadratic performance index* over a finite period of time, $t \leq t_{max}$, with $t_{max}$ being the finite time horizon. The performance index is then provided by the following integral, where $\tau$ represents the variable of time [26]:

$$J = \int_t^{t_{max}} \mathbf{x}^T(\tau)\mathbf{Q}(\tau)\mathbf{x}(\tau) + \mathbf{u}^T(\tau)\mathbf{R}(\tau)\mathbf{u}(\tau)\, d\tau \tag{2.14}$$

  Matrices $\mathbf{Q}(\tau)$ and $\mathbf{R}(\tau)$ are known respectively as the *state weighting* and *control weighting* matrices, and are important design parameters in Optimal Control. The performance index in (2.14) takes into account both the effort to drive $\mathbf{x}$ to zero from the first part of the integral, as well as the actuator effort from the second one, through the respective weighting matrices. A brief note regarding these matrices will be given shortly.

  The gains that enable the control action $\mathbf{u}$ to minimize the $J$ performance index in (2.14) can be determined solving the *Matrix Differential Riccati Equation*, leading to a time-varying state feedback control law [26]. However, solving a matrix differential equation for each instant can be impractical and will certainly consume elevated computational resources. Therefore, a suboptimal solution can be found since the solution of the Riccati Equation converges to a constant matrix $\mathbf{P}$ as $t$ approaches infinity. Hence, the performance index can be rewritten for $t_{max}$ to be infinite [28, 29]:

$$J_\infty = \int_t^\infty (\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{u}^T\mathbf{R}\mathbf{u})\, d\tau \tag{2.15}$$

In this case, the equation to be solved will be reduced to an algebraic one, also known as the *Algebraic Riccati Equation* (ARE), which has the following form [26]:

$$\mathbf{PA} + \mathbf{A}^T\mathbf{P} - \mathbf{PBR}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{Q} = \mathbf{0} \tag{2.16}$$

The equation is then solved in order to determine matrix $\mathbf{P}$, which is symmetric positive-definite. The resulting gain matrix is then obtained from:

$$\mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} \tag{2.17}$$

Despite not being an optimal solution for every time instant, the resulting input computed with the gains obtained from (2.16) will still minimize the performance index of (2.15) over an infinite time period, and generally consists on a significantly good approximation of the optimal solution, consuming fewer computational resources. When combined with one servo architecture, namely the one expressed by 2.13, the tracking controller can then be designated by Linear-Quadratic-Tracking (LQT) controller).

Regarding the weighting matrices, $\mathbf{Q}$, which is non-negative, symmetrical and often diagonal, represents the importance of the error of its corresponding states, and large values on its entries will lead to a faster convergence to the errors of these states. On the other hand, matrix $\mathbf{R}$, a positive-definite matrix, takes into account the energy required to drive these same errors to zero, and large entries represent smaller control actions. Together, these matrices allow for a fine-tailoring of the response of the system, as well as the energy consumed when driving the errors to zero, and are the design parameters of this control method, allowing the LQR controller to be a powerful tool in control design from its direct balancing of input and output efforts, as well as its effectiveness.

As for the choice of the entries of the $\mathbf{Q}$ and $\mathbf{R}$ matrices, Bryson's Rule usually presents itself as a good starting point for the design process. These matrices are then chosen to be diagonal with their entries being [28]:

$$Q_{ii} = \frac{1}{\text{maximum acceptable value of } x_i^2} \tag{2.18a}$$

$$R_{jj} = \frac{1}{\text{maximum acceptable value of } u_j^2} \tag{2.18b}$$

In the case of $\mathbf{Q}$, if state $x_i$ should not be allowed to have a large value, (2.18) ensures the correspondent entry will be large. On the other hand, if it is wished that a given input $u_j$ should not be high, due to actuator limits for example, then equation (2.18) will lead to a larger value of the respective entry on matrix $\mathbf{R}$, which will in turn lead to a smaller control action.

This method consists on a good starting point, generally leading to satisfying initial results. However, the fine-tuning process usually resorts to trial-and-error, accounting for specificities of the control requirements of a given system.

### 2.1.4 Observer design

The control structures present on figures 2.2, 2.3 and 2.4, with an underlying state feedback law given by (2.10), rely on the assumption that each and every state of vector $\mathbf{x}$ is readily available, whether being as an output or through some form of measuring. But this is not the case to all systems, since some of these states may be impossible or impractical to obtain. In these cases, an *estimator* must be designed with the goal of feeding back these inaccessible states.

The central idea of an estimator, which from now on will be referred to as *state observer*, is to build a simulation of the system in study, in order to provide state estimation $\widehat{\mathbf{x}}(t)$, which is distinguished from the actual state $\mathbf{x}(t)$ with the usage of the hat sign. A block diagram for such structure is presented on figure 2.5, representing the architecture of the *Luenberger observer*, in which matrix $\mathbf{L}$ stands for the *observer gain matrix*, responsible for the correction of the estimated states.

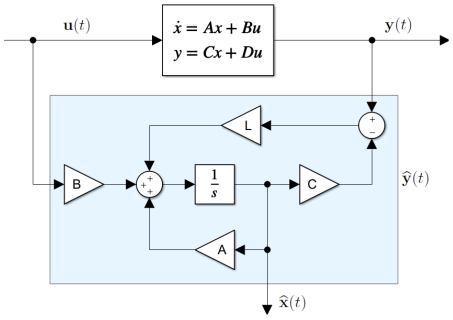


Figure 2.5: Block diagram for a state observer.

If one defines the estimator error as the difference between the actual state and the estimated one, $\mathbf{e}(t) = \mathbf{x}(t) - \widehat{\mathbf{x}}(t)$, it is possible to determine the dynamics of the observer error [24, 26]:

$$\dot{\mathbf{e}}(t) = \dot{\mathbf{x}}(t) - \dot{\widehat{\mathbf{x}}}(t) = (\mathbf{A} - \mathbf{LC})\mathbf{e}(t) \tag{2.19}$$

The above expression clearly evidences a parallelism to (2.11), but being applied to the dynamics of the estimator error instead. In a similar way, the poles of the observer, given by $(\mathbf{A} - \mathbf{LC})$, can be arbitrarily defined if the pair of matrices $(\mathbf{A}, \mathbf{C})$ are observable, as defined in (2.8), and the values of the observer gain matrix $\mathbf{L}$ will be computed accordingly.

The issue of choosing the poles arises in observer design as well, since these may be arbitrarily placed in the complex plane but there is no indication of the most appropriate location for them. As a way to surpass this, with a tremendous advantage of taking into account the noise that might be present in the different parts of the system, the Kalman filter presents itself as a good alternative to manually choosing the pole location, and it is described next.

- Kalman Filter:

  As stated above, the Kalman, or more properly the *Kalman-Bucy*, filter manages to incorporate the noise present in the system. In order to describe such filter, the first step is to rewrite (2.2) as to

include such stochastic processes:

$$\begin{aligned}\dot{\mathbf{x}}(t) &= \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{w}(t)\\ \mathbf{y}(t) &= \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) + \mathbf{v}(t)\end{aligned} \tag{2.20}$$

where $\mathbf{w}(t)$ stands for the noise in the different processes of the system, and $\mathbf{v}(t)$ represents the noise in the measurements provided by the sensors. If both these noise processes are assumed to be uncorrelated regarding each other and to have zero mean with covariances $\mathbf{Q}_0$ and $\mathbf{R}_0$ respectively, where the subscript $0$ has been used to distinguish these matrices from similar ones used before, then the Kalman observer gain matrix can be computed from the solution of a modified Riccati Equation provided in (2.16), as the following states [26]:

$$\mathbf{A}\mathbf{P}_0 + \mathbf{P}_0\mathbf{A}^T - \mathbf{P}_0\mathbf{C}^T\mathbf{R}_0^{-1}\mathbf{C}\mathbf{P}_0 + \mathbf{Q}_0 = \mathbf{0} \tag{2.21}$$

The solution matrix, $\mathbf{P}_0$ is symmetric positive-definite. Hence, the optimal gains for the Kalman filter can be computed resorting to the following equation:

$$\mathbf{L} = \mathbf{P}_0\mathbf{C}^T\mathbf{R}_0^{-1} \tag{2.22}$$

One last note should given regarding matrices $\mathbf{Q}_0$ and $\mathbf{R}_0$: despite having a stochastic meaning associated with the noise present in the system, whether being in the process or in the output, due to the fact that such noise may be difficult or even impossible to measure and evaluate, such matrices may be faced as design parameters, particularly $\mathbf{Q}_0$, since process noises can be caused by internal and, as so, inaccessible, system dynamics, or by dynamics that were not modelled. With this in mind, the fine-tuning of the Kalman filter should be achieved with trial-and-error, in order to obtain the desired state estimations, yet avoiding feeding back unnecessary noise.

### 2.1.5 Separation theorem and LQG control

Together, the state feedback law present in (2.10) and an estimator designed resorting to (2.19) allow for complete state feedback, with the second component providing the estimations of the states that in turn the first uses to control the system. The *Separation Theorem* ensures that the controller gain matrix $\mathbf{K}$ can be designed resorting to state estimates instead of the actual states, and that the observer gain matrix $\mathbf{L}$ can be computed assuming there is no state feedback control. The following matrix equation states illustrates this theorem [26, 28]:

$$\det\begin{bmatrix}\mathbf{A}-\mathbf{B}\mathbf{K} & \mathbf{B}\mathbf{K}\\ \mathbf{0} & \mathbf{A}-\mathbf{L}\mathbf{C}\end{bmatrix} = \det(s\mathbf{I}-\mathbf{A}+\mathbf{B}\mathbf{K})\cdot\det(s\mathbf{I}-\mathbf{A}+\mathbf{L}\mathbf{C}) \tag{2.23}$$

From the direct application of this notion, a Kalman filter can be designed for a given linear system that ensures full state feedback for a LQR controller to use. The ensemble of the optimal state feedback con-

trol and optimal state estimation results in a generalized optimal control technique, and can be achieved computing matrix $\mathbf{K}$ using (2.17), and matrix $\mathbf{L}$ with (2.22).

## 2.2 Nonlinear Control Guidance Strategies

Besides the linear controllers that can be obtained from methods described in the previous section, there are multiple other control techniques that can be helpful in solving the problem at hand. Nonlinear controllers are common both in path-following and trajectory-tracking approaches for developing a method of steering a vehicle autonomously. With such aim, two nonlinear controllers are applied in this work, with some required modifications, in order to adapt them to the path-following objective: the Stanley controller, originally used in off-road driving [30]; and the L1 controller, which is denominated "L1" throughout this dissertation from the original nomenclature on [31], referring to a lookahead line from the vehicle's position to a given reference point, and used for trajectory tracking of unmanned aerial vehicles (UAVs). These two controllers were chosen and implemented in order to test their performance in tracking the path in a race scenario, as well as to provide some basis of comparison with the remaining ones, and are now explained.

### 2.2.1 Stanley controller

As stated before, the Stanley controller consists on a method that was used for guiding a car in off-road scenarios, having obstacles and tortuous terrain. Such controller was implemented on a vehicle to compete on a race called the DARPA Challenge on 2005, a competition over such terrain promoted by the Defense Advanced Research Projects Agency of the United States [32], taking its initials for its name. The car was a Volkswagen Touareg, retrofitted with the necessary equipments for environment perception, namely sensors and cameras, as well as the required computers for the implementation of the controller. The race presents its difficulty not only from the terrain, but also from the length of the course, which was 175 miles (approximately 282 kilometres), and a time limit of 10 hours. The Stanley controller provided excellent results, and won the 2005 DARPA Challenged, and its simplicity and effectiveness were the central reason to choose it as one of the nonlinear controllers to apply to the problem at hand.

The Stanley controller consists on a simple control law, requiring the cross-track error, $e_y$, which is the orthogonal distance from the closest point of the path to the car, as well as the orientation error, $e_\psi$, being the angular difference between the tangent at this same point and the orientation of the vehicle or heading. These concepts, namely the errors and the coordinate systems utilized, are described extensively in Chapter 4, as well as the concrete application of this controller to the problem, on section 4.3, where the lateral control of the car is explored. For now, a brief introduction is provided to this controller, and figure 2.6 illustrates how these errors are defined in relation to the vehicle. It can be seen from [15] that the provided control law for the steering angle results in a globally asymptotically

equilibrium when the cross-track error is null, and the steering is between its usual physical limits.

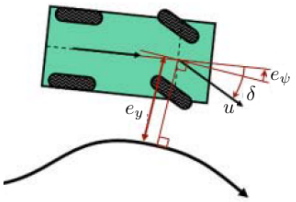


Figure 2.6: Cross-track error $e_y$ and orientation error $e_\psi$ in a path-following vehicle (adapted from [15]), where $u$ represents the longitudinal velocity of the vehicle, and $\delta$ the steering angle

### 2.2.2 L1 controller

The L1 controller is a strategy applied as a guidance logic for UAVs, presenting a simple control law with only one parameter utilized as an error, the angle between the velocity of the vehicle and the lookahead line of constant length from the vehicle to the reference point, denoted by $\eta$. Figure 2.7, adapted from [31], illustrates this scenario, representing an aircraft moving in relation a given path.

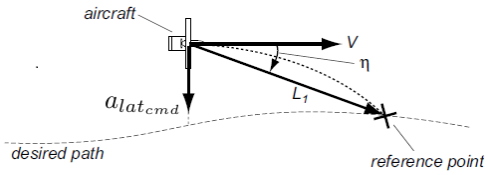


Figure 2.7: Path tracking logic applied for the L1 controller (adapted from [31]), denoting an aircraft travelling at speed $V$, the lookahead line $L_1$ from the vehicle to the reference point of the path, the angle $\eta$ formed by these two vectors, and the acceleration command to drive the aircraft towards the path, $a_{lat_{cmd}}$.

From the velocity of the vehicle $V$, $\eta$ and the length of the lookahead distance $L_1$, which is constant, a command for the lateral acceleration can designed to drive the vehicle to the path, as (2.24) indicates.

$$a_{lat_{cmd}} = 2\frac{V^2}{L_1}\sin(\eta) \qquad [\ \text{m/s}^2\ ] \tag{2.24}$$

Similarly to the previous controller, it can be seen from [31] that this guidance logic results in a asymptotically stable control law, for a distance to the reference point inferior to $L_1$ and $|\eta| < \frac{\pi}{2}$. The application of this logic to a scenario where the vehicle to be controlled is a car is made in section 4.3, providing for a nonlinear control law utilizing a lookahead distance, which represents a significant difference from the Stanley controller in its original version.

# Chapter 3

# Vehicle Models

The design of appropriate models is a crucial step of developing control algorithms for the objective of any control-related work. In the scope of this thesis, such models are related to the multiple ways a car-like vehicle can be represented: they can be purely kinematics, kinematics with the inclusion of certain dynamics effects, and dynamics models, among others, which will naturally have different complexity levels. Another important aspect that should be noted is the scope of a given model, meaning which part of of the vehicle they intend to represent: they can approximate the steering of the vehicle, its lateral and/or longitudinal behaviour.
Additionally to the above explanation, a model can be used to appropriately simulate the behaviour of a vehicle having a higher degree of complexity and interchangeability between its individual subsystems, while other models can be used for controller design, which frequently are simplifications of one or more parts of the complex full system. This approach was adopted, firstly developing a model of the car, or a simulator, that would recreate approximately the vehicle in the aspects that are central in the scope of this work, and then using different simplified models to design the controllers that will be used to control the simulator. Therefore, on section 3.1, the model of the vehicle used in the simulation will be deduced and explained in detail, and the physical constants it relies upon are presented, since these will play an important part of the controller design, most of which were kindly provided by FST Lisboa. Then, on section 3.2, the simplified models used for the lateral control of the vehicle, initially assumed to be travelling at a constant speed, are derived from the kinematics and dynamics of the vehicle. Lastly, a model representing the longitudinal dynamics of the car is presented on section 3.3, in order to include these dynamics into the velocity control of the vehicle, being crucial when the path-following approach is complemented with varying velocity references, which will be explained in detail in Chapter 4.

## 3.1 Formula Student Vehicle

Vehicles in general have complex underlying dynamics due to its multiple subsystems, which translates into intricate relationships between them, and representing them appropriately as well as these relationships is no simple task. Furthermore, in the case of motor-sports scenarios, the difficulty is even greater

since these will involve high velocities and elevated accelerations caused by cornering at such speeds, and certain simplifications are no longer valid, which leads to models even more complex than the ones used for urban driving scenarios, for example. Hence, modelling of race cars is indeed challenging work, in order to be able to represent them in a race scenario. The Formula Student vehicle that will be adapted to function autonomously is the FST10d, which was the one used as a ground work of the model used for simulation. An photography of a previous piloted version, FST09e, is presented on figure 3.1, courtesy of FST Lisboa.

Despite the described complexity of the whole vehicle, when it comes to modelling its behaviour in the

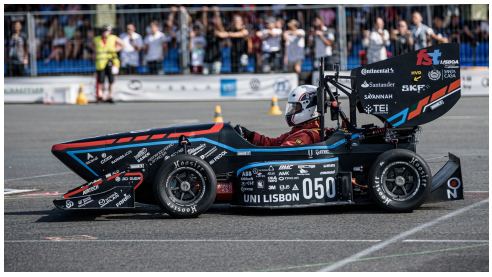


Figure 3.1: Formula Student Vehicle FST 09e.

aspects that are important for path following or path tracking objectives, the influence of many of the smaller subsystems can be approximated as constant, assumed to be controlled independently by the means of a low-level controller, or even neglected, by using a set of certain *assumptions*. When applying this reasoning, a simplified model of the vehicle can be obtained, which still simulates the vehicle with a high level of fidelity in the aspects that are crucial in the scope of the work being developed.

The first assumption to be made in the scope of this thesis is to assume that the vehicle's behaviour when it comes to steering and accelerating can be represented by a planar model, which means that all the vertical influences of the suspension, the pitch and roll angles of the car are neglected. Another vital assumption is to treat the vehicle as a rigid body, which means that the deformation of its components, namely the chassis, is negligible. Based on this conjecture, for a vehicle travelling at a speed $V_{local} = [u, v]$ [m/s], where $u$ is the component of the velocity along its longitudinal axis, aligned with the orientation of the vehicle, and $v$ is the component perpendicular to $u$, the dynamics can be obtained by a balance of forces in the longitudinal and lateral directions, combined with a balance of moments. If the coordinate system of the vehicle has its $x$ axis pointing in front of the car, aligned with $u$, and $y$ to the right as figure 3.2 denotes, these balances are expressed by the following equations [9, 18]:

$$F_x = m\dot{u} - mrv \qquad [\,\mathrm{N}\,] \tag{3.1a}$$

$$F_y = m\dot{v} + mru \qquad [\,\mathrm{N}\,] \tag{3.1b}$$

$$M_z = \dot{r}I_z \qquad [\,\mathrm{N}\cdot\mathrm{m}\,] \tag{3.1c}$$

where $m$ [kg] and $I_z$ [kg· m$^{-2}$] represent the mass of the vehicle and its inertia moment around its $z$ axis, and $r$ represents the rate of change of the yaw angle of the vehicle, which will be denoted by $\psi$ and

referred as the heading of the car.

It can already be realized that (3.1a) will play the central role on the velocity dynamics, and the two remaining equations will have an analogous importance on the lateral dynamics. From this point, the next step is to identify the sum of forces $F_x$ and $Fy$, and moments, $M_z$, acting upon the car. In order to provide a thorough insight on tyre forces, these forces are represented in green in 3.2 as well.

The absolute distance from the $CG$ to the front axis is represented simply by $L_F$ [m], whilst the equiv-

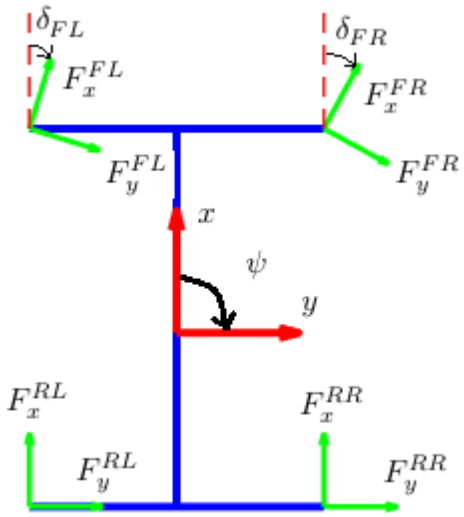


Figure 3.2: Tyre forces acting on the vehicle, in green, and local referential marked by the red vectors, as well as the steering angles, $\delta_{FR}$ and $\delta_{FL}$ [rad].

alent one to the rear axis is denoted by $L_R$ [m], and the distance between wheels of the same axis, or track-width, being the same for both axes, is represented by $L_W$ [m], all being present in figure 3.2. It should be noted that $L = L_R + L_F$ is the wheelbase of the car.

As can be seen from the above image, there will be a total of eight different tyre forces acting on the vehicle, each pair ($F_x^{tyre}$, $F_y^{tyre}$) representing the pair of forces acting on the respective wheel, in Newtons. In terms of notation, $F_x^{RL}$ will stand for the longitudinal tyre force being applied on the *rear-left* wheel, and analogous expressions are used to refer to the longitudinal and lateral tyre forces on the rear-right ($RL$), front-left ($FL$) and front-right ($FR$) wheels. These forces represent the means of interaction between the vehicle and the ground and are largely dependant on the tyre model applied, the steering of the vehicle and the kind of transmission it has. Lastly, one should note that the vertical forces are assumed to be constant due to the assumption that the vehicle can be modelled as planar and horizontal, and the moments due to these forces are neglected.

### 3.1.1 Steering

The internal operation of steering mechanisms depend on the existence of a differential that could induce different angular velocities in wheels of the same axes, as well as the geometry of the car. In order to approximate the steering of an FST vehicle, three different aspects were considered: firstly, a time constant was used to model how the actual steering angle $\delta$ would behave when a reference steering

angle, $\delta_{ref}$, is required; after, an Ackermann geometry [18] was used in order to properly represent the slightly different steering angles of each wheel, $\delta_{FR}$ and $\delta_{FL}$, as the vehicle is only steered by its front wheels; lastly, a saturation was used in both these angles in order to account for the physical limits of the mechanism, in order to make sure that the steering angles of both front wheels are not out of the bounds defined by a given set of limits, $\delta_{min}$ and $\delta_{max}$. The following equation represents the model adopted for the delay present in the actual steering of the car [19]

$$\delta = \frac{1}{s \cdot c_\delta + 1}\delta_{ref} \qquad \text{[ rad ]} \tag{3.2}$$

which is provided in the above form to have a unitary gain transfer function of a time-constant of $0.05$ (s). From this, the steering angles of each wheel, $\delta_{FR}$ and $\delta_{FL}$, can be determined from:

$$\begin{aligned} \delta_{FR} &= \arctan\Big(\frac{(L_F + L_R)\tan\delta}{L_F + L_R - (L_W/2)\tan\delta}\Big) \in [\delta_{min}, \delta_{max}] \qquad \text{[ rad ]} \\ \delta_{FL} &= \arctan\Big(\frac{(L_F + L_R)\tan\delta}{L_F + L_R + (L_W/2)\tan\delta}\Big) \in [\delta_{min}, \delta_{max}] \qquad \text{[ rad ]} \end{aligned} \tag{3.3}$$

As it can be seen from the above expressions, the difference between the steering angles at each wheel will increase with the track width, $L_W$. The expressions (3.2) and (3.3) constitute an important part of the modelling procedure, since they approximate the steering behaviour and its limits, as well as the angular differences in each steered wheel.

### 3.1.2 Tyres

When high velocities are involved, specially in a race scenario where the vehicle is required to turn at such velocities, longitudinal and lateral slip are natural occurring phenomena, and play an important part of how such vehicles behave. At a velocity of $V = [u, v]$ [m/s], the slip angles for each wheel can be computed with aid of the following equations [9, 19]:

$$\alpha_{FR} = \arctan\Big(\frac{v + r \cdot L_F}{u + r \cdot (L_W/2)}\Big) - \delta_{FR} \qquad \text{[ rad ]} \tag{3.4a}$$

$$\alpha_{FL} = \arctan\Big(\frac{v + r \cdot L_F}{u - r \cdot (L_W/2)}\Big) - \delta_{FL} \qquad \text{[ rad ]} \tag{3.4b}$$

$$\alpha_{RR} = \arctan\Big(\frac{v - r \cdot L_R}{u + r \cdot (L_W/2)}\Big) \qquad \text{[ rad ]} \tag{3.4c}$$

$$\alpha_{RL} = \arctan\Big(\frac{v - r \cdot L_R}{u - r \cdot (L_W/2)}\Big) \qquad \text{[ rad ]} \tag{3.4d}$$

One should also note that the sideslip of the car is defined by the following expression:

$$\beta = \arctan\Big(\frac{v}{u}\Big) \qquad \text{[ rad ]} \tag{3.5}$$

Once the slip angles have been defined, the next step is to include them in deriving expressions for the lateral forces acting on the tyres. These depend largely on the tyre composition, as well as the road

conditions and the characteristics of the vehicle, and there are several different tyre models [33, 34], which intend to model the tyre in the multiple aspects of importance for race car modelling. However, such models are intricate and fall out of the scope of this work. Therefore, a simplified Magic Formula [6, 33] was employed for obtaining the lateral forces, which, although being a simplified version, still allows for the lateral slip to be included in the dynamic equations. According to this model, the lateral forces acting on the tyre can be obtained by the following [6]:

$$\begin{aligned} F_y^{FL/FR} &= -D_F \cdot \sin\Big(C_F \cdot \arctan\left(B_F \cdot \alpha_{FL/FR}\right)\Big) \qquad [\text{ N }] \\ F_y^{RL/RR} &= -D_R \cdot \sin\Big(C_R \cdot \arctan\left(B_R \cdot \alpha_{RL/RR}\right)\Big) \qquad [\text{ N }] \end{aligned} \tag{3.6}$$

in which $B_{F/R}$, $C_{F/R}$ and $D_{F/R}$ are tyre constants and have the following meaning [6, 33]: $D_{F/R}$ represents the peak value of the lateral force [N]; $C_{F/R}$ is a shape factor [-]; and $B_{F/R}$ is the stiffness factor of the tyre [-]. One should note that the lateral tyre forces are negative since they are contrary to the vehicle's movement.

The lateral slip $\alpha$ is often complemented with a longitudinal slip $\sigma$, or slip ratio, which intends to model how much the actual velocity of the wheel differs from the one that it would have given its angular velocity, $\omega_{wheel}$ [rad/s], and outer radius, $r_{wheel}$ [m]. However, models including longitudinal slip are complex as well, and involve detailed expressions in order to obtain the actual rotation velocity of the wheel, $\omega_{wheel}$. Thus, taking into account that this thesis focuses mainly on the lateral control of the vehicle in a path-following scenario, it was assumed that there was no longitudinal slip, which meant that all the longitudinal forces generated by the wheel in its rotation are indeed utilized for the propulsion of the vehicle.

### 3.1.3 Propulsion

As it is commonly known, a car owes its movement to the force or torque generated by its one or more motors, which is then channelled throughout the drivetrain to the wheels, that in then propel the vehicle. As one can expect, both engine and transmission models that compose the propulsion of a given car are not plain and require extensive knowledge about the internal workings of such components. However, these can be assumed to be controlled with the aid of a low-level controller and the dynamics of the propulsion can be neglected for the objectives of lateral control of the vehicle. With this in mind, it can be assumed that the velocity is dependent of a given velocity command $d_{cmd}$ provided by the acceleration pedal in case of a pilot scenario or by the velocity control algorithm in the case of a driverless car, considered to be between $-1$ and $1$. Under these assumptions, the force representing the engine and drivetrain of the vehicle can be approximated by the following equation, and assumed to be acting on the $CG$ [6, 8]:

$$F_{motor} = \eta_{motor} \frac{T_{max} \cdot GR}{r_{wheel}} d_{cmd} \qquad [\text{ N }] \tag{3.7}$$

where $\eta_{motor}$ [-] represents the efficiency of the engine and drivetrain, $T_{max}$ [Nm] stands for the maximum torque of the motor, and $GR$ [-] is the gear ratio of the transmission. For this work, it was considered that the motors had no losses, resulting in $\eta_{motor} = 1$, and that only the rear wheels drive the car, resulting:

$$F_{prop} = 2F_{motor} = 2\frac{T_{max} \cdot GR}{r_{wheel}} d_{cmd} \qquad [\text{ N }] \tag{3.8}$$

### 3.1.4 Resistive forces

Before returning to the set of equations on (3.1) in order to derive the the dynamic equations of the vehicle using equations (3.2) to (3.7), two additional forces should be considered: the rolling resistance, $F_{roll}$, and the aerodynamic drag, $F_{aero}$. This forces should be accounted for as otherwise there would be no forces opposing the movement of the car, which is obviously an unrealistic scenario, specially when all the available force of the motors, assumed to have no losses, are considered to be transmitted to the tyres, under the no slip assumption.

Regarding the rolling resistance force, this term can be considered to proportional to the normal force being applied directly by the vehicle on the four tyres, so that [6, 18]:

$$F_{roll} = C_r mg \qquad [\text{ N }] \tag{3.9}$$

$g$ being the gravitational acceleration [m/s$^2$] and $C_r$ a unitless constant. To further simplify this notion, it is assumed that $F_{roll}$ is applied directly on the $CG$ of the model.

Lastly, the aerodynamic drag, $F_{aero}$, neglecting the influence of the wind, is proportional to the square of the speed of the car, as the following equation states [18]:

$$F_{aero} = \frac{1}{2}\rho\, C_d A_F V^2 \qquad [\text{ N }] \tag{3.10}$$

The first constant on the above equation represents the air density [kg/m$^3$], which is considered to be constant, $C_d$ is the aerodynamic drag coefficient, being unitless, and $A_F$ [m$^2$] is the frontal area of the vehicle. Similarly to $F_{roll}$, the aerodynamic drag force $F_{aero}$ is assumed to be acting on the $CG$ of the car.

Having defined all the forces acting on the vehicle as well as the assumptions these require, the final expressions for the dynamic equations of motion can now be deduced.

### 3.1.5 Model of the vehicle

Reporting again to figure 3.2, with the expressions of the different subsystems of the car having been derived, the balances of forces and moments present on (3.1) can now be derived.

Firstly, assuming the vehicle is rear driven only, there will be no longitudinal force in the front tyres: $F_x^{FL} = F_x^{FR} = 0$ [N]. This means that only the rear tyres will exert longitudinal forces on the vehicle, which can then be combined into a single rear force in the $x$ axis, that in turn is equal to the propulsion force. Assuming this force generates no moment in yaw and that all the forces of the motors is being

used for the propulsion [6, 9]:

$$F_x^R = F_x^{RL} + F_x^{RR} = F_{prop} \qquad [\text{ N }] \tag{3.11}$$

With this simplification, the final expression for the balance of forces in the $x$ axis is the following:

$$\dot{u} = \frac{1}{m}\Big(-F_y^{FL}\sin(\delta_{FL}) - F_y^{FR}\sin(\delta_{FR}) + 2\frac{T_{max}GR}{r_{wheel}}d_{cmd} - C_r mg - \frac{1}{2}\rho C_d A_F V^2\Big) + v\cdot r \qquad [\text{ m/s}^2\text{ }] \tag{3.12}$$

The above nonlinear equation is the central one when representing the vehicle's forward motion, and will be the one used when deriving appropriate controllers for the velocity.
Applying an analogous method, the balance of forces in the $y$ axis and moments on the $z$ axis can be derived as well. These are presented together since they are complementary to each other when designing lateral control strategies:

$$\dot{v} = \frac{1}{m}\Big(F_y^{FL}\cos(\delta_{FL}) + F_y^{FR}\cos(\delta_{FR}) + F_y^{RR} + F_y^{RL}\Big) - u\cdot r \qquad [\text{ m/s}^2\text{ }] \tag{3.13a}$$

$$\dot{r} = \frac{1}{m}\Big(L_F(F_y^{FR}\cos(\delta_{FR}) + F_y^{FL}\cos(\delta_{FL})) - L_R(F_y^{RR} + F_y^{RL})\Big) \qquad [\text{ m/s}^2\text{ }] \tag{3.13b}$$

The set of equations (3.12), (3.13a) and (3.13b), which were left with the explicit lateral tyre forces $F_y^{tyre}$ in order to not complicate its display, describe the dynamics of a given vehicle for a set of input actions $[\delta_{ref}, d_{cmd}]$. All the shown expressions that serve as basis for these motion equations rely greatly on the characteristics of the vehicle, and, as so, values for the multiple constants that were used for simulation of a numerical model of the car. Regarding the time constant used for the first steering equation, $c_\delta$, its value was estimated to be 0.05 [s], as to include some time delay. On the other hand, since FST Lisboa did not have an updated moment of inertia around the $z$ axis of its latest model, and a value of 120 was used for FST06e [9], an estimation of $I_z = 80$ [kg·m$^2$] was chosen, which accounted for a reduction of dimensions and mass that the newer cars have, specially for a driverless vehicle, which does not have a pilot contributing for the rotational inertia. The remaining values for all the referred constants of the car were kindly provided by FST Lisboa, and used for the derivation of a numerical model in simulation. Regarding environmental constants as the air density and gravity acceleration, it considered that the first was equal to 1.18 [kg/m$^3$], and $g = 9.807$ [m/s$^2$]. All these values are presented on table 3.1, shown in the end of this section.

- Available sensors and Sampling Time:

  The different control strategies described in Chapter 4 require information, either trough measurement or estimation, about some of the states of the car model, in order to fulfil the objective of controlling the vehicle described on this section. Therefore, it is crucial to provide an insight about how such states can be accessed, with the aim of providing a realistic scenario for simulation.

  The control of the proposed system relies mainly on the detection of cones that define the limits of the race track, which are then used for generation of a path for the car to follow. The usual approaches in such detection is to use computer vision or LIDAR sensors, and often a comple-

mentary combination of both, as described in section 1.2. This is sufficient to permit to the vehicle to navigate in the track outlined by these cones, and this was the scenario built as a basis to this thesis and it was assumed that the computer vision algorithm correctly detects the cones and generates the path to be followed without errors.

When it comes to velocity control, vehicles are commonly equipped with standard sensors to measure the angular speed of the wheels, like a speedometer. Under no longitudinal slip conditions, this angular speed will directly correspond to the longitudinal speed $u$ of the vehicle.

Despite not being necessary for the lateral and velocity control of the car, information about some states regarding a fixed global coordinate system should be included, either as possible redundant or complementary measurements, or as validation tools, which are crucial in experimental scenarios. Hence, it was assumed that the position of the vehicle $P = (X, Y)$ [m] could be accessed with the aid of a GPS system, and its orientation $\psi$ [rad] resorting to an appropriate compass or gyroscope.

In the scope of this thesis, the sampling time for the acting of the path-following and controllers was defined as $T_s = 0.1$ [s], as it corresponds to the frequency of the sensors expected to be used in the real vehicle.

Table 3.1: Table of parameters of the vehicle and environment constants used in simulation.

| Constant | Meaning | Value | SI unit |
|---|---|---|---|
| $c_\delta$ | Time constant of steering | 0.05 | s |
| $\delta_{max}$ | Maximum steering angle (clockwise) | 0.47 | rad |
| $\delta_{min}$ | Maximum steering angle (counter-clockwise) | -0.47 | rad |
| $L$ | Wheelbase | 1.54 | m |
| $L_F$ | Distance of front axis to the $CG$ | 0.832 | m |
| $L_R$ | Distance of rear axis to the $CG$ | 0.708 | m |
| $L_B$ | Track width | 1.20 | m |
| $m$ | Mass | 255 | kg |
| $I_z$ | Moment of Inertia around $z$ | 80 | kg·m$^2$ |
| $B_{F/R}$ | Stiffness factor of front and rear tyres | 10 | - |
| $C_{F/R}$ | Shape factor of front and rear tyres | 1.38 | - |
| $D_{F/R}$ | Peak-value of lateral force for front and rear tyres | 1500 | N |
| $T_{max}$ | Maximum engine torque | 21 | N·m |
| $GR$ | Gear ratio of the transmission | 15.74 | - |
| $r_{wheel}$ | Outer radius of the wheel | 0.23 | m |
| $C_r$ | Rolling resistance coefficient | 0.092 | - |
| $C_d$ | Aerodynamic drag coefficient | 1.2 | - |
| $A_F$ | Frontal area | 1.18 | m$^2$ |
| $\rho$ | Air density | 1.18 | kg/m$^3$ |
| $g$ | Acceeration of gravity | 9.807 | m/s$^2$ |

## 3.2 Models for Lateral Control

As it was shown in the previous section, the equations of motion of a car-like vehicle are complex and highly nonlinear, which makes the deduction of appropriate controllers a task of equal complexity. In the case of lateral control of vehicles, the usual approach consists of simplifying the geometrical composition of the vehicle, as well as approximating or even neglecting many of the dynamic effects present on (3.13a) and (3.13b). Different simplified models were adapted to develop, implement and validate controllers for path following, which, for now, is assumed to be at constant speed, $u = u_{const}$ [m/s]. This is a vital condition to impose as such will be required in order to linearize the obtained models, so that linear control strategies can be applied.

The next subsections deduce and describe three different models with this objective: the bicycle kinematics model, acknowledged for its simplicity, but neglecting dynamics of the vehicle; the bicycle kinematics model with sideslip, which includes simplified lateral slip on the first one; and the bicycle dynamics model, that already has a similar structure to the dynamics of the car, although with the necessary simplifications.

### 3.2.1 Bicycle kinematics model

This model is commonly the first approach in problems of path following or trajectory tracking of a car, since in reduces the it to a single-track vehicle, and neglecting all the dynamic effects due to its movement, as well as any sort of tyre lateral or longitudinal slip. Additional information and description of the usage of local and global coordinate systems is provided on section 4.1, but, for now, it is considered only that the vehicle is moving in a fixed global referential $(X_G, Y_G)$, in metres, at a fixed velocity $V$ [m/s], for which it is assumed only to be along its longitudinal axis, resulting in $V = u$. By moving with velocity $u$ [m/s] and with a given steering angle $\delta$ [rad], the position and heading of a vehicle of wheelbase $L$ will change in the global referential according to the following equations [16]:

$$\dot{X} = u\cos(\psi) \qquad [\text{ m/s }] \tag{3.14a}$$

$$\dot{Y} = u\sin(\psi) \qquad [\text{ m/s }] \tag{3.14b}$$

$$\dot{\psi} = \frac{u}{L}\tan(\delta) \qquad [\text{ rad/s }] \tag{3.14c}$$

From the above equations, with proper integration, the position $P = (X, Y)$ of the vehicle in the global frame $G = (X_G, Y_G)$ can be obtained, as well as the angle its longitudinal axis defines in relation to the $X$ axis, being positive from $X_G$ to $Y_G$ (clockwise), being the orientation or heading of the vehicle, denoted by $\psi$. It should be noted that model presented here uses the centre of the rear axle as the reference point, but different points can be used, namely the centre of the front axis or the geometric centre. As can be seen, it consists on a minimal representation of a car, reducing it to its input actions $[\delta, u]$ and a single parameter $L$. A diagram that allows comprehensive insight for this model is presented on figure

3.3 showing the position of the rear axle, $P(X, Y)$ in the global referential $(X_G, Y_G)$, the heading angle $\psi$, formed between a parallel line to $X_G$ passing throuhg $P$, in red, and the wheelbase of the vehicle, $L$, and the steering angle $\delta$, as the angular difference between the heading and the orientation of the wheels (denoted by the angle between the dashed blue and green lines). Both these angles are taken as positive in the clockwise direction, as referred before.

A brief remark should be given in the reference point used: many of the controllers require errors with

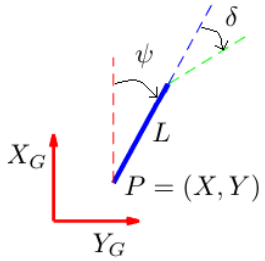


Figure 3.3: Kinematic bicycle model of length $L$ with position $P$ in a global referential $(X_G, Y_G)$, heading $\psi$, and steering angle $\delta$.

respect to the path, as it will be explained on section 4.2, and the approach used for these errors was to obtain them with respect to the centre of gravity of the model being used. As so, the coordinates of the equivalent centre of mass (as this kinematics model does not have mass at all), assuming an equal weight distribution along the vehicle, can be obtained simply by taking into account the distance from the rear axle to the $CG$ of the vehicle, $L_R$, resulting in:

$$\begin{aligned} X_{CG} &= X + \left(\frac{L_R}{L}\right)\cos(\psi) \qquad [\text{ m }] \\ Y_{CG} &= Y + \left(\frac{L_R}{L}\right)\sin(\psi) \qquad [\text{ m }] \end{aligned} \tag{3.15}$$

In order to apply the linear control techniques described in Chapter 2, the model must be linearized at an equilibrium point as (2.4) defines. In the case of this model, assuming it is travelling at constant longitudinal speed, the equilibrium point for linearization is corresponds to $[\delta_{eq}, u_{eq}] = [0, u_{const}]$. For a given state resultant of the equilibrium conditions, $[X_{eq}, Y_{eq}, \psi_{eq}]$, the linearized equations are the following, on the same SI units:

$$\dot{\tilde{X}} = u_{eq} \qquad [\text{ m/s }] \tag{3.16a}$$

$$\dot{\tilde{Y}} = u_{eq}\tilde{\psi} \qquad [\text{ m/s }] \tag{3.16b}$$

$$\dot{\tilde{\psi}} = \frac{u_{eq}}{L}\tilde{\delta} \qquad [\text{ rad/s }] \tag{3.16c}$$

where the equivalent resulting states $\tilde{X} = X - X_{eq}$, $\tilde{Y} = Y - Y_{eq}$ and $\tilde{\psi} = \psi - \psi_{eq}$, as well as the input steering angle, $\tilde{\delta} = \delta - \delta_{eq}$, represent the deviations from the equilibrium conditions. One should note

that analogous results could be obtained if one applies small angle assumptions to the steering angle $\delta$ to (3.14).

Since $\dot{\tilde{X}} = u_{eq}$, which is expected for a null steering angle and constant velocity, the focus will be on the two remaining equations, as these express the lateral behaviour of the kinematic bicycle model. Re-arranging these by the direct application of (2.5), the lateral representation of this model in state-space can be calculated as it follows:

$$\begin{bmatrix} \dot{\tilde{Y}} \\ \dot{\tilde{\psi}} \end{bmatrix} = \begin{bmatrix} 0 & u_{eq} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \tilde{Y} \\ \tilde{\psi} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{u_{eq}}{L} \end{bmatrix} \tilde{\delta} \tag{3.17}$$

Computing the poles of the above state-space model for a positive value of $u_e$, it can be proven that these are both placed at the origin of the complex plane, and the system is marginally stable. On the other hand, the controllability matrix of this system has a rank equal to its order, $\mathrm{rank}(\mathcal{C}) = \mathrm{rank}[\mathbf{B} \quad \mathbf{AB}] = 2$, and thus an appropriate linear controller can be designed for it. Lastly, as explained on the end of section 3.1, the available sensors can provide the path-following algorithm with the reference path in front of the vehicle, for which the lateral and angular deviations from it are computed. As such, $\mathbf{C} = \mathbf{I}_{2\times 2}$, and the observability matrix $\mathcal{O}$ will have a rank equal to the order of the system, which is 2, meaning no state estimators are needed to apply the resultant controller to the real model. Therefore, this state-space model allows for the use of the application of the proposed control techniques described in section 2.1, and can be used for the corresponding nonlinear model, as it will be seen in Chapter 4.

### 3.2.2 Bicycle kinematics model with sideslip

The bicycle kinematics model described on the previous section consists on a valid approximation of how a vehicle will move in a global coordinate system. However, disregarding completely the dynamics will lead to a loss of accuracy that increases with the longitudinal velocity $u$, aggravated in accentuated turns. This also means that the designed controllers will not account for these effects, leading to poor results when it comes to following a proposed path.

In an attempt to include these dynamic consequences, some sideslip may be included on such model, resulting in a hybrid model, that already dictates the existence of a slip velocity $v$ in the frame of the car, perpendicular to $u$. This model was kindly suggested by the supervising professors, and can be deduced from a equilibrium of acting on the lateral direction of the car, namely the force due to centrifugal effects and a single friction force symbolizing the lateral tyre forces at a linear assumption, in a simpler version of (3.13b):

$$m\dot{v} = mu\dot{\psi} + \mu_{lat} mg \arctan\left(\frac{u}{v}\right) \qquad [\,\mathrm{N}\,] \tag{3.18}$$

where $\mu_{lat}$ represents the lateral friction coefficient in the interaction of the tyres and the track, being unitless and equal to one in this work. The differential equation for $v$ can then be obtained, simplifying the above equality, and the new state of the system will then be $[X, Y, \psi]^T$. The remaining three state

equations are similar to (3.14), but include the contributions of $v$ in $\dot{X}$ and $\dot{Y}$. However, the resulting slip was not significant, and an empirical adjustment factor proportional to the sideslip $\beta$ defined on (3.5) was included, as it is presented now:

$$n(\beta) = 1 + |\beta| + 166\beta^2 \qquad [\text{ - }] \tag{3.19}$$

This factor corresponds to a specific vehicle model, illustrative of the yawing sideslip sensitivity. With the inclusion of this corrective term, the final model of the kinematic bicycle model with slip dynamics is represented by the following set of equations:

$$\dot{X} = u\cos(\psi) - v\sin(\psi) \qquad [\text{ m/s }] \tag{3.20a}$$

$$\dot{Y} = u\sin(\psi) + v\cos(\psi) \qquad [\text{ m/s }] \tag{3.20b}$$

$$\dot{\psi} = \frac{u}{L} n(\beta)\tan(\delta) \qquad [\text{ rad/s }] \tag{3.20c}$$

$$\dot{v} = \frac{u^2}{L}\tan(\delta) - \frac{\mu_{lat} g}{u} v \qquad [\text{ m/s}^2\text{ }] \tag{3.20d}$$

Applying the same procedure regarding the linearization as it was used in the kinematic bicycle model, the representation of the linearized equations in (3.20) in a state-space form, containing only the lateral subsystem, is the following:

$$\begin{bmatrix} \dot{\tilde{Y}} \\ \dot{\tilde{\psi}} \\ \dot{\tilde{v}} \end{bmatrix} = \begin{bmatrix} 0 & u_{eq} & 1 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{-\mu_{lat} g}{u_{eq}} \end{bmatrix} \begin{bmatrix} \tilde{Y} \\ \tilde{\psi} \\ \tilde{v} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{u_{eq}}{L} \\ \frac{u_{eq}^2}{L} \end{bmatrix} \tilde{\delta} \tag{3.21}$$

Similarly to the previous model, for a positive value for $u_{eq}$, the system described by (3.21) will have the same two poles at the origin, and an additional stable one, due to the third state. The rank of the controllability matrix will be the equal to the order of the system as well: $\text{rank}(\mathcal{C}) = \text{rank}[\mathbf{B} \quad \mathbf{AB} \quad \mathbf{A^2B}] = 3$, which means that controllers can be designed with the methods provided in chapter 2. However, there is no direct access to the third state of this model, the lateral velocity $v$, and it must be estimated. Since $\text{rank}(\mathcal{O}) = 3$, equal to the order of such system, an estimator can be designed for the objective of applying full state feedback.

### 3.2.3 Bicycle dynamics model

The last model that reports used in designing lateral controllers for the vehicle is the bicycle dynamics, and can be derived either by including full lateral dynamics into the bicycle kinematics model, or through simplification of (3.13a) and (3.13b). The later was the approach used, starting with the collapse of the two front wheels into one, and the same with the rear ones. This results in only one angle $\delta$ for the steering input, instead of $\delta_{FR}$ and $\delta_{FL}$, and the slip angles of (3.4) become [18]:

$$\alpha_F = \arctan\left(\frac{v + rL_F}{u}\right) - \delta \qquad [\text{ rad }] \tag{3.22a}$$

$$\alpha_R = \arctan\left(\frac{v - rL_R}{u}\right) \qquad [\text{ rad }] \tag{3.22b}$$

And the lateral tyre forces are given by the following expressions, in which the previously described nonlinear model for the tyres was applied:

$$\begin{aligned} F_y^F &= -2D_F \sin\Big(C_F \arctan\left(B_F \cdot \alpha_F\right)\Big) \qquad [\text{ N }] \\ F_y^R &= -2D_R \sin\Big(C_R \arctan\left(B_R \cdot \alpha_R\right)\Big) \qquad [\text{ N }] \end{aligned} \tag{3.23}$$

Since this model is intended to model a four-wheeled vehicle, the tyre forces acting on each of the bicycle tyres should be duplicated, in order to simulate the ones that would operate on a full car equivalent. An analogous diagram of forces to 3.2 but adapted to a bicycle configuration is presented on figure 3.4, in order to allow an understanding of the forces acting on the vehicle. The tyre forces are represented in green, and the local referential in centred at the $CG$ of the vehicle, shown in red. Performing a balance

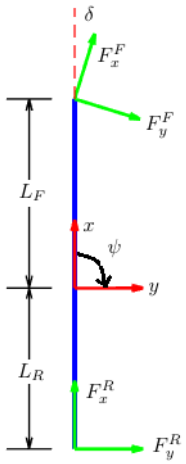


Figure 3.4: Tyre forces acting on the bicycle vehicle, in green, and local referential, in red.

of forces along the local $y$ axis and a balance of moments around the $z$ axis results in the final dynamic equations for the dynamic bicycle model [16, 18]:

$$\dot{v} = \frac{1}{m}\Big(-F_y^F \cos(\delta) - F_y^R\Big) - u \cdot r \qquad [\text{ m/s}^2\text{ }] \tag{3.24a}$$

$$\dot{r} = \frac{1}{I_{zz}}\Big(F_y^F L^F \cos(\delta) - F_y^R L^R\Big) \qquad [\text{ rad/s}^2\text{ }] \tag{3.24b}$$

$$\dot{\psi} = r \qquad [\text{ rad/s }] \tag{3.24c}$$

$$\dot{X} = u\cos(\psi) - v\sin(\psi) \qquad [\text{ m/s }] \tag{3.24d}$$

$$\dot{Y} = u\sin(\psi) + v\cos(\psi) \qquad [\text{ m/s }] \tag{3.24e}$$

in which the forces $F_y^F$ and $F_y^R$ were left in the expressions in order not to extend them. Through time integration, $v$ and $r$ can be computed for the vehicle. However, this does not provide directly with information about how the vehicle moves in a global frame, as opposed to the the previous models, in (3.14) and (3.20). As such, taking into account that $\dot{\psi} = r$ [rad/s$^2$], three additional equations for $\dot{X}$, $\dot{Y}$ and $\dot{\psi}$ were included with the objective to compute the position $P = (X, Y)$ [m] and heading $\psi$ [rad] of the vehicle.

Once again, in order to obtain matrices required for a state-space representation, a linearization must be performed. The same assumptions as before are then taken: the car is travelling at a constant longitudinal speed $u_{eq}$ and the steering angle $\delta$ as well as the slip angles $\alpha_F$ and $\alpha_R$ are assumed to be small. This procedure, complemented by a reorganization of the resulting terms, allows for the attainment of the following linear model [9, 18]:

$$\begin{bmatrix} \dot{\tilde{Y}} \\ \dot{\tilde{\psi}} \\ \dot{\tilde{v}} \\ \dot{\tilde{r}} \end{bmatrix} = \begin{bmatrix} 0 & u_e & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{-2(S_F+S_R)}{u_{eq}m} & \frac{2(L_RS_R-L_FS_F)}{u_{eq}m} - u_{eq} \\ 0 & 0 & \frac{2(L_RS_R-L_FS_F)}{u_{eq}I_{zz}} & \frac{-2(L_R)^2S_R-2(L_F)^2S_F}{u_{eq}I_{zz}} \end{bmatrix} \begin{bmatrix} \tilde{Y} \\ \tilde{\psi} \\ \tilde{v} \\ \tilde{r} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{2S_F}{m} \\ \frac{2S_FL_F}{I_{zz}} \end{bmatrix} \tilde{\delta} \tag{3.25}$$

An alternative representation was used for the tyre constants using $S_F = B_FC_FD_F$, $S_R = B_RC_RD_R$, to simplify the display of (3.25). Regarding the stability of the system, the first two states, $\tilde{Y}$ and $\tilde{\psi}$, will correspond to poles at the origin of the complex planes, as it happened with the previous bicycle models, but states $\dot{\tilde{v}}$ and $\dot{\tilde{r}}$ will introduce real-negative poles. Therefore, the system remains marginally stable, taking into account what was explained on section 2.1. The rank of the correspondent controllability matrix is equal to four, which is the order of the system, and thus linear control strategies can be applied. Similarly, the observability matrix will have rank $\mathrm{rank}(\mathcal{O}) = 4$, and an appropriate observer can be designed to estimate states $v$ and $r$, which were assumed as inaccessible, as explained before.

Lastly, as can be concluded from (3.24) and (3.25), the dynamic bicycle model represents a significant step in complexity regarding the previous kinematic or kinematic-based models, and already resemble the final equations obtained for the realistic vehicle for simulation in (3.13a) and (3.13b).

## 3.3 Models for Velocity Control

The models required to design velocity controllers are not simple, since, as stated before, these often include motor, drivetrain and tyre forces. However, (3.12) already represents many of the dynamics that act in the direction of motion of the car. As such, this will be the starting point for deriving a model to be used in velocity control, with the aid of the referred assumptions, naturally.

If one applies the equilibrium conditions of the steering input, $\delta_{eq} = 0$, to the mentioned expression, the lateral front tyre forces, $F_y^R$ and $F_y^L$, will no longer have influence in the vehicle's longitudinal acceleration, $\dot{u}$. On the other hand, the rolling resistance $F_{roll}$ can be treated as a disturbance to the system, since it does not depend on the states or inputs of the system. Additionally, assuming that there is no

lateral velocity, $V = [u, 0]$ since the vehicle is only being driven longitudinally, the following equality holds [6, 18]:

$$\dot{u} = \frac{1}{m}\Big(2\frac{T_{max}GR}{r_{wheel}}d_{cmd} - \frac{1}{2}\rho_{air}C_d A_F u^2\Big) \qquad [\text{ m/s }] \tag{3.26}$$

which is a first order nonlinear differential equation with $u$ as a variable, and the driving command $d_{cmd} \in [-1, 1]$ as an input. Additionally, if the velocity $u$ is also constant, so that $u = u_{eq}$, a proper linearization of the longitudinal dynamics can be made:

$$\dot{\tilde{u}} = \left(\frac{-\rho_{air}A_F C_r u_{eq}}{m}\right)\tilde{u} + \left(2\frac{T_{max}GR}{r_{wheel}}\right)\tilde{d}_{cmd} \qquad [\text{ m/s }] \tag{3.27}$$

It can be seen that this model will consist on a balance of two factors: the propelling part, proportional to $\tilde{d}_{cmd}$; and a resisting one, caused by the aerodynamic drag, $F_{aero}$. It was also considered a limitation on the longitudinal velocity for control purposes, as the FST electric car, FST09e, has an approximate maximum speed of 110 km/h. This should be taken into account, namely for the normalization required for $d_{cmd}$, but driving the vehicle at its maximum velocity may not be the advisable, specially at an initial design phase. Therefore, it was considered that $u_{max} = 25$ [m/s], accounting for a margin regarding the vehicle's actual maximum velocity.

This system corresponds is asymptotically stable for $u > 0$ as its only pole will be real and negative, and it can also be deduced that the controllability matrix of this system is equal to its order, given positive $u$. Despite its simplicity, this model allows for the design of controllers that allow for the regulation of the velocity in a path-tracking scenario, as it will be seen on the next chapter.

# Chapter 4

# Path Following

Throughout this dissertation and the development of the algorithm that could enable a Formula Student vehicle to drive autonomously, or be driverless, the approach was made using a path following logic, instead of a trajectory tracking one. This means that the objective relied on a decoupled approach, developing first a method that allowed the vehicle to follow a given set of waypoints of a given path, regardless of the velocity the vehicle would describe this same path, which was kept constant at an initial stage. Later, the regulation of the vehicle's velocity would be achieved with the aid of the curvature of the path, as it represents a key factor for lateral control when the dynamics of the car are at work. Therefore, the first key step is to derive a way that the vehicle's guidance mechanism can know its position regarding the track, or in relation to its centre, which are called *path following errors*. These are defined as *errors* since they consist on a comparison between some particular disposition of the vehicle, like its orientation or position, and how this same disposition should be, according to the track. With this in mind, such errors in a path-following problem are fundamentally of two types: the distance errors, like the cross-track error, which measures the orthogonal distance from the position of the vehicle to the closest point to it in the path, referred to as *reference point* (RP); and angular errors, which the orientation error is an example of, that compare the tangent of the path at this same reference point and compares it with the orientation of the vehicle, or with the direction it is moving along, using the vectorial velocity of the vehicle. The expressions for such errors will be derived in section 4.2 in order to explain how they were used in the implementation stage. However, since these errors are expressed in the frame of the car, as if a driver would use them to effectively steer the car, the coordinate systems for both the global and local coordinate systems should be defined, which is done next.

## 4.1 Global and Local Coordinate Systems

In automotive systems, and in control of vehicles in general, the direction of the local $x$ axis of the vehicle is usually placed along the direction it travels, aligned with its longitudinal velocity $u$, and the $y$ axis is placed going to the right of the vehicle, perpendicular to $x$. Additionally, the orientation of the car $\psi$, despite not having a direct meaning in the its own coordinate system, is measured as positive when

going from the $x$ to the $y$ axis. The global coordinate system can be extracted from the local one: the $X$ axis will be parallel to the local $x$ axis when the vehicle has a null heading angle ($\psi = 0$), and the $Y$ axis will going to be orthogonal to $X$, with its positive direction to the right. Lastly, the orientation of the vehicle $\psi$ in this global frame will be taken as positive when in a clockwise direction, going from $X$ to $Y$. This particular version of a global coordinate system is known as the NED (*"North-East-Down"*) referential, with the *North* direction being the front, which is the $X$ axis, the *East* direction being to the right, which is the $Y$ axis, and the *Down* direction is placed in order to complete a right-handed frame. It is important now to state that capital letters will be used to describe coordinates in the global frame, which is assumed to be fixed, and lowercase will be used regarding the local frame, as were in the previous chapters. In order to fully understand these coordinate systems, a figure 4.1 contains the described elements related to local and global coordinate systems.

As can be seen the referred figure, the car is in a given pose regarding the global referential with its

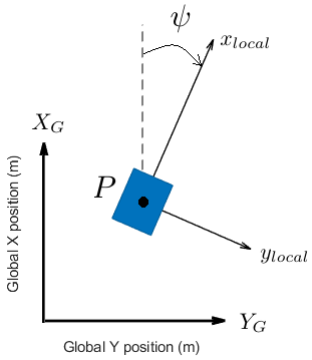


Figure 4.1: Local and global coordinate systems.

position given by $P = (X, Y)$, assumed to be its centre of gravity ($CG$), which represents positive values for both axes, and a positive-valued heading $\psi$, measured between a parallel line to the $X$ axis of the global coordinate system passing through $P$ and the direction it is facing, $x_{local}$.

The last important aspect that one should note is that these two coordinate systems have a certain equivalence between them: if the coordinates of a given point in the vehicle's frame are known, then these can be expressed in the global referential as well, requiring only the knowledge of the position and orientation in the global frame. For example, for a given point in the local frame given by $p_{point} = (x_p, y_p)$, its position in the global coordinate system, $P_{point} = (X_p, Y_p)$ [m], can be obtained if one knows the position and orientation of the vehicle, $P$ and $\psi$, respectively, in the global frame. The following equation illustrates this equivalence [35]:

$$P_{point} = P + R_\psi \cdot p_{point} \qquad [\text{ m }] \quad \text{with} \quad R_\psi = \begin{bmatrix} \cos(\psi) & -\sin(\psi) \\ \sin(\psi) & \cos(\psi) \end{bmatrix} \tag{4.1}$$

The matrix $R_\psi$ represents a rotation around the global $Z$ axis, defined by the heading angle $\psi$ of the vehicle, in radians. Inverting (4.1) and solving for $p_{point}$, the coordinates of the same point on the local frame can be computed as well.

These are quite basic concepts of frame transformation, which are usually quite more complex than what

is exposed and utilized in this work. However, they are still essential tools in order to compute positions of points on different frames, in order to simulate the detection of cones, or to evaluate performance of different path-tracking algorithms, simulating a GPS system acting on a global frame.

## 4.2 Path Following Errors and Curvature Computation

As stated in the beginning of this chapter, the groundwork of most path following mechanisms is the error computation, since it is this fundamental element that will enable the vehicle to know how its disposition is in relation to the track, or regarding a certain point of it. In turn, this will allow for the car, or its control algorithm, to correct such errors together with appropriate controllers, which ultimately enable the driverless vehicle to travel the provided course.
In this section, mathematical definitions of these path-following errors will be provided, explaining and suggesting how they can be computed in various situations, namely different geometries of the track. These will then be used in next section, 4.3, for the design of controllers for the correction of the car's lateral position regarding the track, which will be done at constant speed. Lastly, a brief explanation will be given regarding curvature computation of a given track or small portion of it, since the velocity control described in 4.4 takes this as an input in order to provide the velocity references for the longitudinal speed of the vehicle.

#### Errors for line segments

The first case for path-tracking is to consider that the car must go through multiple successive waypoints, which have certain distances between them, and assuming these are connected by straight lines. This is a quite basic approach, and it causes some problems when applied by itself, as there is no continuity of the first derivative of the path since successive points frequently will not be aligned with each other, leading to "jumps" in such errors when changing from one waypoint to another. However, it is a decent first step as it allows successful navigation, and can still be used as a complement to other approaches when several points are aligned, resulting in a line segment.
In this case, as well as in the future images presented for the different error definitions, it is assumed that the vehicle is travelling at speed $V = [u, v]$, where $u$ represents its $V$ component along its longitudinal axis, and $v$ the one perpendicular to $u$, as depicted by figure 4.1. For simplicity, it can be thought that $V = u$, meaning the heading of the car $\psi$ and its velocity vector are aligned. This is not true for all facts since having a $v$ component will result in a slip or lateral velocity, but can facilitate the understanding of the concepts exposed here.
With the aim of illustrating these errors, an appropriately diagram is shown on figure 4.2, where the reference track is assumed to be a line segment. The reference segment is denoted by the straight line in blue, going from $W_1$ to $W_2$, and $P$ stands for the position of the centre of gravity of the vehicle, which is travelling at velocity $V$, represented by the black vector in the image. The waypoints $W_1$ and $W_2$ in the global coordinate system define a displacement vector, $D_{W_1,W_2}$, denoted $D_{1,2}$ from now on

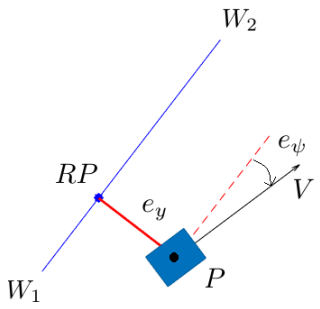


Figure 4.2: Cross-track and orientation errors for a line segment reference.

for simplicity of notation, and an additional vector going from $W_1$ to the vehicle's position $P$ can also be defined, represented by $D_{1,P}$. With these two, the cross-track error $e_y$ and orientation error $e_\psi$ are expressed by the following equations [1]:

$$e_y = -\frac{D_{1,2} \times D_{1,P}}{\| D_{1,2} \|} \qquad [\text{ m }] \tag{4.2a}$$

$$e_\psi = \arcsin\left(\frac{V \times D_{1,2}}{\| V \| \cdot \| D_{1,2} \|}\right) \qquad [\text{ rad }] \tag{4.2b}$$

in which "$|| \cdot ||$" represents the Euclidean norm of the respective vector, and "$\times$" denotes the cross-product between two vectors. These errors are defined in relation to the reference point of the segment, $RP$, which in this case is defined as the closest point to $P$ of $D_{1,2}$. The cross-track error $e_y$ will then be equal to the shortest distance from $P$ to the reference segment, represented above by the red vector, and the orientation error $e_\psi$ will be the angle between the velocity vector $V$ and a line passing through $P$ and parallel to $D_{1,2}$, denoted by the red dashed line.

Some brief notes should be given regarding the above expressions: these are both based on vectorial cross-products, and, as such, the resulting errors will also be vectors of same dimensions as the vectors that originate them, which is three. For two vectors in the $X$-$Y$ plane in a three-dimensional space, having null $Z$ coordinate, the only non-null component of the cross-product between will be the $Z$ component. Therefore, in both equations of (4.2), only the third component is being considered for the error computation. Additionally, the minus sign added in $e_\psi$ is to ensure that the cross-track error is positive when the vehicle is on the right-side of the line-segment, as the example in figure 4.2. The orientation error is then defined with the velocity $V$ instead of the heading $\psi$ as it will be an advantage when these are not aligned, and the vehicle is moving on the global frame with a non-null lateral velocity $v$. In this case, the orientation error will then be computed in relation to its movement defined by $V = [u, v]$ instead of its orientation $\psi$, which will help to minimize this error.

Lastly, one should point out that the definitions present on (4.2) can also be applied when the coordinates of points $W_1$ and $W_2$ of the line are expressed in their local frame equivalents, $w_1$ and $w_2$, respectively, which is important when the path-following is being made in the vehicle's perspective, and the coordinates of the multiple reference points of the track are not known in the global frame.

### Errors for arcs of circumference

The second situation where path-following errors can be defined is when it is known that two waypoints, once again exemplified as $W_1$ and $W_2$, are part of a circumference of radius $R$ and centre $C = [X_C, Y_C]$ in the global frame, which are both assumed to be known. Such scenario is illustrated by figure 4.3.

The cross-track error will be once again defined as the distance of $P$ to the closest point of the reference

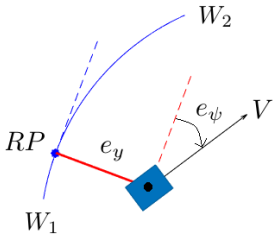


Figure 4.3: Cross-track and orientation errors for an arc of circumference reference.

path, $(\overset{\frown}{W_1 W_2})$, which is now an arc. As the vehicle navigates close to the arc, the reference point $RP$ will change, in order to being able to remain the closest point to $P$. Naturally, the tangent of the circumference at the $RP$ will also vary, marked as the dashed-blue line, and the orientation error is calculated with this in mind. The crosstrack error $e_y$ can then be obtained by subtracting $R$ to the distance of $P$ to the centre of the circumference, $C$, defined by $\overline{PC}$. On the other hand, the orientation error $e_\psi$ is computed from an analogous expression to 4.2, but using the orthogonal vector to $PC$, $PC_\perp$, as the tangent to the path. However, as can be seen from figure 4.3, $W_1$ and $W_2$ can be part of two different circumferences for a fixed $R$: the first being when the centre is at the right of the arc, when navigating the arc from $W_1$ to $W_2$, which is the case being shown; and the second being when the centre is in the left-side of the arc. This will undoubtedly influence the errors that are obtained, therefore impacting the driving of the vehicle. To avoid such a question, a sign can be attributed to $R$, which is positive when the centre is placed on the right of the segment, from the perspective of $W_1$ to $W_2$, and negative when $C$ is placed on the left. These two situations are appropriately shown in figure 4.4.

If $C$ is assumed to be known, the sign of $R$ is indifferent, as only one circumference will pass through


Figure 4.4: Reference arcs with positive $R$ on the left, and negative $R$, on the right, for the same two waypoints, $W_1$ and $W_2$.

both points for that radius. However, if only $R$ is known, which could be obtained from a curvature analysis (explained at the end of this section), then the attribution of a sign to $R$ will uniquely define circumference for a set of two points, as $C$ will be fixed and unique as well, and can be computed

knowing $R$ and the points that are part of that circumference and define the reference arc. Therefore, the errors can be calculated from:

$$e_y = \text{sign}(R) \cdot (\overline{PC} - |R|) \qquad [\text{ m }] \tag{4.3a}$$

$$e_\psi = -\text{sign}(R) \cdot \arcsin\left(\frac{V \times PC_\perp}{\| V \| \cdot \| PC_\perp \|}\right) \qquad [\text{ rad }] \tag{4.3b}$$

In this work, the orthogonal vector to $PC$ was defined as $PC_\perp = [(PC)_Y, -(PC)_X, 0]$, where $(PC)_X$ and $(PC)_Y$ are the coordinates of the original vector going from $P$ to $C$.
Similarly to the line segment version of these errors, the equations are valid if $P_1$ and $P_2$ are expressed in the local frame, and $C$ will be computed in this frame as well.

#### Errors with a polynomial function

In the scenarios described before, it was assumed that the reference path was either a line segment or an arc of circumference, with known radius in case of the later. However, it is difficult to find real circumstances in which these are the exact dispositions of the track in all its portions, specially if the path-following is being accomplished in the point of view of the car, without prior information about the track and the radii of its arcs. Therefore, a different method can be utilized that avoids the disadvantages of both previous approaches, particularly the discontinuity of the first derivative of the path in the case of only line segments, and the requirement of knowledge about the radii of different parts of the track that can be decomposed into circumference arcs. Such method consists on assuming that $n$ waypoints of the track, $w_1, ..., w_n$, expressed in the car's frame for convenience, can be interpolated by a certain function, then computing the reference point $RP_{local}$ with the necessary mathematical tools, and ultimately using this point in the frame of the vehicle for error computation.
In this work, such function was chosen to be a second-order polynomial one, as it requires a minimum of three points of the path to define it uniquely, but several more can be utilized in order to generate it, allowing it to approximately model the path. Figure 4.5 presents a second-order polynomial $f_{pol}$, resultant of the interpolation of the $n$ points [36] being used, $w_1, ..., w_n$, which are provided in the vehicle's local frame.
This logic succinctly consists on the idea that the $x$ and $y$ coordinates of points $w_1, ..., w_n$, in the frame of

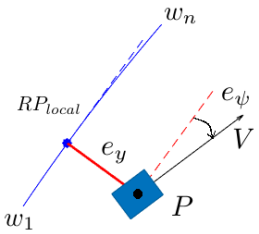


Figure 4.5: Cross-track and orientation errors using a polynomial function as reference.

the car are related to each other by the means of the polynomial function $f_{pol}$, and the position $P$ of the

car in the local frame will correspond to the origin of such coordinate system. Hence, the reference point in the local frame, $RP_{local}$ will be the intersection of the line perpendicular to $f_{pol}$ that passes through the origin. If $f_{pol}(x) = ax^2 + bx + c$ is used as the function interpolating the $n$ points of the path being considered, then the reference point will be the solution of the third order polynomial equation:

$$c_3x^3 + c_2x^2 + c_1x + c_0 = 0 \tag{4.4}$$

where $c_3$, $c_2$, $c_1$ and $c_0$ are the coefficients of this new polynomial equation, and can be determined from the $a$, $b$ and $c$ coefficients of $f_{pol}(x)$, as the following illustrates:

$$c_3 = 2a^2 \tag{4.5a}$$

$$c_2 = 3ab \tag{4.5b}$$

$$c_1 = 2ac + b^2 + 1 \tag{4.5c}$$

$$c_0 = bc \tag{4.5d}$$

However, since (4.4) results of the intersection of a quadratic and a linear functions, it will have three distinct solutions. Therefore, a procedure to determine the solution that will correspond to the $x$ coordinate of $RP_{local}$ among all the solutions is also necessary. The first step is to discard the ones that have an imaginary component, as such has no meaning in the problem at hand. If more than one solution remains, the absolute distance to each should be computed, and the one that minimizes it will correspond to the reference point, denoted by $x_{sol}$. This will allow for the attainment of the definitive reference point, since $RP_{local} = (x_{sol}, f_{pol}(x_{sol}))$.

Once the reference point has been successfully calculated, both the cross-track and orientation errors can be computed in order to match what figure 4.5 illustrates. The derivative of $f_{pol}$ at $x_{sol}$ can also be determined by $f'_{pol}(x_{sol}) = 2ax_{sol} + b$, which can be used to obtain a tangent vector to $f_{pol}$ at this point, denoted by $T_{sol}$, and marked by the blue-dashed line in the figure. Since $T_{sol}$ will be perpendicular to the vector going to the origin to $RP_{local}$, analogous definitions to 4.2 can be used to compute $e_y$ and $e_\psi$:

$$e_y = \frac{T_{sol} \times RP_{local}}{\| T_{sol} \|} \qquad \text{[ m ]} \tag{4.6a}$$

$$e_\psi = \arcsin\left(\frac{V \times T_{sol}}{\| V \| \cdot \| T_{sol} \|}\right) \qquad \text{[ rad ]} \tag{4.6b}$$

One should note that the first $RP_{local}$ represents the vector going from the vehicle to the reference point, but, since this is expressed on the car's frame, such vector will have its components equal to the coordinates of $RP_{local}$ in this referential.

This method has some more complexity to it than the ones before, but it allows for the path-following errors to be obtained relatively to a course of continuous first derivative. Additionally, the interpolation can be made "*on-the-go*", obtaining a polynomial to be used as a reference path as the vehicle moves along several waypoints, discarding the ones behind it and using new ones in front.

**Errors with a polynomial function and a lookahead distance**

In most of the situations that required vehicles to follow, the usage of a lookahead distance may present itself as fruitful option, as it allows for a timely correction of the errors, and enabling an overall more accurate path-following due to the anticipation capability [20, 21]. When dealing with high-speed scenarios, this option can be even more beneficial for appropriate tracking, both for error computation and for regulation of the vehicle's velocity, as it will be done in this thesis.

Lookahead distance logic can be applied to line segments, circumference arcs, among others. In this thesis, due to the advantages of using a second-order polynomial to interpolate the waypoints of a course, the usage of a lookahead is applied to this case, in order to anticipate the path and compute the errors relatively to it, as well as the curvature, which will be explained in detail after this part.

Similarly to the case of a polynomial-generated path without a lookahead, the objective is to find the reference point in the vehicle's frame. However, instead of using a line passing through the origin of such frame as described before, a circumference of radius equal to the lookahead distance and centred on origin the is to be considered, $C^L$, where naturally the superscript $L$ denotes *lookahead*, and is applied to the following terms in order to avoid confusion regarding the previously presented ones. With this in mind, if $L_A$ is the length of the lookahead distance, the reference point $RP^L_{local}$ is a root of the following polynomial:

$$c_4^L x^4 + c_3^L x^3 + c_2^L x^2 + c_1^L x + c_0^L = 0 \tag{4.7}$$

where $c_4^L$, $c_3^L$, $c_2^L$, $c_1^L$ and $c_0^L$ are the coefficients of the fourth-order polynomial that results of the intersection of the circumference $C^L$ and $f_{pol}(x)$. These can be obtained from the following expressions:

$$c_4^L = a^2 \tag{4.8a}$$

$$c_3^L = 2ab \tag{4.8b}$$

$$c_2^L = 2ac + b^2 + 1 \tag{4.8c}$$

$$c_1^L = 2bc \tag{4.8d}$$

$$c_0^L = c^2 - L_A^2 \tag{4.8e}$$

Being a fourth-order polynomial, it will have four distinct roots, and once again a method to find the solution $x^L_{sol}$ that corresponds to the reference point with the lookahead distance, $RP^L_{local}$, must be found. Discarding the non-real roots, the most common scenario is to have two solutions, one being in front of the car, and one behind it. Since the lookahead is introduced in order to provide anticipation capabilities, the solution behind it can be excluded as well, and $x^L_{sol}$ obtained. An additional safeguard should be made for the case when there are no solutions at all, since this will mean that the car is further away from the path than the lookahead distance. In this situation, a safety method should be implemented in order for the algorithm to be able to still track the path, namely guiding the vehicle back to the path using a line segment scenario, present in (4.2). Such situation should not arise as it will mean the controller

and path-tracking algorithm are not following the track effectively, or that the imposed velocity impedes this, but it is still useful to provide a guarantee mechanism.

The usage of a lookahead of length $L$ is illustrated on 4.6, in light blue, which originates the reference point $RP^L_{local}$ on a second-order polynomial resulting of the interpolation of $w_1, ..., w_n$ points, depicted in blue.

Similarly to the previous cases, the vector tangent at the reference point, $T^L_{sol}$, can be obtained from

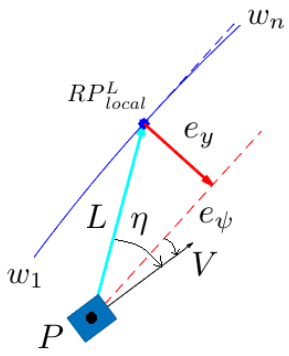


Figure 4.6: Cross-track and orientation errors for a polynomial function with a lookahead distance.

the derivative of the original polynomial $f_{pol}$. The cross-track error $e_y$ will be the distance between this vector and the line parallel to it passing through the origin, and the orientation error $e_\psi$ will once again be defined by the angle between $T^L_{sol}$ and the velocity of the vehicle $V$, expressed in its frame. Hence, the expressions are the same that those used for the case where no lookahead distance was used, with the appropriate changes in notation, as presented next:

$$e_y = \frac{T^L_{sol} \times RP^L_{local}}{\| T^L_{sol} \|} \qquad [\text{ m }] \tag{4.9a}$$

$$e_\psi = \arcsin\left(\frac{V \times T^L_{sol}}{\| V \| \cdot \| T^L_{sol} \|}\right) \qquad [\text{ rad }] \tag{4.9b}$$

An additional error parameter can also be considered, since it will be used in one of the controllers on the next section. It consists on the angle between the vectorial lookahead line and the velocity vector, denoted by $\eta$, as presented on figure 4.6. The lookahead vector can be obtained once the reference point as been established, as its elements will be equal to the coordinates of $RP^L_{local}$ in the local frame, and $\eta$ can be computed as it follows:

$$\eta = \arcsin\left(\frac{V \times RP^L_{local}}{\| V \| \cdot \| RP^L_{local} \|}\right) \qquad [\text{ rad }] \tag{4.10}$$

The method of obtaining reference points ahead of the car using a lookahead distance will be applied thoroughly in this thesis, with two central goals: for error computation, as described before; and for obtaining the curvature of the path in its local representation, ir order to provide appropriate references for the longitudinal velocity $u$. In order to avoid confusion, $L_e$ will denote the lookahead distance used for error computation, whilst $L_k$ represents the one for obtaining the curvature of the path, topic that will be explained next.

**Curvature of a given path**

The last topic on this section is related to how the curvature of a path can be obtained. This is useful in many different aspects, namely when information about the radii is required in order to apply (4.3), or to obtain the curvature at a given lookahead reference point. With these two objectives in mind, different expressions for computation the curvature of the course, or a portion of it, are described in this part.
If a given track is provided as a list of $n$ points expressed in the global frame, $(W_1, ..., W_n)$, and their coordinates $[X_i, Y_i]$ are taken as parts of a parametric function of a given variable path $s_{path}$, then the curvature can be expressed as a function of $s_{path}$, as the following equation denotes:

$$K(s_{path}) = \frac{X'Y'' - Y'X''}{((X')^2 + (Y')^2)^{3/2}} \qquad [\text{ m}^{-1}\text{ }] \tag{4.11}$$

where $X'$ and $Y'$ represent the first derivatives of $X$ and $Y$ with respect to $s_{path}$, $\frac{dX}{ds_{path}}$ and $\frac{dY}{ds_{path}}$, while $X''$ and $Y''$ stand for the analogous second derivatives, and can be computed using an appropriate method when applied to discretized data [36]. The curvature $K(s_{path})$ is not to be confused with previous or future gain matrices, hence the explicit inclusion of $(s_{path})$ in (4.11). This representation of curvature as a function of $s_{path}$ are advantageous when $Y$ cannot be given as a function of $X$, which happens with most of race circuits, as each value of $Y$ may correspond to more than one $X$, regardless of the global frame chosen.
In the cases that the one coordinate can indeed be expressed as a function of the other, particularly when a polynomial is used to interpolate a given set of waypoints as figures 4.5 and 4.6 illustrate, another definition of curvature can be used. If $x_i$ and $y_i$ are the coordinates expressed in the vehicle's frame of the $n$ points interpolated for the reference path generation, the curvature can be obtained from the derivatives of $y$ with respect to $x$, as the following equation states:

$$k(x) = \frac{y''}{(1 + (y')^2)^{3/2}} \qquad [\text{ m}^{-1}\text{ }] \tag{4.12}$$

where $y = \frac{dy}{dx}$ and $y'' = \frac{d^2y}{dx^2}$. In the particular case of the interpolating function being a second-order polynomial, given by $f_{pol}(x) = ax^2 + bx + c$, the above expression can be rearranged in order to allow a more direct way of obtaining the curvature as a function of $x$:

$$k(x) = \frac{2a}{(1 + (2ax_{sol} + b)^2)^{3/2}} \qquad [\text{ m}^{-1}\text{ }] \tag{4.13}$$

Both these definitions are valid only if the condition of function remains true when the polynomial is obtained from interpolation: for each $x$ coordinate, only one $y$ value can be corresponded to it through such function. This must be kept in mind as tight curves and largely spaced waypoints could cause the inviability of this condition, and therefore resulting in no adequate polynomial.

## 4.3 Lateral Control

In the previous section, the path-tracking errors were appropriately defined for multiple situations. This, together with the control strategies explained in Chapter 2 and the models described on section 3.2 and 3.3, can be utilized in order to design controllers for the a simulation of the FST driverless vehicle, modelled in 3.1. With such objective, in this section, the methods to derive such controllers are explained in detail, detailing their implementation as well.

The error logic chosen for this thesis, whether in applying the controllers presented in this section, as well as the final implementation on the real model described in Chapter 5 with the respective results presented in 6, is the one where the waypoints of the path are interpolated by a second-order polynomial in a coordinate system centred on the vehicle. This was the method picked since it allowed for a reference of continuous first (and second) derivative, which results in smoother error curves in time, and ultimately provides smoother command inputs to the steering angle $\delta_{ref}$ as well. The strategy using a lookahead distance is also applied for these situations, and its effect on the overall path-tracking is an important aspect to be studied. Such analysis and comparison is done in Chapter 6.

Applying the aforementioned error computation logic, the multiple controllers of this section are then validated, either in the model used for their design, which is the case for the different LQR-based controllers, or with the kinematic bicycle model described in (3.3), for the Stanley and L1 controllers, since these are kinematic-based controllers. For every process of validation, both the time constant of the steering system, $c_\delta = 0.05$ [s], and the saturations on its final value, $\delta_{min}$ and $\delta_{max}$, were kept in the simulation in order to represent the physical constraints of the actuation system. This validation should be achieved in two different situations, both at constant speed $u$. Firstly, they are tested supplying a simple line of approximately 21 metres as reference, and with the simulated vehicle starting at a distinct orientation than the one of the segment. The second test scenario is a semi-circle of radius equal to 10 metres, requiring that the algorithm tracks a continuously curved path. The results in each can be easily validated using the errors for line segments and circumference arcs, using equations (4.2) and (4.3), as the radius of the arc is known.

The validations are made for all five controllers, in the same order as these are presented, and the results are provided in respective tables. However, only the images corresponding to the first two are shown, as they validate not only the controllers, but the path-following algorithms as well, since the Stanley controller is validated without a lookahead, computing errors from (4.6), whilst the L1 controller requires such lookahead, using (4.10). The remaining figures obtained from the validation processes for the three LQR and LQG controllers are made available in the first appendix, A.

### 4.3.1 Stanley Controller

The first path-following strategy to be successfully implemented was the Stanley controller, already described in section 2.2, which consists on a nonlinear control strategy, feeding back both the cross-track and orientation errors [15]. However, three modifications were applied during such implementation, differing from its original implementation: firstly, the point of the vehicle at which the errors were measured

was the $CG$ of the vehicle as it was done for the remaining controllers, instead of the centre of the front axis; the orientation error was computed with the velocity vector as (4.6) and (4.9) state (depending of the existance of a lookahead or not), instead of being computed as the angular difference between the orientation of the path, $\psi_{ref}$, and the car's heading angle, $\psi$; lastly, the controller was also used together with a lookahead line, in order to study its influence in general path-tracking performance, as it will be done with all the posterior controllers in this thesis.

The control law for the steering angle in terms of such errors is provided by the following equation [15]:

$$\delta(t) = e_\psi + \frac{k_{Stanley} \cdot e_y}{u} \quad , \ \delta \in [\delta_{min}, \delta_{max}] \qquad [\text{ rad }] \tag{4.14}$$

where $u$ is the vehicle's longitudinal velocity [m/s], and $k_{Stanley}$ is a gain parameter of this controller, having units of rad·s$^{-1}$. The logic behind such control law is simple: adjust the steering angle $\delta$ with a value equal to $e_\psi$, in order to drive it to zero, adding a component that incorporates $e_y$, taking into account the velocity of the car. This second part is necessary since that, for large velocities, the adjustment that must be made in order $e_y$ to zero should be small, as this error will be corrected quickly. Additionally, this second part of the control law will force the vehicle to drive almost perpendicularly to the path when large cross-track errors arise. The gain parameter $k_{Stanley}$ then acts a measure of how aggressively the controller will drive $e_y$ to zero, when compared to $e_\psi$.

The validation was then made for the implementation of the controller as 4.14 states, as well as using a lookahead distance equal to 5 metres. In both situations, it was considered $u = 5$ [m/s] and $k_{Stanley} = 3$ [rad/s], and the nonlinear bicycle kinematics model of (3.14) was used.

- **Validation with the bicycle kinematics model:**

  The first test, as stated before, is a simple line of approximately 21 metres, starting on the origin and finishing at $(X, Y) = (15, 15)$ [m] of the global frame. The vehicle starts with zero longitudinal velocity, $u = 0$ [m/s], and with null heading angle, $\psi = 0$ [rad], meaning it· is aligned with the global $X_G$ axis. As for the second validation method, the circumference of 10 metres radius, centred on $C = (0, 10)$ [m], was used as the reference path. The results for each of this validation processes are present on figures 4.7 and 4.8, the mean and root mean square (RMS) values of the crosstrack, $(\mu)_{e_y}$ and $(\text{RMS})_{e_y}$, and orientation errors, $(\mu)_{e_\psi}$ and $(\text{RMS})_{e_\psi}$, were computed, in order to evaluate the performance of the controllers for validation effects, summarized in table 4.1.

Table 4.1: Results for validation of the Stanley controller.

| Validation scenario | $(\mu)_{e_y}$ [m] | $(\text{RMS})_{e_y}$ [m] | $(\mu)_{e_\psi}$ [rad] | $(\text{RMS})_{e_\psi}$ [rad] |
|---|---|---|---|---|
| Line segment | 0.21 | 0.50 | 0.00 | 0.27 |
| Circumference | 0.23 | 0.24 | 0.00 | 0.02 |

As can be seen from figures 4.7 and 4.8, the simulated model is able to approximately track the path in both situations. In the first, the difference in the starting heading takes a toll as the

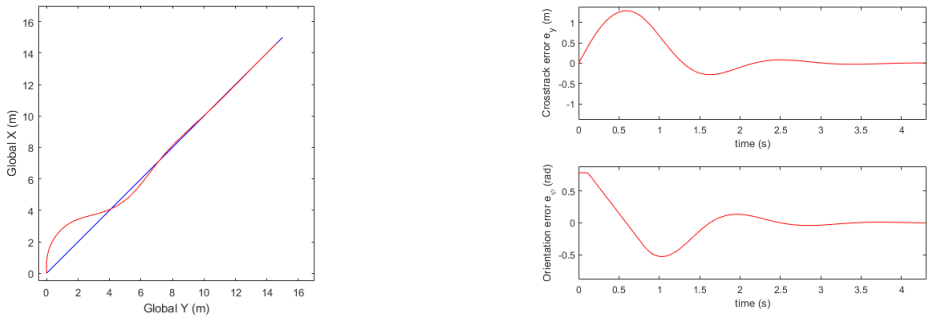


Figure 4.7: At the left, the path described by the vehicle (in red) when the reference is a line segment (in blue). On the right, the resulting crosstrack error $e_y$ [m] in the upper-right, and orientation error $e_\psi$ [rad], in the lower-right, plotted with the time to reach the end-point.

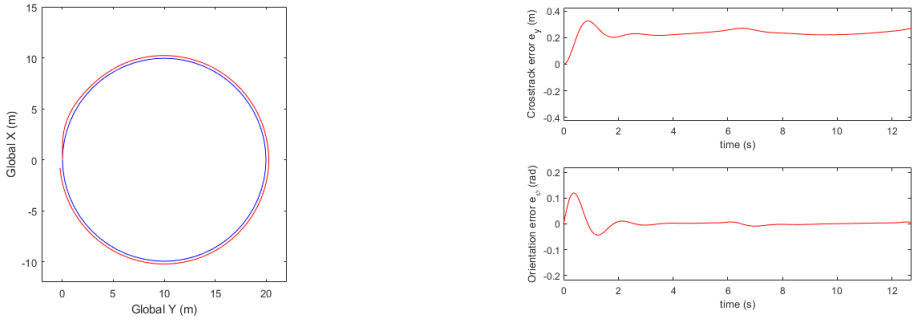


Figure 4.8: At the left, the path described by the vehicle (in red) when the reference is a circumference (in blue). On the right, the resulting crosstrack error $e_y$ [m] in the upper-right, and orientation error $e_\psi$ [rad], in the lower-right, plotted with the time to reach the end-point.

vehicle starts with a large orientation error but a virtually null crosstrack one, and then drives the one to zero at the cost of increasing momentarily $e_y$. In the second case, present in 4.8, the crosstrack error remains constant, which represents the struggle of the algorithm to track a reference that continuously changes its orientation, but $e_\psi$ is soon driven to zero. Increasing the gain $k_{Stanley}$ would drive $e_y$ closer to zero, but at the cost of inducing greater oscillation, and since these situations are validation-only scenarios and real-tracks have much different characteristics, the option was to maintain $k_{Stanley} = 3$. Having said this, taking into account the values present on table 4.1, the controller is therefore validated.

### 4.3.2 L1 Controller

The second controller to be applied at the problem at hand was the L1 controller, as described previously in section 2.2, originally used for autonomous guidance of UAVs [31], and now adapted to a situation where the vehicle to be guided is a car. Such control strategy relates the angle $\eta$ as defined in (4.10), being the angular difference between the vectorial velocity $V$ of the vehicle and the line going from it to the reference point, as shown on figure 2.7, to a given acceleration command, with the objective of guiding the vehicle towards the path, as (2.24) denotes. In the case of a moving object, such lateral acceleration can be represented by the product of its linear, $V$ [m/s], and angular velocity, $\dot{\psi}$[rad/s].

Considering $V = u$ and the using the expression for $\dot{\psi}$ of the bicycle kinematic model from (3.14), the lateral acceleration becomes:

$$a_{lat} = \frac{u^2}{L}\tan(\delta) \qquad [\text{ m/s}^2\text{ }] \tag{4.15}$$

in which $L$ denotes the wheelbase of the vehicle in metres, not to be confused with the lookahead distances that will be used. Combining (4.15) and (2.24), and solving for $\delta(t)$, the expression for the L1 controller for guiding a car-like vehicle is the following:

$$\delta(t) = \arctan\left(\frac{2L\sin(\eta)}{L_e}\right) \quad , \quad \delta \in [\delta_{min}, \delta_{max}] \qquad [\text{ rad }] \tag{4.16}$$

In (4.16), the lookahead distance was referred as $L_e$ in order to implicitly indicate that the angle $\eta$ [rad] is obtained from the lookahead used for error computation, and applying the the methods described in section 4.2. $L_e$ is naturally expressed in metres.

The above control law is similar to some applications of a pure pursuit controller [16], and incorporates information of both the orientation error, since $\eta$ will be greater when the car has a larger difference in orientation, as well as some form of crosstrack error, since it will also be greater when the vehicle is further from the path.

This represents an elegant and simple controller, based only in kinematic aspects of the vehicle, but it still yields quite good results in path-following scenarios, and its validation is presented next.

- **Validation with the kinematic bicycle model a lookahead distance of 5 metres:**
  The controller was used in the kinematic bicycle model, and a lookahead of $L_e = 5$ [m], in order to validate the application of a lookahead distance into this model. The results of such procedures are shown on figures 4.9 and 4.10, obtained from a line and a circumference as a reference, respectively.

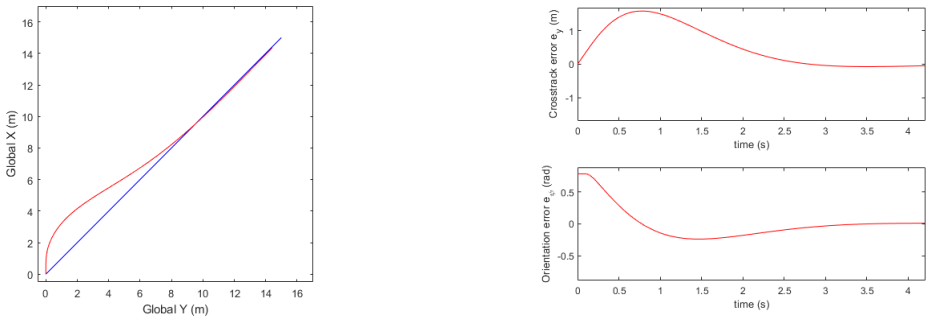


Figure 4.9: At the left, the path described by the vehicle (in red) when the reference is a line segment (in blue). On the right, the resulting crosstrack error $e_y$ [m] in the upper-right, and orientation error $e_\psi$ [rad], in the lower-right, plotted with the time to reach the end-point.

Despite having a slower convergence to the reference line, as figure 4.9 denotes, the tracking of the

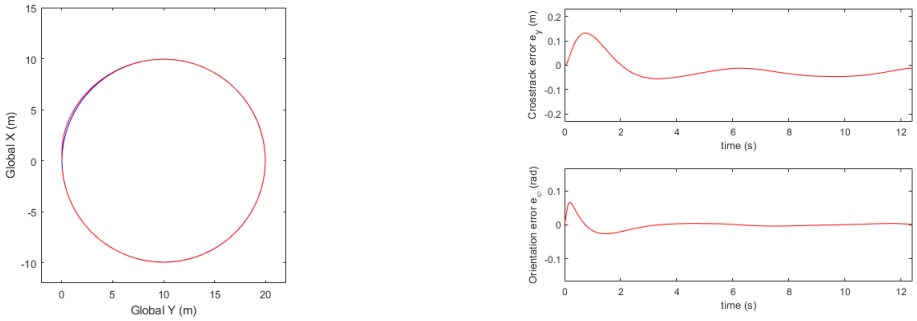


Figure 4.10: At the left, the path described by the vehicle (in red) when the reference is a circumference (in blue). On the right, the resulting crosstrack error $e_y$ [m] in the upper-right, and orientation error $e_\psi$ [rad], in the lower-right, plotted with the time to reach the end-point.

Table 4.2: Results for validation of the L1 controller.

| Validation scenario | $(\mu)_{e_y}$ [m] | $(\text{RMS})_{e_y}$ [m] | $(\mu)_{e_\psi}$ [rad] | $(\text{RMS})_{e_\psi}$ [rad] | $(\mu)_{L_e}$ [m] |
|---|---|---|---|---|---|
| Line segment | 0.53 | 0.80 | 0.00 | 0.25 | 4.88 |
| Circumference | -0.02 | 0.05 | 0.00 | 0.01 | 4.96 |

curved path on figure 4.10 is significantly better when compared to the Stanley controller. This is confirmed by the values presented on table 4.2, in which the mean value of the lookahead distance $L_e$ was also included for validation effects. From these, it can be concluded that this controller indeed tracks the reference path and that the lookahead point is approximately at 5 metres of distance to the vehicle, therefore validating both the controller and the lookahead method.

### 4.3.3 2 state LQR controller

In the domain of LQR/LQG controllers, the first one to be implemented is the Linear Quadratic Regulator that corresponds to the linearized model of (3.17). In this case, this controller will have the errors $e_y$ and $e_\psi$ as feedbacks, similarly to the Stanley controller, which will replace the original state on the model equation. With these two errors being fed back, the control law for a LQR controller with two states is the following:

$$\delta(t) = K_y^{2S} e_y + K_\psi^{2S} e_\psi \quad , \quad \delta \in [\delta_{min}, \delta_{max}] \qquad [\text{ rad }] \tag{4.17}$$

where $K_y^{2S}$and $K_\psi^{2S}$ are the gains obtained from the Algebraic Riccatti Equation (ARE) as described by (2.16), for given matrices $\mathbf{Q}^{2S}$ and $\mathbf{R}^{2S}$, that have SI units compliant with such equation. The superscript $2S$ was placed in order to distinguish these parameters in each different application of the LQR/LQG. Since the ARE that arises, for a diagonal matrix $\mathbf{Q}^{2S}$ with elements $(Q_y^{2S}, Q_\psi^{2S})$ and a scalar $\mathbf{R}^{2S}$, corresponds to a sum of $2 \times 2$ matrices, it can be solved manually, with the gains being calculated from:

$$K_y^{2S} = \frac{\sqrt{Q_y^{2S} \cdot R^{2S}}}{R^{2S}} \qquad [\text{ rad/m }] \tag{4.18a}$$

$$K_\psi^{2S} = \frac{\sqrt{R^{2S}}}{R^{2S}} \cdot \sqrt{Q_\psi^{2S} + 2L\sqrt{Q_y^{2S} R}} \qquad [\text{ - }] \tag{4.18b}$$

which means that both gains are independent of the velocity, and that the wheelbase of the vehicle, $L$, has no influence on the value of $K_y^{2S}$.

The starting point of tuning this LQR is to have $\mathbf{Q}^{2S} = \mathbf{I}$, where $\mathbf{I}$ is the $2 \times 2$ identity matrix, and $\mathbf{R}^{2S} = 1$. Despite these being values that already enables the validation of this controller, some changes were made to matrices applying Bryson's rule: considering that the maximum values for the states, $e_y$ and $e_\psi$, to be 0.20 [m] and 0.50 [rad], respectively, and that the input $\delta$ must not surpass 0.47 [rad], the new LQR matrices, in their respective SI units, can be chosen as:

$$\mathbf{Q}^{2S} = \begin{bmatrix} 25 & 0 \\ 0 & 4 \end{bmatrix} \tag{4.19a}$$

$$\mathbf{R}^{2S} = 4.5 \tag{4.19b}$$

The matrices already include some desired aspects for the driving of errors to zero, as well as the actuator limits, but are still in terms of the kinematic model. This must be kept in mind, as the simulated real vehicle in a real-track scenario could have some limitations regarding this. Final tuning should be made when applying this controller, as well as the remaining LQG-based ones, in order to account for such limitations. This matrices $\mathbf{Q}^{2S}$ and $\mathbf{R}^{2S}$ result in the following gains of the two-state LQR controller:

$$K_y^{2S} = 2.3570 \qquad [\text{ rad/m }] \tag{4.20a}$$

$$K_\psi^{2S} = 2.8546 \qquad [\text{ - }] \tag{4.20b}$$

The validation os this controller is presented next.

- **Validation with the kinematic bicycle model:**

  The two validation tests where repeated for this control strategy, applying it to the nonlinear version of the kinematic bicycle model. Table 4.3 illustrates the results obtained with the two tests:

Table 4.3: Results for validation of the LQR controller with 2 states.

| Validation scenario | $(\mu)_{e_y}$ [m] | $(\text{RMS})_{e_y}$ [m] | $(\mu)_{e_\psi}$ [rad] | $(\text{RMS})_{e_\psi}$ [rad] |
|---|---|---|---|---|
| Line segment | 0.18 | 0.49 | 0.00 | 0.31 |
| Circumference | 0.04 | 0.05 | 0.00 | 0.01 |

In can be seen that the values for the validation with a line segment are quite similar to the ones obtained with the Stanley, but with much better tracking for the circumference as a reference, rivalling with the L1 controller, despite not having a lookahead distance used for the validation process. Such comparison is valid since these controllers were tested using the same kinematic model. Taking into account the values of table 4.3, the controller is considered to be validated. As stated in the beginning of this section, the figures obtained from this validation are made available in the appendix A, if additional insight is desired.

### 4.3.4 3 state LQG controller

The second implementation of a proportional controller reports to the bicycle kinematics model with sideslip, with equation (3.21) representing its linearized form. However, since only $e_y$ and $e_\psi$ are available from the sensors, as specified in the end of section 3.1, the third state $v$, the slip or lateral velocity in the vehicle's frame, must be estimated. Therefore, this implementation consists on a Linear Quadratic Gaussian (LQG) controller, since it will use a Kalman filter as an aid to provide full-state feedback to the LQR. Since there are no reference inputs to $v$, or to its estimate $\tilde{v}$, the applied configuration was the servo system with this state in the inner loop, as (2.12) and figure 2.3 illustrate. Therefore, the control law is the provided by the following equation:

$$\delta(t) = K_y^{3S} e_y + K_\psi^{3S} e_\psi + K_v^{3S} \tilde{v} \quad , \quad \delta \in [\delta_{min}, \delta_{max}] \qquad [\text{ rad }] \tag{4.21}$$

The gains $K_y$ and $K_\psi$ will have the same units as before, and $K_v$ will ne expressed in rad·(m/s)$^{-1}$.
Regarding the implementation of this controller, one brief note should be given: since the Riccati equations required to be solved in order to compute explicitly the observer and controller gains, $\mathbf{L}^{3S}$ and $\mathbf{K}^{3S}$ already have some complexity to it and are difficult to solve directly, the choice was to compute both these gain matrices at each instant of simulation, since the main parameter that causes variations on their values is $u$, the longitudinal velocity of the vehicle. The sampling time of $T_s = 0.1$ [s] allows for a smooth simulation with this method being applied, but a gain scheduling for varying $u$ was also developed, and further details about this are addressed on Chapter 5. As such, only the gains for $u = 5$ [m/s] are used for the validation, and shown on this section.
Starting with the estimation of $v$, $\tilde{v}$, the first step is to obtain $\mathbf{Q}_0^{3S}$ and $\mathbf{R}_0^{3S}$, in order to obtain the gains for the Kalman filter, $\mathbf{L}^{3S}$. As stated on section 2.1, namely on the part reporting to observer design, matrices $\mathbf{Q}_0^{3S}$ and $\mathbf{R}_0^{3S}$represent the covariances of process and sensor noises, respectively, which are assumed to be uncorrelated and have zero mean. Since process noises are hard to estimate, it was assumed that the values of $\mathbf{Q}_0^{3S}$ were in the order of greatness of a decimal of the respective states, and the uncertainties of $0.05$ [m] and $0.05$ [rad] were used for $\mathbf{R}_0^{3S}$, resulting of the sensor noise. Therefore, the matrices, expressed in their SI units, for the Kalman filter are the following:

$$\mathbf{Q}_0^{3S} = \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.1 \end{bmatrix} \tag{4.22a}$$

$$\mathbf{R}_0^{3S} = \begin{bmatrix} 0.05 & 0 \\ 0 & 0.05 \end{bmatrix} \tag{4.22b}$$

In order to check if the observed provided accurate estimates of $v$, the validation tests were run with $\mathbf{Q}^{3S} = \mathbf{I}_{3\times 3}$ and $\mathbf{R}^{3S} = 1$, and the values of $v$ obtained directly from the model were compared to the

ones resultant of the estimation, $\tilde{v}$. Such comparison is provided on figure 4.11, where it can already

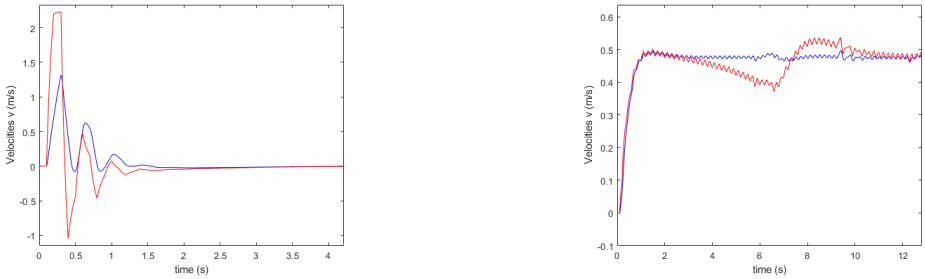


Figure 4.11: Comparison between the lateral velocity $v$ of the simulated model (blue) and the estimation $\tilde{v}$ (red), using the line-segment as reference, on the left, and the circumference, on the right.

be seen that there are some differences between the estimation and the real value from the model. However, the observer still manages to portray the general tendency of the state it is estimating. The oscillation on for the circumference test in figure 4.11 is due to the path-following algorithm, and it does not invalidate the observer.

Defining the estimation error for this state as $e_v = v - \tilde{v}$, its mean and RMS values can also computed for both cases, and their values are presented on the table 4.4, in which an additional decimal place was used in order to give further information about the values.

Table 4.4: Results for estimation of $v$ using a Kalman filter.

| Validation scenario | $(\mu)_{e_v}$ [m/s] | $(\text{RMS})_{e_v}$ [m/s] |
|---|---|---|
| Line segment | 0.031 | 0.331 |
| Circumference | 0.005 | 0.037 |

The differences between $v$ and its estimate $\tilde{v}$ are small as noted by table 4.4, indicating that the designed observer is indeed tracking its respective state. The observer gain matrix used for the validation at $u = 5$ [m/s] is the following, where its units must be according to (2.22):

$$(\mathbf{L}^{3S})_{u=5} = \begin{bmatrix} 3.415 & 1.065 \\ 1.065 & 0.931 \\ 0.074 & -0.027 \end{bmatrix} \tag{4.23}$$

Moving on to the LQR design, an analogous process to the one applied to the 2-state LQR was followed, namely for the values of $\mathbf{Q}^{3S}$ and $\mathbf{R}^{3S}$. Nevertheless, Bryson's rule depends on the maximum tolerable value for the states, which is somehow hard to define for the third one since $v$ is an internal state, having no outside reference. With this in mind, it was assumed that the acceptable maximum value $v$ was of 1 [m/s]. The value of $\mathbf{Q}^{3S}$ corresponding to $e_y$ was increased to account for the slip effects on the crosstrack error, and actuation was penalized with a higher value for $\mathbf{R}^{3S}$ since variations on this value

could result in higher lateral slip. Thus, the final matrices are the following, with its values in SI units:

$$\mathbf{Q}^{3S} = \begin{bmatrix} 40 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{4.24a}$$

$$\mathbf{R}^{3S} = 50 \tag{4.24b}$$

As explained before, the computation of the LQR gain matrix that results from these values was implemented in order for it to be computed at each time step, or as a scalling with increasing $u$. Therefore, the value of $\mathbf{K}^{3S}$ presented here is for validation effects only, at a constant speed of $u = 5$ [m/s], in the appropriate SI units:

$$(\mathbf{K}^{3S})_{u=5} = \begin{bmatrix} 0.894 & 1.249 & 0.194 \end{bmatrix} \tag{4.25}$$

The validation os this controller is presented next, this time including slip dynamics into the kinematic bicycle model.

- **Validation with the kinematic bicycle model with slip dynamics:**
  Having obtained the required observer and gain matrices for the validation process at constant speed, $(\mathbf{L}^{3S})_{u=5}$ and $(\mathbf{K}^{3S})_{u=5}$ respectively, the controller was then tested using a line segment and a circumference as references, as it was done with the previous ones, but applied to the kinematic model including slip dynamics, as present on (3.20). The validation results are provided in table 4.5, indicating the good tracking ability of the controller, and being on pair with the ones obtained for previous implementations, even for a more complex model. Thus, the controller is validated.

Table 4.5: Results for validation of the LQG controller with 3 states.

| Validation scenario | $(\mu)_{e_y}$ [m] | (RMS)$_{e_y}$ [m] | $(\mu)_{e_\psi}$ [rad] | (RMS)$_{e_\psi}$ [rad] |
|---|---|---|---|---|
| Line segment | 0.04 | 0.14 | 0.00 | 0.25 |
| Circumference | 0.15 | 0.16 | 0.00 | 0.01 |

### 4.3.5 4 state LQG controller

The last linear control strategy is the implementation of the LQR controller with a Kalman filter to the dynamic bicycle model, for which the linearization is provided on equation (3.25). When combined with the kinematic equation for $y$, and knowing that $\dot{\psi} = r$, a four-state controller can be designed. Nevertheless, states $v$ and $r$ are not accessible directly from sensors, and thus the design of a Kalman filter to estimate them is necessary. The control law is then given by:

$$\delta(t) = K_y^{4S} e_y + K_\psi^{4S} e_\psi + K_v^{4S} \tilde{v} + K_r^{4S} \tilde{r} \quad , \quad \delta \in [\delta_{min}, \delta_{max}] \qquad [\text{ rad }] \tag{4.26}$$

The fourth gain, $K_r$, is expressed in s$^{-1}$, while $K_y$, $K_\psi$ and $K_v$ will have the same units as explained for the previous controller. The values for $\mathbf{Q}^{4S}$ and $\mathbf{R}^{4S}$ were analogous to the ones for the previous LQG, since it was assumed an equal process noise for $r$, and the noise of the sensors should be the same since it concerns the same two states, given by errors $e_y$ and $e_\psi$. Therefore, the matrices for the Kalman estimator are the following, in their respective SI units:

$$\mathbf{Q}_0^{4S} = \begin{bmatrix} 0.1 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0.1 \end{bmatrix} \tag{4.27a}$$

$$\mathbf{R}_0^{4S} = \begin{bmatrix} 0.05 & 0 \\ 0 & 0.05 \end{bmatrix} \tag{4.27b}$$

Similarly to the Kalman filter applied for the previous controller, the verification of the estimated states was made using $\mathbf{Q}^{4S} = \mathbf{I}_{4\times4}$ for the LQR gain computation, but the input matrix had to have some penalty for $\delta$, as it was seen that a unitary value for it would destabilize the plant. Comparing the states $v$ and $r$, with their respective estimates $\tilde{v}$ and $\tilde{r}$, holds the results presented on figure 4.12.

In this case, the effects of the sampling time of the sampling time $T_s$ on the observer are more notori-

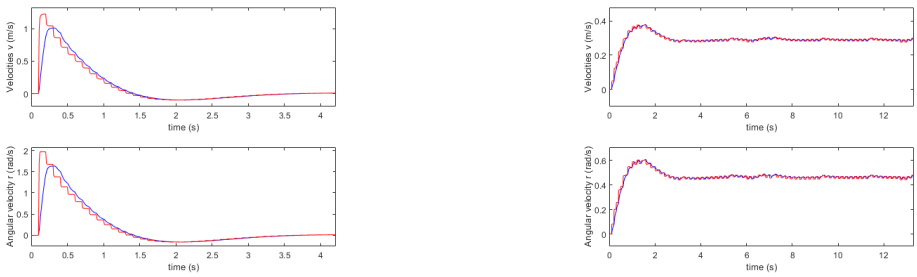


Figure 4.12: Comparison between the lateral velocity $v$ (blue) and its estimation $\tilde{v}$ (red) on the two top pictures, and comparison between angular velocity $r$ (blue) and the respective estimation $\tilde{r}$, on the lower ones. The figures on the left correspond to the line-segment simulation, whilst the ones on the left are resultant of the circumference as a reference path.

ous, nonetheless allowing to see that in both scenarios the estimator can track the actual states $v$ and $r$ of the simulated model. Defining $e_r = r - \tilde{r}$ as the error between the model's angular velocity and the estimated one, and $e_v$ being the analogous for $v$ as it was defined for the previous observer, the mean and RMS values of these differences can be computed and are presented on table 4.6.

Table 4.6: Results for estimation of $v$ and $r$ using a Kalman filter.

| Validation scenario | $(\mu)_{e_v}$ [m/s] | $(\text{RMS})_{e_v}$ [m/s] | $(\mu)_{e_r}$ [rad/s] | $(\text{RMS})_{e_r}$ [rad/s] |
|---|---|---|---|---|
| Line segment | -0.003 | 0.118 | -0.003 | 0.1920 |
| Circumference | 0.000 | 0.008 | 0.000 | 0.0140 |

As can be seen from table 4.6, in both scenarios the tracking is almost perfect, being slightly worse in the case of the line segment as reference path. Similarly to the previous controller, both the gains of the observer $\mathbf{L}^{4S}$ and of the control matrix $\mathbf{K}^{4S}$ were either computed at each time instant for different $u$ values, or scaled with this velocity. For validation effects, the gain matrix for the Kalman filter at $u = 5$ [m/s] is the following, presented in appropriate SI units:

$$(\mathbf{L}^{4S})_{u=5} = \begin{bmatrix} 3.463 & 1.126 \\ 1.126 & 1.016 \\ 0.000 & -0.032 \\ -0.002 & 0.150 \end{bmatrix} \tag{4.28}$$

As for the feedback gain counterpart of the LQG, a similar approach to the one taken for the 3 state LQG was made, but the value of $\mathbf{Q}^{4S}$ that corresponds to $e_y$ was increased again, in order to counteract the effects of the remaining states. As such, the final $\mathbf{Q}^{4S}$ and $\mathbf{R}^{4S}$ are presented next, in their respective SI units:

$$\mathbf{Q}^{4S} = \begin{bmatrix} 40 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0.1 & 1 \end{bmatrix} \tag{4.29a}$$

$$\mathbf{R}^{4S} = 50 \tag{4.29b}$$

And the resulting $\mathbf{K}^{4S}$ gain matrix when $u = 5$ [m/s], for validation effects, is the following:

$$(\mathbf{K}^{4S})_{u=5} = \begin{bmatrix} 1.414 & 1.594 & 0.037 & 0.045 \end{bmatrix} \tag{4.30}$$

Having gathered all the required elements, the validation of this controller is presented next.

- **Validation with the dynamic bicycle model:**
  Directly applying the controller using the gains of (4.30) and the observer with (4.28) to the non-linear bicycle model provided by the equations in (3.24), simulations were run in order to attest for the two validation scenarios, which provide the values for table 4.7. With it, it can be concluded that the controller has a worse performance than the ones before. Nevertheless, one must also remember that this model already accounts for many of the dynamic effects, and some additional resistance is to be expected, whilst the models that were used in validating the previous controllers did not. Therefore, this controller is validated, concluding the presentation of the controllers used for the steering of the vehicle.

Table 4.7: Results for validation of the LQG controller with 4 states.

| Validation scenario | $(\mu)_{e_y}$ [m] | $(\text{RMS})_{e_y}$ [m] | $(\mu)_{e_\psi}$ [rad] | $(\text{RMS})_{e_\psi}$ [rad] |
|---|---|---|---|---|
| Line segment | 0.16 | 0.39 | 0.00 | 0.24 |
| Circumference | 0.11 | 0.11 | 0.00 | 0.01 |

## 4.4 Velocity Control

In the previous sections of this chapter, the errors that allow for a path-following algorithm to know where the vehicle is in relation to the reference path, as well as different control methods to driving such errors to zero, were presented assuming that the car was travelling at a constant speed. However, path-following approaches in general do not require such condition, and adopt a decoupled strategy, consisting on executing the lateral control first, allowing the vehicle to effectively track the runway, and then focusing on the control of its speed. Nevertheless, as to be expected, increasing the velocity should have a negative impact on the tracking of the path, as the dynamic effects are more present with such increase. Therefore, a velocity control strategy should also include a mechanism that provides the appropriate references for the controller, in order make the vehicle travel at speeds that ensure an approximate, at least, following of the path. In a race situation, this assumes a new importance since the objective is to be faster, but still not crossing the track's boundaries.

### 4.4.1 References for velocity regulation

One way to provide appropriate references for the velocity is to take into account the curvature of the path, as high curvatures mean tight curves, which should be faced with lower speeds. In an analogous way, lower values for curvatures are to be expected, the velocity of the vehicle should be increased. Naturally, some anticipation in this regulation is advantageous, as it allows for timely deceleration before reaching a curve, and accelerating when the next segment is straight. Therefore, a lookahead for the curvature, $L_k$, can be used in order to provide said anticipation.

Assuming the curvature of the path is not previously known to the algorithm, the curvature at a distance $L_k$, can be obtained by interpolating the waypoints in front of the car, computing the reference point as a solution to (4.7) and (4.8) with $L_A = L_k$, and then applying equation (4.13), as it allows for the calculation of a curvature for a second-order polynomial.

$$k_{L_k} = \frac{2a}{(1 + (2ax_{RP} + b)^2)^{3/2}} \qquad [\text{ m}^{-1}\text{ }] \tag{4.31}$$

where $x_{RP}$ is the $x$ coordinate of the referred reference point in the vehicle's frame, defined as one of the roots of (4.7) as explained on section 4.2. One should also remember that the lookahead distance applied for curvature computation, $L_k$, may be used independently that the one used for error computation, $L_e$, as each one can provide its own advantages.

The next question that arises is how to relate the velocity to provide to the vehicle, $u_{ref}$, with values for curvature of the path at the reference point, $k$. The maximum speed of the vehicle for control purposes was assumed to be $u_{max} = 25$ [m/s], in order to have a tolerance regarding the actual maximum speed. On the other hand, a minimum velocity of $u_{min} = 5$ [m/s] to travel the different courses. The lateral ac-

celeration should never exceed 1 G [9.807 m/s$^2$], which, from $a_{lat} = u \cdot \dot{\psi} = \frac{u^2}{R}$ with the specified values for $u_{max}$ and $u_{min}$, would result in approximately 0.392 [m$^{-1}$] for the maximum and 0.016 [m$^{-1}$] for the minimum curvature, knowing that $k = \frac{1}{R}$. However, such limits would cause the vehicle to easily loose traction at the curves, and the maximum acceleration was reduced so the car could track the reference path appropriately. By setting $a_{lat,max} = \frac{9.807}{2.4} = 4.086$ [m/s$^2$], which presented a satisfying value for this acceleration, the maximum and minimum curvatures can be obtained, and the reference velocity can is provided by the following:

$$u_{ref}(k) = \sqrt{\frac{4.086}{|k|}} \qquad \text{with} \quad u_{ref} \in [5, 25] \qquad [\text{ m/s }] \tag{4.32}$$

Two brief notes should be given regarding (4.32): firstly, a modulus sign was used in $k$ as the velocity should be adjusted regarding its absolute value; secondly, the values of $k$ should be kept between 0.0065 and 0.163 [m$^{-1}$] in order to provide the reference velocities this equation provides. This relation between $k$ and $u_{ref}$ can be observed on figure 4.13, where can also be seen that $u_{ref}$ is limited between 5 and 25 [m/s], even if $k$ is outside the mentioned interval. It can be verified that this equation will result in a rapid decrease in the reference velocity as the curvature increases, which should be the appropriate behaviour to regulate the vehicle's speed.

This concludes the process of supplying the controller with the reference velocities in order for it to

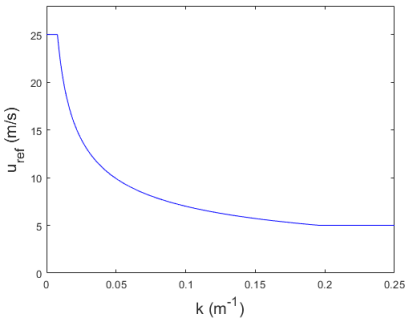


Figure 4.13: Applied profile for obtaining the reference velocity $u_{ref}$ [m/s] from the curvature of the path $k$ [m$^{-1}$], in blue.

adjust the vehicle's velocity at each instant. Thus, such controller and how it is obtained is explained next.

It will be seen in Chapter 6 that this relation between the curvature and the reference velocity is demanding to the controllers and lookaheads applied, as in some cases it will provide velocities too high for appropriate tracking. However, this velocity profile was chosen to explore such differences between the several controllers, and how the anticipation provided by a lookahead distance could influence the following of the path.

### 4.4.2 LQT controller

Already having designed an appropriate way to provide the desired longitudinal velocities for the vehicle taking into account the path, mainly the variation of its curvature, the final step is to design a controller

that ensures the car follows these references. The starting point for such controller is the linearized model of the longitudinal dynamics, presented in equation (3.27), as the choice was to apply a linear method for regulating the velocity $u$. However, instead of using the servo configuration as it was done with the LQG controllers with 3 and 4 states, the approach was to feed back the integral of the error, in order to use the configuration that image 2.4 depicts. This was made as to assure the controlled system has both fast error dynamics and null steady-state error, which is important for rejecting disturbances, like the wind. The result of applying (2.13) is the following augmented system:

$$\begin{bmatrix} \dot{u} \\ \dot{u}_i \end{bmatrix} = \begin{bmatrix} \frac{-\rho A_F C_r u_{eq}}{m} & 0 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} u \\ u_i \end{bmatrix} + \begin{bmatrix} 2\frac{T_{max} GR}{r_{wheel}} \\ 0 \end{bmatrix} d_u + \begin{bmatrix} 0 \\ 1 \end{bmatrix} d_{cmd} \tag{4.33}$$

where $d_{cmd} = \frac{u_{ref} - u}{u_{max} - u_{min}}$, which corresponds to the driving command, and is limited between $-1$ and $1$, standing for full brake or full throttle, respectively. $d_u = \frac{u}{u_{max} - u_{min}}$ will be an analogous variable, expressing the current velocity $u$ in the same interval of $d_{cmd}$. By defining an appropriate set of values for $\mathbf{Q}^{long}$ and $\mathbf{R}^{long}$, where the superscript $long$ was used to denote these correspond to the subsystem referent to the longitudinal velocity, a LQR can be designed for controlling it. The choice was then to only penalize the state correspondent to the error integral, which can be seen from the following values for these parameters, expressed in their SI units:

$$\mathbf{Q}^{long} = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} \tag{4.34a}$$

$$\mathbf{R}^{long} = 0.1 \tag{4.34b}$$

In can be seen that the values for the gains $\mathbf{K}^{long}$ that result from these matrices have very little variation as $u$ increases, being virtually the same for all velocity values. As such, there was no requirement for scheduling or "on-the-go" computation as the previous LQG on the lateral control, and the gains were fixed, as the following equation states with $\mathbf{K}^{long}$ expressed in the respective SI units:

$$\mathbf{K}^{long} = [10 \quad 10] \tag{4.35}$$

With such gains, the system can appropriately track the reference $u_{ref}$, as can be seen from the validation, presented next.

- **Validation with the nonlinear longitudinal model:**
  In order to attest for the performance of the designed controller for the velocity, a set of curvatures were given to the module that generates the reference velocity, which in turn was provided to the controller. This way, one could verify if both parts of this mechanism are working as intended. The model used for this validation was the nonlinear longitudinal model present in (3.26), and (4.32) was applied for supplying the reference velocities. Providing curvature values of 0.05, 0 and -0.02 [$m^{-1}$] should result in approximate values for $u_{ref}$ of 9.9, 30 and 15.6 [m/s], respectively obtained from (4.32). The validation is then performed by using these three different curvatures as an input

to the system, each one during five seconds. Figure 4.14 shows the values for the curvature along time, as well as the reference velocity generated $u_{ref}$, and the one of the controlled system, $u$.

As can be seen from figure 4.14, the displayed tracking of the system fulfils the control design

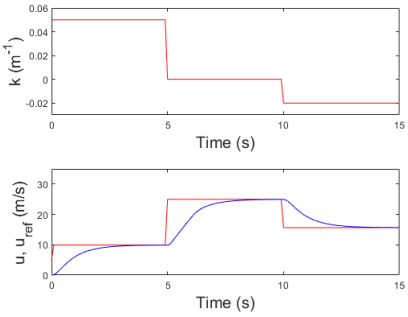


Figure 4.14: Curvatures used in simulation in the above image, and resulting reference velocities in red in the lower image, compared to the ones obtained from the model, in blue.

requirements for this case: the error converges quickly to zero, with no oscillatory behaviour or overshoot. Further tuning could be made aiming to have even faster response, but one must be aware of the physical limits of the system under analysis, and requiring almost immediate increases or decreases in velocity could provoke longitudinal slip in the tyres in a real vehicle or simulation that already such dynamic effects. Additionally, it is natural that the curvature profile of a path is much smoother than the one used for this validation, and faster tracking will certainly occur under normal race conditions. Therefore, the response was considered to be satisfactory for controlling the velocity of the vehicle, and the controller was validated.

## 4.5 Path-Following Architecture

With the conclusion of the last section, all the elements required for solving the problem at hand, the path-following of a FST vehicle, have been properly presented and explained. Before describing the simulation developed for testing the controllers with a model for the real vehicle, it is important to summarize the different parts that compose the solution developed for the path-tracking problem, as to clarify all the concepts and avoid posterior confusions.

The developed path-following strategy can be separated into four different parts:

- Simulation of detection and computation of reference course:

  Since this thesis focuses on the lateral control of the vehicle, the path to be followed was assumed to be already computed, resultant of the detection of the cones of the track. As such, the position of the vehicle in the global coordinate system, $P = (X, Y)$, is used to iterate the $n$ sequent waypoints from the path, $(W)_{1,\dots,n}$, expressed in the same frame. From this block, the position of the reference points adjacent and in front of the vehicle in the local frame are obtained, $w_{i,\dots k}$, $w_i$ standing for the position of the closest point of the path, and $w_k$ for the last point to be included

for interpolation at each instant, both expressed in the frame of the vehicle. The number of cones depends on the lookahead distance, but a minimal of 6 was always used (the adjacent and 5 in front), in order for the interpolation to represent the path with some accuracy;

- Error and curvature computation:
  Knowing the position of the points in relation to the car, these are interpolated for attaining a second-order polynomial in the $x$ axis of the vehicle. From this, knowing the lookahead distances for the errors, $L_e$, and for the curvature, $L_k$, the respective reference points, $RP_e$ and $RP_k$, can be calculated. From these points, the errors $e_y$, $e_\psi$ and $\eta$ can be computed, as well as the curvature $k$, respectively as well, as section 4.2 demonstrates;

- Lateral control:
  From the obtained errors, a proper steering angle input $\delta_{ref}$ is obtained from one of the multiple controllers depicted on section 4.3. However, it should be noted that the LQG controllers require information about the outputs of the simulated model to estimate some states, and the Stanley controller needs the longitudinal velocity $u$ as well, which are fed back to these controllers;

- Velocity control:
  Having obtained $k$ at its reference point on the path, this module computes an appropriate longitudinal velocity to execute the path, $u_{ref}$. Knowing this and $u$, the driving command $d_{cmd}$ is used to either increase or decrease the velocity of the model.

From this solution result the two inputs of the model: $\delta_{ref}$ and $d_{cmd}$. In turn, these are supplied to the vehicle model with the objective of it adjusting its steering and/or longitudinal velocity. A block diagram of the developed architecture and model is now presented on figure 4.15 for clarification.

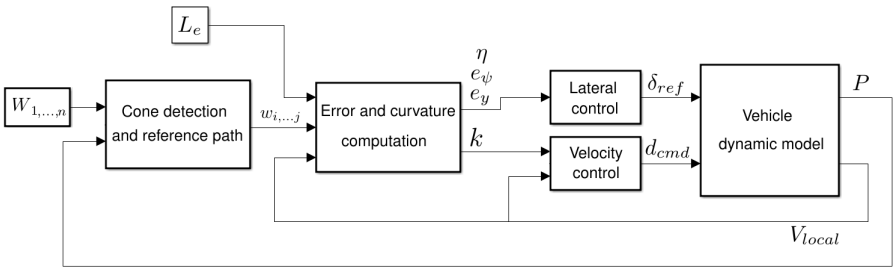


Figure 4.15: Block diagram of the path-following architecture developed, including the vehicle model for clarification on how the solution receives information from it.

One note should be provided regarding the lookahead distances: as can be seen from figure 4.15, the lookahead distance for error computation $L_e$ is used as a parameter of the subsystem responsible for obtaining the errors and curvature. This was made as to explore the effect of this lookahead at constant and varying velocity scenarios, making it fixed at each simulation, as it is described on Chapter 6. However, there may exist an advantage in assigning some dependency of $L_e$ to $u$, in order to have an increasing anticipation as the velocity increases, and thus this option was also used for regulating $L_e$.

On the other hand, for the velocity regulation, it was always assumed that $L_k$ depended directly from $u$, increasing the lookahead distance with the velocity. Figure 4.16 illustrates how the designed solution will work when operating two distinct lookahead distances.

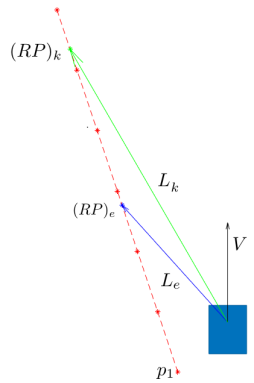


Figure 4.16: Usage of distinct lookaheads, $L_e$ (in blue) and $L_k$ (in green), for computation of the respective reference points, $RP_e$ and $RP_k$, on an path generated by interpolation of the waypoints (in red).

As stated before, $L_k$ was made directly dependant on $u$, and using a minimum and maximum values of $5$ and $20$ [m], and varying with $u$ as the following equation denotes:

$$L_k(u) = 0.8u \qquad \text{with} \quad L_k \in [5, 20] \qquad [\text{ m }] \tag{4.36}$$

On the other hand, four different cases were considered for the error lookahead, $L_e$: firstly, to be zero, which will mean that the no lookahead will be used, and the errors must be computed with the normal to the path passing through the vehicle's position (except for the L1 controller, which requires such lookahead by definition); assumed to be constant and equal to 5 [m]; thirdly, assumed to be constant and equal to 10 [m]; and lastly, equal to $0.4u$ but limited between 2 and 10 metres. The following equation summarizes these values for $L_e$:

$$L_e(u) = \{0,\ 5,\ 10,\ 0.4u \in [2, 10]\} \qquad [\text{ m }] \tag{4.37}$$

Therefore, two different central aspects will be varying during the simulations to obtain the results: the lateral control strategy employed, from the ones described on section 4.3; and the lookahead distance for error computation, as the above equation demonstrates. This is made in order to analyse the effects of the multiple options for each one of these parameters, and how they relate with each other. Naturally, a third aspect to vary is the track, but this will be analysed on the next chapter, describing the different courses used for simulation.

# Chapter 5

# Simulation Environment

As this thesis consists on a simulation-based development, with a model of the vehicle intended to drive autonomously, the environment where such simulations will occur assumes a central importance. Therefore, in this chapter, the final concepts and tools used for developing such environment are explained, in order to provide the utmost clarity for the next chapter, where the results of this work will be presented, described and commented.

This chapter focuses on three different aspects of the simulation: the tracks that were used for the car to follow and how these were processed from the initial data; how the performance of the path-tracking is evaluated using quantitative indexes; and lastly how the simulation environment works, which was developed in MATLAB/Simulink 2020a.

## 5.1 Tracks for Simulation and Testing

With some test courses having already been utilized to validate the controllers, like a simple line or circumference as it was seen on section 4.3, the evaluation of performance of these controllers with a simulation of the real car should naturally be done together with a simulation of a real track. FST Lisboa kindly provided three different tracks, with data about the position of the cones of the right and left sides. For simplicity, these tracks are simply referred as *Track 1*, *Track 2* or *Track 3*. These are presented at the end of this section as there are still some concepts that must be explained before, for which Track 1 is used as an example. It should be noted that Track 1 and Track 2 correspond to courses designed specifically for FST driverless competitions.

It is convenient to define a right and a left side of the track, in order to know the track's limits at each. The convention adopted was based in assuming that the vehicle always did the course in a counter-clockwise manner. With this, the waypoints corresponding to the outer cones would always be to the right of the centre of the track, and the inner would be to the left. Therefore, directly applying (4.11), with first-order finite differences to compute the required derivatives, the curvature profiles of each side of the track can be computed.

All the figures in this section follow this same convention regarding the sides of the track, as well as

a colour scheme: red will always represent the right (or outer) side of the track; blue will do the same regarding the left (or inner) side of the track; and pink will correspond to its centre, the computation of which will be explained shortly. Figure 5.1 illustrates this method for Track 1, showing the original waypoints obtained from the data in its left and right sides, as well as the respective curvature of each.

Direct computation of a centre line between these two tracks could be a possible method of obtaining

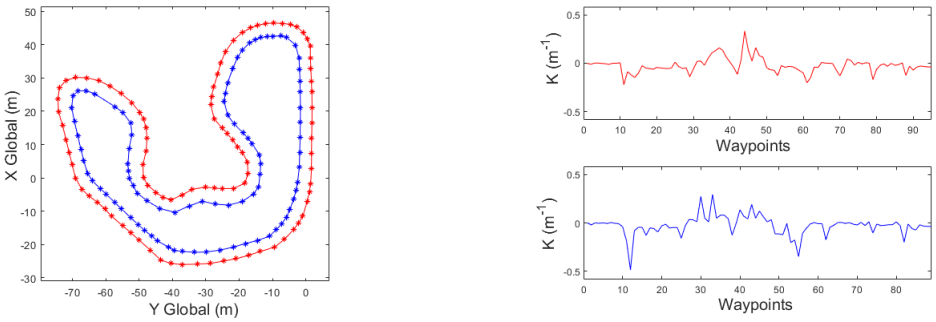


Figure 5.1: At the left, original waypoints of Track 1 and curvatures of its inner side, in blue, and outer side, in red. At the right, the corresponding curvatures of the left and right tracks, in blue and red respectively.

a reference path, but it was seen that would lead to an non-smooth reference, as the waypoints are spaced differently from each other. With this in mind, after spacing the original waypoints in order to eliminate those that were to close, these two sides of the tracks were smoothed using a smoother function, which generates a $C^2$-continuous path from the interpolation of this initial points, meaning that it has continuous derivatives of first and second order, at least. This was done in order to simulate how the path-planning algorithm would visualize the cones and generate a line to be followed. Therefore, the original number of waypoints of each track could be extended, and from these the centre of the track could be approximately defined, resulting in a smoother reference path. Applying this procedure for Track 1 and establishing a number of 200 waypoints, the smoothed version of Track 1 can be seen from figure 5.2.

Finally, the curvature along the track could be computed from the smooth reference track of figure

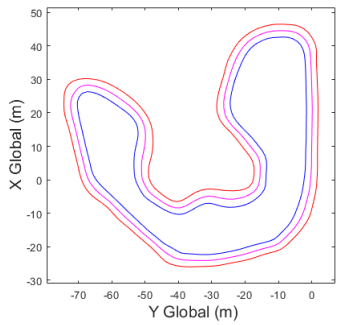


Figure 5.2: Smoothed right (red) and left (blue) sides of Track 1, and its centre (pink).

5.2, presented in pink. However, a filter must be taken into account as closely-placed waypoints could generate spikes in this curvature, caused by the derivatives required to equation 4.11, and a median filter was used in these calculations. Additionally, the distance of each waypoint from the centre line

to the limits of the track can be obtained as well, which is particularly useful for validation purposes. Therefore, the curvature and track limits for Track 1 are presented on figure 5.3.

The distance to the left side of the track, presented at the right in figure 5.3, is considered to be negative

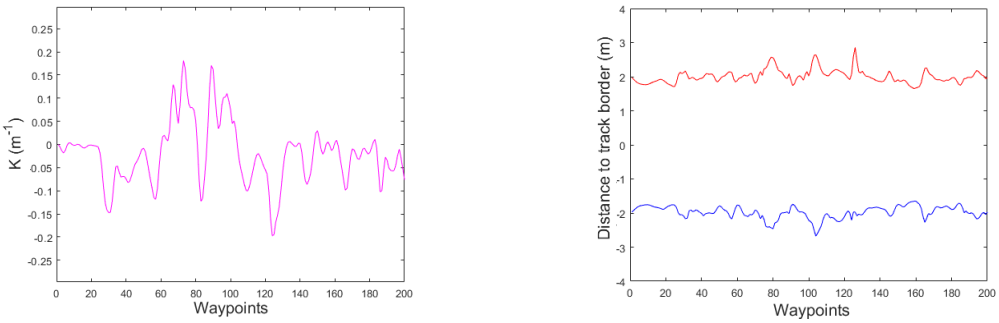


Figure 5.3: Curvature of the smoothed centre of Track 1 at the left, and distances to the right and left borders of this track, respectively in red and blue.

as it will be in the direction of the negative $y$-axis of the vehicle, as defined in 4.1.

Obtaining the curvature of the reference path is essential for the validation of the controllers, as (4.2) and (4.3) can be used for computing the cross-track and orientation errors. Additionally, these errors can be used to evaluate the performance of the car, namely the crosstrack error. Lastly, knowing the boundaries of the track along its extension is also very useful for evaluation, both visual and computational. The application of these two tools will be explained on the next section of this chapter

The procedure depicted for Track 1 was applied to the other two as well. For simplicity of the exposition of this work, only the final smoothed paths of Tracks 2 and 3 are respectively shown in figures 5.4 and 5.5, along with the curvature reference lines of each, as well as lateral limits.

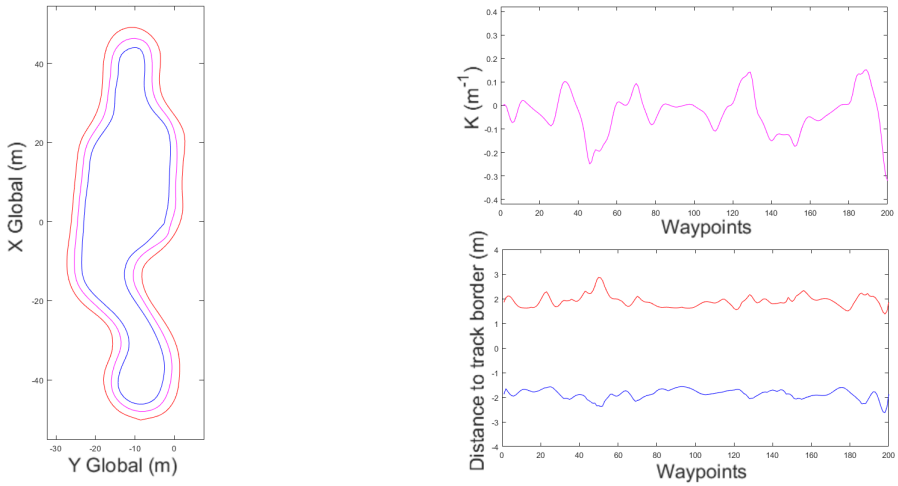


Figure 5.4: At the left, Track 2, following the same colour scheme for its right, left and centre courses. At the right-upper corner, respective curvature of the centre line, and in the lower, the lateral distances the borders of the right track, in red, and to left, in blue.

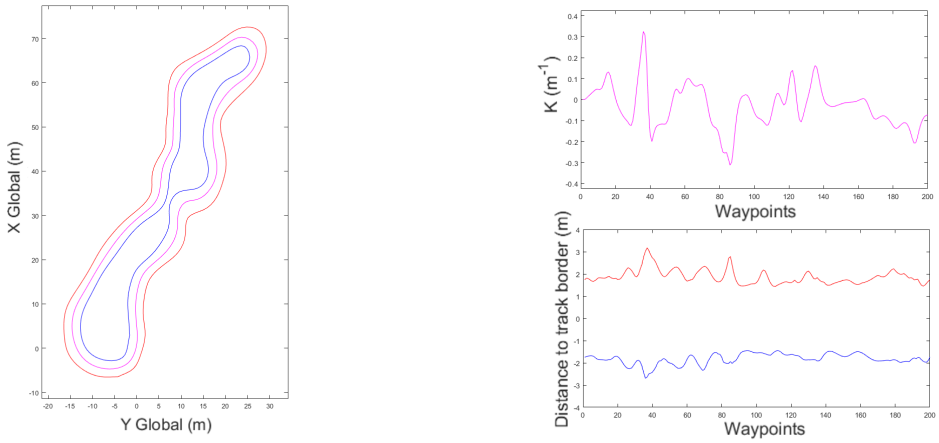


Figure 5.5: At the left, Track 3, with the limits of its sides in the lower-right, as well as the curvature of its centre course, in the upper-right.

## 5.2 Evaluation Metrics

As referred in the previous section, the information about the tracks is useful for evaluation the performance of the different controllers, as well the application of lookahead distances and velocity profiles. Using this, the methods to provide an objective and quantifiable evaluation for each combination of controller and lookahead are described in this section. These form a set of four performance metrics, in which only the last differs for the evaluation at constant speed and when it is varying.

The aspect to be considered for evaluation is the cross-track error that can be obtained from (4.2) and (4.3), which will be denoted by $e^1_{eval}$. Such error is fundamental in such evaluation as it measures the deviation at each instant that the car has from the reference path, which is the centre of the track.

The crosstrack error can then be computed every two waypoints, from the previous to the next, distinguishing a line from a arc of circumference if the curvature of the path between those two points has an absolute value below $0.002$ [m$^{-1}$]. In the case of a circumference, the centre of it is computed using the sign of the respective curvature. Therefore, if $W_i$ represents the previous waypoint and $W_{i+1}$ corresponds to the waypoint the vehicle is currently targeting, and $R_{i,i+1} = \frac{1}{K_{i,i+1}}$ is the radius of the reference path between these, $e^1_{eval}$ can be defined by:

$$e^1_{eval} = \frac{D_{i,i+1} \times D_{i,P}}{\parallel D_{i,i+1} \parallel} \quad \text{, if } |K_{i,i+1}| < 0.002 \qquad [\text{ m }] \tag{5.1a}$$

$$e^1_{eval} = \text{sign}(R_{i,i+1}) \cdot (\overline{PC} - |R_{i,i+1}|) \quad \text{, otherwise} \qquad [\text{ m }] \tag{5.1b}$$

in which $D_{i,i+1} = W_{i+1} - W_i$ is the current displacement vector from point $W_i$ to $W_{i+1}$, $P$ is the position of the car in the global frame and $C$ is the centre of the arc of circumference defined by these two waypoints, resulting $PC = P - C$.

Complementing this, an additional cross-track error was used for evaluation consisting on the interpolation of the two closest waypoints behind the vehicle and two in front with a second-order polynomial, in the frame of the car, obtaining another cross-track error from (4.6). If $p_{eval}$ is the point that results from the interception of the local polynomial with the normal to it passing through $P$, which will be the closest

to the vehicle, this second cross-track error used for evaluation, $e^2_{eval}$, can be obtained from:

$$e^2_{eval} = \frac{(p)_{eval} \times T_{eval}}{\| T_{eval} \|} \qquad [\text{ m }] \tag{5.2}$$

where $T_{eval}$ is the tangent of the local evaluation polynomial at $p_{eval}$.

This second way of defining the cross-track error for assessing the performance may seem redundant with the first, but one must remember that the path-following algorithm is computing the errors from an analogous interpolation of the waypoints, whether using a lookahead or not, and that the resulting polynomial could have a different curvature than that of the track at the same point. As such, both $e^1_{eval}$ and $e^2_{eval}$ should be taken into account when evaluating the performance of each controller and related aspects to it, and the root mean square (RMS) of both consists on the two first evaluation metrics, $m^1_{eval}$ and $m^2_{eval}$ (m).

The third metric is related to the steering of the vehicle, as it should be smooth in order to drive the car gently and not have undesirable oscillation, which could be harmful for the components of the car. Therefore, such oscillation must be defined: by smoothing the obtained values from the steering values $\delta$ with a median filter, a softer curve for the steering is obtained, denoted by $\delta_{filt}$. Therefore, by defining $m^3_{eval}$ as the RMS of $e_\delta = \delta - \delta_{filt}$ [ º ], such oscillation can be objectively measured and quantified, consisting on the fourth and last metric used for evaluation. $m^3_{eval}$ will be presented in degrees to provide a more familiar representation

Finally, the fourth evaluation index is different for the performance evaluation at constant speed, on section 6.1, and for the ones with varying velocity, on 6.2. As for the first case, the average velocity will be taken, as it will provide information about how the velocity controller is acting for constant speed in situations were both the vehicle model and track simulate the real one. The velocity used is the vectorial norm of $V$, $\overline{V}$, instead of only its longitudinal component $u$, as the car could use some slippage in the curves for its advantage, which is indeed expected in race scenarios. Therefore, for these tests, it is considered $m^4_{const} = \mu(\overline{V})$ [m/s]. On the other hand, for tests with the speed of the vehicle being regulated from the curvature at a lookahead point, as explained in 4.4 and 4.5, taking the mean of the velocity as an indicator looses its meaning in evaluating performance of the lateral controllers. Instead, the total time to complete a the course is taken, as to provide a basis for a real competition scenario. Thus, for the controllers tested at varying velocity, the third metric is $m^4_{var} = t_{max}$ [s], but the average velocity was included as well in the tables of results, namely to provide some insight to how the lookahead distances are varying, for these specific cases, despite not being used as an evaluation metric for these tests.

The evaluation metrics for tests at constant and at varying velocity are summarized in tables 5.1 and 5.2, respectively, corresponding to sections 6.1 and 6.2. The subscripts $const$ and $var$ was added to all these parameters, in order to distinguish them for each of these scenarios.

Table 5.1: Evaluation metrics used for tests at constant velocity.

| Evaluation metric | Meaning | units |
|---|---|---|
| $m^1_{const}$ | $(\text{RMS})_{e^1_{eval}}$ | [ m ] |
| $m^2_{const}$ | $(\text{RMS})_{e^2_{eval}}$ | [ m ] |
| $m^3_{const}$ | $(\text{RMS})_{e_\delta}$ | [ º ] |
| $m^4_{const}$ | $\mu(\overline{V})$ | [ m·s$^{-1}$ ] |

Table 5.2: Evaluation metrics used for tests at varying velocity.

| Evaluation metric | Meaning | units |
|---|---|---|
| $m^1_{var}$ | $(\text{RMS})_{e^1_{eval}}$ | [ m ] |
| $m^2_{var}$ | $(\text{RMS})_{e^2_{eval}}$ | [ m ] |
| $m^3_{var}$ | $(\text{RMS})_{e_\delta}$ | [ º ] |
| $m^4_{var}$ | $t_{max}$ | [ s ] |

Two brief notes should be provided in this section regarding the performance evaluation. Firstly, a simple safeguard for situations where the car steers in a way that there are no reference points can be obtained from (4.4), in case of no lookahead distance ($L_e = 0$ [m]), or (4.7), if one is being used. In this case, the algorithm will consider just the adjacent and next waypoint and compute the errors assuming these are connected by a line-segment, using (4.2). However, this will fire a flag, indicating clearly that the car had been driven it a way that it should not. The second note is also related with these situations: in the extreme scenarios where the path-tracking algorithm not only looses the reference path but also cannot compute the errors assuming it as a line, the simulation is stopped, and it is considered that the vehicle did not finish (*DNF*) the course. Similarly, if the vehicle strays away from the reference path further than what the boundaries of the track at the corresponding point allow, plus a tolerance of 1 metre to account for additional curvature of the side-track, the simulation is terminated, indicating *DNF* as well. Despite not consisting in actual values, both the flags and indication of *DNF* complement the performance evaluation metrics with essential indication of the performance of the controller and path-following algorithm.

## 5.3 Simulator Description

At this point, having already shown the different tracks in which the multiple path-following strategies and controllers will be tested, as well as the tools that will be applied in evaluating and comparing them, it is now time for a brief description of the designed simulator for such process. The structure of the simulation is based on the one presented on figure 4.15, with additional components related to the control of

the simulation and visualization. The developed Simulink model is presented now:

As can be seen from 5.6, the structure of the model is indeed similar, but the part that detects the

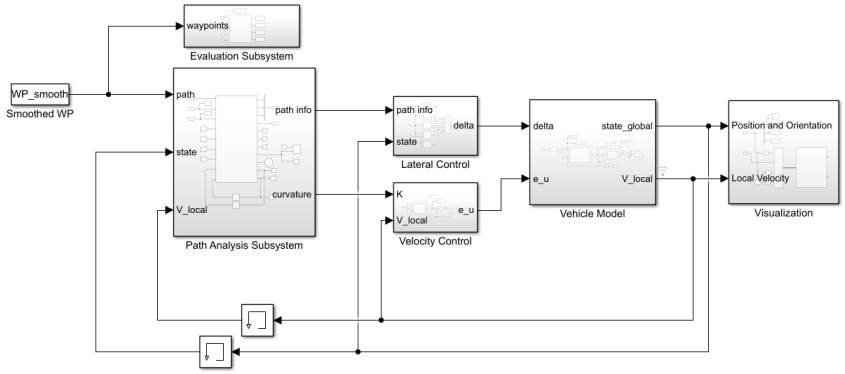


Figure 5.6: Simulink model used for simulations.

cones and forming the reference path and the one computes the errors have been combined in the *Path Analysis* subsystem. The remaining subsystems on the central model are then: the *Lateral Control* subsystem, where the different controllers are selected either from control directly in the simulation, or by the provided script; the velocity controller block, which is an analogous subsystem but allows to choose from constant velocity (in which it only regulates it), or varying it with the profile explained on section 4.4; the vehicle dynamic model, as described on section 3.1, including the kinematic equations as well, in order to place it in a global frame; lastly, the visualization subsystem. This last block allows to see the simulation results as it evolves, as its outputs are the $X$ and $Y$ positions in the global coordinate system, as well as the orientation $\psi$ of the vehicle. The sampling time was $T_s = 0.1$ [s] for the path-following algorithm and controllers was used, as explained in the end of section 3.1, and $0.01$ [s] for the remaining blocks, the vehicle model and the visualization ones. Lastly, a brief explanation of the simulation is in order. The parameters of the car, as well as the scaled gains for the controllers are loaded into the simulation running the main script provided. Once this is done, there are four different in-simulation controls: the choice of control strategy, between those described in section 4.3; the selection of the lookahead distance for this controller, allowing it to be zero or automatic, being dependent on the longitudinal speed; the choice of velocity profile, between the one exposed or a constant value; and the last parameter is the constant velocity, which naturally is only activated the simulation if the the constant value option has been selected in the previous switch. These four-different controls allow for a multitude of different varying parameters, which will ultimately produce the results of this thesis, which are presented on the next chapter.

# Chapter 6

# Results

In this chapter, the multiple controllers will be evaluated with data of real tracks, as described in section 5.1, with the model intended to simulate an actual Formula Student vehicle, explained in Chapter 3. Three central aspects will be changing for these simulations: the controller used; the lookahead distance; and the velocity of the vehicle. The variation of the velocity is explored on the second section of this chapter, 6.2, as the first one presents the results with fixed velocities $u = 5$ and $u = 10$ [m/s], in order to attest the tracking capacity of the controllers before introducing speed regulation. Section 6.2 explores this part, including control of the velocity with the references being generated from the curvature of the path and an appropriate velocity controller, as depicted on section 4.4. Lastly, the discussion of the obtained results is provided in the last section, 6.3. Due to the extensive number of simulations, such results are provided with appropriate tables, and the figures from the simulations at varying velocity are provided in the second appendix, B. The choice not to include the ones at constant velocity was made since these contemplate only a step in the process of providing a solution, and the tests with varying velocity assume higher significance in developing a path-following algorithm for the current problem.
It should be stressed that all the combinations that are present in the several tables correspond to simulations where the vehicle managed to finish the course without crossing the borders of the track further than 1 metre, regardless of the cross-track error or flags activated, therefore being valid. Before presenting the results, an additional note must be made: for the case of the linear controllers, the required matrices were tuned for these simplified models used for validation. However, new values for such matrices must be defined for the simulation of the real model, which were obtained from fine-tuning with the simulation of the real model, and are presented next, in their respective SI units:

- 2 state LQR controller: $\mathbf{Q}^{2S} = \text{diag}\{20, 1\}$, $\mathbf{R}^{2S} = \{100\}$
- 3 state LQG controller: $\mathbf{Q}^{3S} = \text{diag}\{10, 1.0.1\}$, $\mathbf{R}^{3S} = \{50\}$, $\mathbf{Q}_0^{3S} = \text{diag}\{0.5, 0.01, 5\}$, $\mathbf{R}_0^{3S} = \text{diag}\{0.001, 0.5\}$
- 4 state LQG controller: $\mathbf{Q}^{4S} = \text{diag}\{10, 4, 1, 1\}$, $\mathbf{R}^{4S} = \{100\}$, $\mathbf{Q}_0^{4S} = \text{diag}\{0.5, 0.1, 0.1, 1\}$, $\mathbf{R}_0^{4S} = \text{diag}\{0.01, 0.01\}$
- Velocity controller: $\mathbf{Q}^{long} = \text{diag}\{100, 10\}$, $\mathbf{R}^{long} = \{10\}$

## 6.1 Performance Evaluation at Constant Velocity

As stated in the beginning of this chapter, in this section the different controllers will be evaluated at constant velocities of 5 and 10 [m/s]. Track 1 of figure 5.2 was the one selected, as it was considered the simplest, taking into account the remaining two had significantly more difficult curves, since this section aims to provide some insight about the behaviour and performance of these controllers when the velocity is fixed, specially at the corners and curves of the track. Lastly, it is important to note that only the scenarios of no lookahead distance and a fixed one are explored in this section, as it was stated that if it was made as varying, it would depend on the velocity.

The results are summarized on the next two tables, each one representing a different constant velocity $u$, and the controllers and lookahead distances are also identified accordingly, using zero to denote if none is being applied. Naturally, since the L1 controller requires such distance by its definition, only the cases for $L_e = 5$ and $10$ [m] are used in the simulations.

- Performance at constant velocity $u = 5$ [m/s]:

Table 6.1: Performance evaluation at $u = 5$ [ m/s$^{-1}$ ] for Track 1.

| Controller<br>- | $L_e$<br>[ m ] | $m^1_{const}$<br>[ m ] | $m^2_{const}$<br>[ m ] | $m^3_{const}$<br>[ º ] | $m^4_{const}$<br>[ m·s$^{-1}$ ] | Obs.<br>- |
|---|---|---|---|---|---|---|
| Stanley | 0 | 0.45 | 0.45 | 1.06 | 4.91 | - |
| Stanley | 5 | 0.57 | 0.56 | 0.06 | 4.91 | - |
| Stanley | 10 | 0.77 | 0.77 | 0.24 | 4.91 | - |
| L1 | 5 | 0.25 | 0.25 | 0.03 | 4.91 | - |
| L1 | 10 | 0.76 | 0.76 | 0.02 | 4.91 | - |
| LQR (2 states) | 0 | 0.28 | 0.28 | 0.95 | 4.91 | - |
| LQR (2 states) | 5 | 0.08 | 0.09 | 0.33 | 4.91 | - |
| LQR (2 states) | 10 | 1.11 | 1.11 | 1.60 | 4.92 | - |
| LQG (3 states) | 0 | 0.45 | 0.45 | 2.73 | 4.91 | - |
| LQG (3 states) | 5 | 0.23 | 0.24 | 3.20 | 4.91 | - |
| LQG (3 states) | 10 | 1.31 | 1.31 | 6.60 | 4.92 | - |
| LQG (4 states) | 0 | 0.52 | 0.52 | 0.99 | 4.91 | - |
| LQG (4 states) | 5 | 0.20 | 0.20 | 0.48 | 4.91 | - |
| LQG (4 states) | 10 | 1.10 | 1.10 | 1.14 | 4.92 | - |

- Performance at constant velocity $u = 10$ [m/s]:

Table 6.2: Performance evaluation at $u = 10$ [ m/s$^{-1}$ ] for Track 1.

| Controller<br>- | $L_e$<br>[ m ] | $m^1_{const}$<br>[ m ] | $m^2_{const}$<br>[ m ] | $m^3_{const}$<br>[ º ] | $m^4_{const}$<br>[ m·s$^{-1}$ ] | Obs.<br>- |
|---|---|---|---|---|---|---|
| Stanley | 0 | 0.59 | 0.59 | 0.84 | 9.65 | - |
| Stanley | 5 | 0.50 | 0.50 | 0.16 | 9.62 | - |
| Stanley | 10 | 0.70 | 0.70 | 0.65 | 9.62 | - |
| L1 | 5 | 0.20 | 0.19 | 0.10 | 9.63 | - |
| L1 | 10 | 0.65 | 0.65 | 0.05 | 9.62 | - |
| LQR (2 states) | 0 | - | - | - | - | DNF |
| LQR (2 states) | 5 | 0.10 | 0.11 | 0.62 | 9.64 | - |
| LQR (2 states) | 10 | 1.15 | 1.15 | 2.23 | 9.67 | - |
| LQG (3 states) | 0 | 0.57 | 0.57 | 2.21 | 9.65 | - |
| LQG (3 states) | 5 | 0.50 | 0.50 | 2.25 | 9.65 | - |
| LQG (3 states) | 10 | 1.60 | 1.60 | 4.28 | 9.68 | - |
| LQG (4 states) | 0 | 0.67 | 0.67 | 0.70 | 9.65 | - |
| LQG (4 states) | 5 | 0.15 | 0.15 | 0.40 | 9.64 | - |
| LQG (4 states) | 10 | 0.87 | 0.87 | 0.94 | 9.66 | - |

## 6.2 Performance Evaluation at Varying Velocity

Having shown the results from the simulations at constant velocities of $u = 5$ [m/s] and $u = 10$ [m/s], respectively provided in tables 6.1 and 6.2, in this section the ones obtained from having speed regulation are presented. The reference for the longitudinal velocity is obtained from the curvature of the reference point provided by the lookahead for curvature, $L_k$, and adjusted accordingly, as (4.32) implies. Regarding the controllers, the same remark regarding the L1 one applies, as it requires a lookahead distance to function at a minimal acceptance.

The order of the results of each track is of interest since these manifest different complexities, and such order was chosen as to mirror this. Therefore, tables 6.3, 6.4 and 6.5, representing Tracks 1, 2 and 3 respectively, can be regarded as an increase of difficulty in the track. This is detailed in the following section, were the results are discussed.

Table 6.3: Performance evaluation with varying velocity for Track 1.

| Controller<br>- | $L_e$<br>[ m ] | $m^1_{var}$<br>[ m ] | $m^2_{var}$<br>[ m ] | $m^3_{var}$<br>[ º ] | $m^4_{var}$<br>[ s ] | $\mu(\overline{V})$<br>[m/s] | Obs.<br>- |
|---|---|---|---|---|---|---|---|
| Stanley | 0 | 0.63 | 0.63 | 3.14 | 23.5 | 13.66 | - |
| Stanley | 5 | 0.47 | 0.46 | 0.62 | 22.5 | 13.26 | - |
| Stanley | 10 | 0.68 | 0.68 | 1.13 | 22.9 | 13.01 | - |
| Stanley | $0.4u$ | 0.47 | 0.47 | 0.57 | 22.5 | 13.25 | - |
| L1 | 5 | 0.16 | 0.15 | 0.12 | 22.8 | 13.33 | - |
| L1 | 10 | 0.62 | 0.61 | 0.08 | 22.2 | 13.38 | - |
| L1 | $0.4u$ | 0.18 | 0.18 | 0.24 | 22.5 | 13.5 | - |
| LQR (2 states) | 0 | 0.38 | 0.39 | 5.16 | 23.7 | 13.33 | - |
| LQR (2 states) | 5 | 0.15 | 0.15 | 3.83 | 22.9 | 13.55 | - |
| LQR (2 states) | 10 | 1.25 | 1.25 | 7.19 | 26.8 | 12.50 | - |
| LQR (2 states) | $0.4u$ | 0.27 | 0.27 | 4.13 | 23.8 | 13.13 | - |
| LQG (3 states) | 0 | - | - | - | - | - | DNF |
| LQG (3 states) | 5 | 0.86 | 0.86 | 3.49 | 23.9 | 13.59 | - |
| LQG (3 states) | 10 | - | - | - | - | - | DNF |
| LQG (3 states) | $0.4u$ | 0.99 | 0.99 | 3.38 | 24.6 | 13.31 | - |
| LQG (4 states) | 0 | - | - | - | - | - | DNF |
| LQG (4 states) | 5 | 0.11 | 0.11 | 2.88 | 22.8 | 13.48 | - |
| LQG (4 states) | 10 | 0.80 | 0.80 | 2.95 | 25.0 | 12.94 | - |
| LQG (4 states) | $0.4u$ | 0.11 | 0.11 | 2.85 | 23.0 | 13.38 | - |

Table 6.4: Performance evaluation with varying velocity for Track 2.

| Controller<br>- | $L_e$<br>[ m ] | $m^1_{var}$<br>[ m ] | $m^2_{var}$<br>[ m ] | $m^3_{var}$<br>[ º ] | $m^4_{var}$<br>[ s ] | $\mu(\overline{V})$<br>[m/s] | Obs.<br>- |
|---|---|---|---|---|---|---|---|
| Stanley | 0 | 0.71 | 0.71 | 2.64 | 21.9 | 10.49 | - |
| Stanley | 5 | 0.58 | 0.58 | 1.11 | 20.6 | 10.13 | - |
| Stanley | 10 | - | - | - | - | - | DNF |
| Stanley | $0.4u$ | 0.45 | 0.45 | 0.87 | 20.5 | 10.19 | - |
| L1 | 5 | 0.27 | 0.28 | 0.31 | 20.6 | 10.45 | |
| L1 | 10 | - | - | - | - | - | DNF |
| L1 | $0.4u$ | 0.19 | 0.20 | 0.25 | 20.6 | 10.43 | - |
| LQR (2 states) | 0 | - | - | - | - | - | DNF |
| LQR (2 states) | 5 | 0.14 | 0.15 | 4.48 | 21.2 | 10.35 | - |
| LQR (2 states) | 10 | - | - | - | - | - | DNF |
| LQR (2 states) | $0.4u$ | 0.25 | 0.26 | 4.25 | 21.2 | 10.36 | |
| LQG (3 states) | 0 | 0.84 | 0.84 | 6.03 | 21.8 | 10.63 | - |
| LQG (3 states) | 5 | 0.89 | 0.89 | 3.44 | 21.8 | 10.66 | - |
| LQG (3 states) | 10 | - | - | - | - | - | DNF |
| LQG (3 states) | $0.4u$ | 0.96 | 0.96 | 6.42 | 21.9 | 10.77 | - |
| LQG (4 states) | 0 | 0.65 | 0.65 | 2.30 | 21.7 | 10.57 | - |
| LQG (4 states) | 5 | 0.17 | 0.17 | 1.97 | 20.9 | 10.43 | - |
| LQG (4 states) | 10 | - | - | - | - | - | DNF |
| LQG (4 states) | $0.4u$ | 0.14 | 0.14 | 2.75 | 20.9 | 10.46 | - |

Table 6.5: Performance evaluation with varying velocity for Track 3.

| Controller<br>- | $L_e$<br>[ m ] | $m^1_{var}$<br>[ m ] | $m^2_{var}$<br>[ m ] | $m^3_{var}$<br>[ º ] | $m^4_{var}$<br>[ s ] | $\mu(\overline{V})$<br>[m/w] | Obs.<br>- |
|---|---|---|---|---|---|---|---|
| Stanley | 0 | - | - | - | - | - | DNF |
| Stanley | 5 | 0.56 | 0.56 | 1.23 | 19.7 | 9.04 | - |
| Stanley | 10 | - | - | - | - | - | DNF |
| Stanley | $0.4u$ | 0.46 | 0.48 | 1.14 | 19.5 | 9.20 | - |
| L1 | 5 | 0.37 | 0.37 | 0.56 | 19.6 | 9.37 | - |
| L1 | 10 | - | - | - | - | - | DNF |
| L1 | $0.4u$ | 0.25 | 0.27 | 0.54 | 19.4 | 9.52 | - |
| LQR (2 states) | 0 | - | - | - | - | | DNF |
| LQR (2 states) | 5 | 0.20 | 0.22 | 4.13 | 19.9 | 9.58 | - |
| LQR (2 states) | 10 | - | - | - | - | - | DNF |
| LQR (2 states) | $0.4u$ | 0.23 | 0.24 | 3.27 | 20.2 | 9.40 | - |
| LQG (3 states) | 0 | - | - | - | - | - | DNF |
| LQG (3 states) | 5 | 0.92 | 0.91 | 5.36 | 20.2 | 9.97 | - |
| LQG (3 states) | 10 | - | - | - | - | - | DNF |
| LQG (3 states) | $0.4u$ | 0.90 | 0.90 | 5.86 | 20.2 | 9.97 | - |
| LQG (4 states) | 0 | - | - | - | - | - | DNF |
| LQG (4 states) | 5 | 0.24 | 0.25 | 2.31 | 19.7 | 9.58 | - |
| LQG (4 states) | 10 | 0.92 | 0.92 | 4.40 | 20.4 | 9.76 | - |
| LQG (4 states) | $0.4u$ | 0.23 | 0.24 | 2.70 | 19.4 | 9.65 | - |

## 6.3 Discussion

In section 6.1, the evaluation metrics for each combination of controller and lookahead for Track 1 were computed and shown, and in 6.2 the same was made, accounting this time for different tracks and varying velocity. In this section, the last of its chapter, these results are described, explained and commented, in order to assess the validity of the different possible solutions for the path-following problem, and particular emphasis is given to Track 1, being the one from a driverless FST competition, as stated in the beginning of section 5.1.

Such discussion starts with tables 6.1 and 6.2, as to provide some insight about the behaviour of the controllers with different lookahead distances, and forming a basis of comparison for the following results as well. Afterwards, the values for the simulations at varying velocity will be described for each track, corresponding to tables 6.3, 6.4 and 6.5, complemented by an analysis of the most problematic zones of the tracks. Lastly, final considerations about the different control strategies applied as well as the solutions for the path-following problem are provided.

### 6.3.1 Discussion of results at constant velocity

Beginning with table 6.1, the first aspect to be highlighted is that it indeed forms an appropriate starting point for the analysis of both the lateral control and the velocity. Regarding the steering of the vehicle, it can be seen that every combination of controller and lookahead managed to finish the course without activating the flags referred in section 5.2, meaning the path-tracking algorithm effectively steered the vehicle as to not loose the reference path. This is expected as 5 [m/s], or 18 [km/h] if one has more familiarity with this representation, is a relatively low-speed, but, as said, it allows to form a basis of comparison, either between the controllers or regarding the lookahead used. Similarly, regarding the velocity control, table 6.1 clearly indicates the mean of the velocity is virtually the same in all the simulations, assuring that the regulation at a constant value is being successful. Lastly, as the differences between $m^1_{const}$ and $m^2_{const}$, reflecting the crosstrack errors used in evaluation, are not significant, only the first is to be considered in this analysis. Despite inconvenient, this almost absolute similarity between the error obtained from knowing the curvature of the path, $e^1_{eval}$, and the one obtained through polynomial interpolation, $e^2_{eval}$, reinforces the results obtained, and neither should have be disregarded when obtaining results.

These initial considerations allow the reduction of the degrees of freedom expressed by the evaluation metrics, focusing the analysis on the values of $m^1_{const}$ and $m^3_{const}$, which represent the precision of the path-following and the effects it has on the steering of the vehicle.

Going in detail now to the values of $m^1_{const}$, the effects of the lookahead distance, fixed in these simulations at constant velocity, are notorious: with the exception of the Stanley controller, all the remaining ones have the lowest RMS of the crosstrack error for $L_e = 5$ [m], and by margins of more than 20 centimetres to the closest value for each controller. For the Stanley, the best result is for no lookahead at all, suggesting that the inclusion of such as a modification could actually provide worse tracking in general. However, 6.1 also highlights another fundamental aspect regarding the lookahead distance: $L_e = 10$

[m] provides the worst result for every controller, clearly indicating, together with the results from $L_e = 5$ [m], that the length of the lookahead, if fixed, should be carefully chosen. This is a topic to keep in mind throughout this section, as it is central for this dissertation.

Focusing now on $m^3_{const}$, it can be established that the actuation benefits from the usage a lookahead, if its value is appropriate, as once again the lowest value for the RMS of $e_\delta$, as defined in 5.2, is the lowest for $L_e = 5$ [m] for each controller (except for the L1, but the difference is 0.01 [ º ]). On the other hand, $m^3_{const}$ tends to be larger when $L_e = 10$ [m], resulting in more oscillation in the steering input, once again pointing to the idea that attention should be taken when choosing a fixed value for the lookahead distance.

Regarding absolute values for these two metrics and all combinations of controller/lookahead, the lowest value for $m^1_{const}$ is for the LQR of two states and $L_e = 5$ [m], suggesting some absence of dynamical effects at low-speeds, privileging a simple controller that accounts only for the errors as feedbacks. The L1 controller has the smoothest actuation, whether for $L_e$ equal to 5 or 10 metres. One must remember that this controller uses only one error, $\eta$, being different from the two that all the remaining controllers require, $e_y$ and $e_\psi$. Parallelly, the LQG controller of 3 states has the worse value for this same metric, despite having decent tracking of the path, possibly indicting a wrong tuning of its $\mathbf{Q}$ and $\mathbf{R}$ matrices.

Advancing to the results obtained for the $u = 10$ [m·s$^{-1}$], provided in table 6.2, the same remarks regarding the similarities between $m^1_{const}$ and $m^2_{const}$ are in order. Analogously, despite having slightly larger variations, the mean of the velocity given by $m^4_{const}$ is virtually constant for every simulation scenario at this speed. This allows once again to focus in the inspection of $m^1_{const}$ and $m^3_{const}$.

Starting with the crosstrack error related metric, the data of 6.2 reinforces the suggestions regarding the lookahead distance taken from the analysis of the previous table: the minimum $m^1_{const}$ for each controller is when $L_e = 5$ [m], and the maximum for $L_e = 10$ [m]. Similarly, the lowest value for all combinations results from que LQR controller with 2 states, with $m^1_{const} = 0.10$ [m]. However, this same controller introduces a fundamental difference from the simulations at $u = 5$ [m·s$^{-1}$]: without a lookahead, this controller does not manage to finish the course. This is an interesting result, clearly denoting that this LQR controller benefits specially of a properly defined $L_e$.

Lastly, regarding the smoothness of the steering, conclusions are similar: the L1 controller provides the lowest value for $m^3_{const}$, while the 3-state LQG provides the worst ones, which again suggests that the tuning of this last controller could have been better.

This concludes the analysis of the results at fixed velocities. A similar process will be done with the results when the speed is varying, before presenting the final considerations for the multiple control strategies employed.

### 6.3.2 Discussion of results at varying velocity

- Results from Track 1:

  Initiating this topic with the immediate analysis of table 6.3, it can be once again stated that $m^1_{var}$ is virtually equal to $m^2_{var}$ for all combinations (which will be the case for every track). Another aspect that immediately asserts itself is that there are few combinations that cannot finish the course, occurring for $L_e = 0$ and $L_e = 10$ [m].

  Regarding the crosstrack error metric, $m^1_{var}$, all the combinations of controller/lookahead are significantly lower than 1 metre, except the ones for fixed $L_e = 10$ [m]. The LQG with 4 states as feedback and $L_e = 5$ [m] has the lowest value for $m^1_{var}$, but this but closely followed by the 2-state LQR and the L1, when $L_e = 5$ [m] and $L_e = 0.4u$.

  When it comes to $m^3_{var}$, L1 demonstrates once more it provides smoother actuation, differing from the remaining controllers by a large margin. On the other hand, the 3-state LQG and 2-state LQR present the worse results in this matter, while the remaining ones have acceptable values for it, specially for $L_e = 5$ [m] or $L_e = 0.4u$.

  The new evaluation metric, $m^4_{var}$, representing the time that each simulation took to complete the course, also introduces a new degree of freedom in this analysis: from all the combinations, the different combinations of controller and lookahead distance took between 22.2 and 26.8seconds, providing an interval of more than 4 seconds for the fastest and slowest combination. In this area, the L1 and Stanley controllers have the shortest times, specially for $L_e = 5$ [m] and $L_e = 0.4u$.

  For this track, the best results weighting all these metrics are for $L_e = 5$ [m] or $L_e = 0.4u$, as these tend to reduce the crosstrack error with a smoother actuation for each controller, and the considerations regarding the track time have already been provided.

- Results from Track 2:

  Moving on to the results from the second track, in table 6.4, the reason that it was considered a course more difficult than Track 1 is evident: none of the controllers when combined with a lookahead of 10 metres manages to finish the course. Once again, this underlines the attention needed when choosing a fixed lookahead. Regarding the maximum and minimum of each metrics, the conclusions are similar to the ones provided for the previous track: the LQR with 2 states and the 4-state LQG with 5 metres lookahead or $L_e = 5$ [m] have the highest precision in tracking the path; the L1 controller provides with the smoothest steering by a large margin with both lookaheads, naturally excluding the one that did not finish the track; and the Stanley controller is the fastest. However, the 4-state LQG stands out for simulations where it is combined with a 5 metre of varying lookahead, being close to the 2-state LQR in $m^1_{var}$ while having a decent actuation smoothness and a maximum time to complete the course less that 0.4 seconds from the best.

  One last remark when it comes to the results from table 6.4 should be made: as can be seen, for each controller, the values when the lookahead is of 5 metres and when it is automatically regulated are quite similar. As $L_e = 0.4u$ for these cases and the mean of the velocity is around 10 [m/s], this could mean that $L_e$, when varying, was kept close to its minimum value.

- Results from Track 3:

  This track presents itself as the most complex, with 8 of the 19 combinations of controller and lookahead distances not finishing it. Furthermore, the need of anticipation is evidenced in this track, as none of the combinations where there was no lookahead used managed to finish the course. Besides this, once again the 2-state LQR provides the lowest RMS of the cross-track error, L1 shows the highest smoothness in steering, both of these using a lookahead of 5 metres, but closely followed by $L_e = 0.4u$ in each. In fact, the results for the simulations for this course when the $L_e$ is fixed at 5 metres are virtually equal to the ones obtained when it is varying, suggesting once more that it is being kept close to this value. This also corresponds to the best solution for each controller, as it can be verified that $L_e = 10$ [m] continues to provide worse results, and taking into account what has been stated regarding the absence of lookahead. Lastly, it should be noted that the LQG with 4 states manages to have good results in the three metrics under evaluation, in spite of not being the best in any, and only being surpassed by the L1 controller.

  With nothing more to add regarding the analysis of the results, the final considerations about the different control strategies is presented next.

- Problematic zones of the tracks:

  Before presenting the final considerations, it is important to examine the reason why some of the combinations did not finish the course in specific simulations. In most of the cases, it was simply caused by an excessive value for the lookahead distance, causing the vehicle to cut between the limits or to start the curve with too much anticipation, crossing the borders as well. Figure 6.1 illustrates this first scenario in Track 1 for the Stanley controller with $L_e = 10$ [m] and varying velocity. In most situations, it still allowed for the car to complete the track, despite being considered "DNF". However, this was not the case for all simulations, and in tracks 2 and 3 there were some combinations that could not complete the track at all. These situations are of extreme importance as they correspond to simulations where the vehicle was driven in a way that resulted in absolute loss of the reference path, and examples are provided on figure 6.2.

  As the objective is to find a solution that provides better results in general, the loss of reference should be avoided at maximum, and the vehicle should be steered and accelerated in a way that takes this into account.

### 6.3.3 Final considerations

The provided analysis from the results at constant and varying velocities allows for proper and thorough considerations to be taken regarding the different control and path-following strategies employed. The results from simulations at constant speeds made possible to form a ground-work for comparing the performance of these strategies, as all but one combinations completed the track, and it introduced some of the differences between the controllers. The first and fundamental take-away from these tests is that all controllers benefit from a lookahead distance, provided it is in some form adequate to the track (only

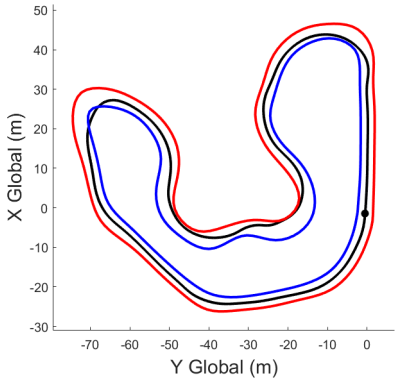


Figure 6.1: Results for the Stanley controller with $L_e = 10$ [m] for Track 1 (black). The boundaries of the track are presented in blue and red, while the green line is the reference path.

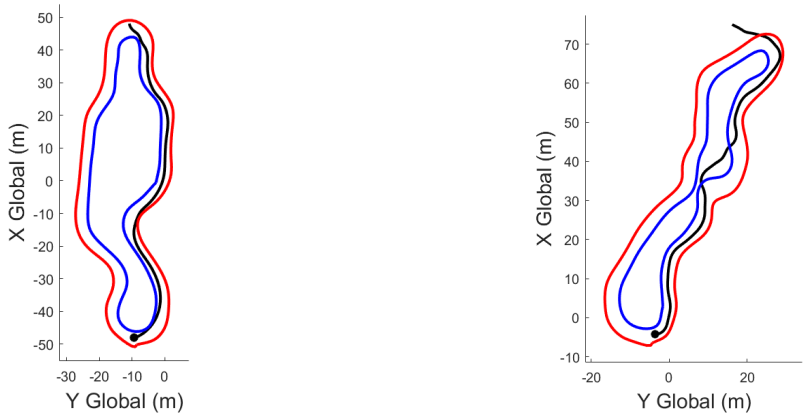


Figure 6.2: At the left, the evident loss of the reference path for the Stanley controller with $L_e = 10$ [m] in Track 2, and an analogous result for the 2-state LQR without a lookahead in Track 3, both represented by the black line in the respective plot. The boundaries of the track are presented in blue and red, while the green line is the reference path.

considering fixed values for now), since the crosstrack errors tend to be lower and the steering smoother. Only the Stanley presents a lower RMS of the crosstrack error when $u = 5$ [m/s] without a lookahead, but a significantly higher steering oscillation, when compared with the simulation for this same controller with $L_e = 5$ [m].

The simulations where the speed is adjusted accounting for the curvature accentuate this hypothesis since the best values for most of the metrics appear when $L_e = 5$ [m], for all the controllers in every track. Varying the velocity also introduced the variation of the lookahead used for error computation, using the same point for obtaining the curvature and errors, resulting $L_e = 0.4u$. Such adjustment of the lookahead distance provided with good results as well, being practically equal to the ones obtained for $L_e = 5$ [m]. However, as it was stated before, this could also mean that $L_e$ is being kept around the value of 5 metres, not taking full advantage of the regulation mechanism, at least not in these tracks. Nevertheless, it consists on an excellent solution as it provided good results, and it is expected to remain adaptable in situations where the velocity assumes larger values, ensuring the anticipation capability of the path-following strategy is maintained with it. Lastly, $L_e = 10$ [m] clearly shows that the length of this

parameter should be chosen carefully, if it is to be fixed, as can be seen from analysing the respective results for every simulation. In this case, the vehicle is trying to correct its course with the errors relative to the reference point, but such anticipation actually diminishes the path-tracking precision. A possible explanation for this is that the velocity adjustment taking into account the curvature already lowers the speed in a way that the lateral controller can effectively steer the vehicle, with a moderate lookahead for better results, and that larger values for it will actually initiate correction before it is needed.

Lastly, some remarks between the individual controllers is in order. The Stanley controller fulfilled its purpose in this work, providing a control strategy known to be successful as a basis of comparison for the remaining ones, and provided with general worse results regarding the crosstrack error, despite being the one that drove the car fastest. It also was the only one for which a lookahead $L_e = 5$ [m] distance did not always provide better results. The LQR with 3 states has the highest values for the RMS of the crosstrack error, similar to the Stanley, but has an additional downside of showing large values of steering oscillation, which is undesirable for both the vehicle and the tracking. It should analysed if a better tuning of the matrices for its LQR and Kalman gains generates less oscillation while retaining or even improving its tracking capabilities. The last three controllers are on pair with each other: the 2-state LQR and 4-state LQG consistently present lower values for the RMS of the crosstrack error, but the L1, having low values for such error as well, compensates with being much smoother regarding the steering, and also allowing the vehicle to drive faster.

These three controllers combined with a fixed lookahead distance of 5 metres, or a varying one as described, therefore represent the best path-following strategies implemented and evaluated in this dissertation, concluding this section, as well as its chapter.

# Chapter 7

# Conclusions and Future Work

In the previous chapter, an extensive analysis of the results obtained from the path-following approaches developed in chapter 4 and implemented in a simulation of a FST vehicle, described in section 3.1, was made, and the best ones that provided the best results were mentioned. As such, conclusions about what was achieved in this dissertation should be taken, as well as some notes about what should follow it.

## 7.1 Achievements

The objectives of this work were clearly stated in the beginning of this document, namely: model a vehicle that simulates a real FST car for lateral and velocity control; develop a path-planning algorithm that permits the autonomous navigation in a track, following a reference line obtained from the perception of the vehicle; implement and validate various controllers including such planning; and evaluate the performance of these controllers when implemented in the simulation of the real vehicle. Therefore, taking into account the obtained results, it can be said that these objectives were effectively fulfilled, but some remarks regarding the encountered limitations should be made.

Firstly, since the scope of this thesis was control, specially the lateral one, affecting the steering of the car, it was assumed that the waypoints to be followed were obtained previously from the perception. As such, in the simulations, if the vehicle was driven in a way that caused that none of this points was in front of the vehicle, it would stop and not complete the track, even if it was still inside its limits. This obviously does not correspond to a real scenario, as in a competition these limits, or the cones that define them, are the ones utilized to generate the line to be tracked. Nonetheless, this situation is not supposed to happen, as it would mean that the controller is not steering the vehicle appropriately, or that the velocity regulation is not compensating well enough, and there were few situations in which it happened. This suggests that the different path-following plus controller combinations are indeed satisfactory solutions, having more to gain if tested with a simulation of the perception algorithm, for instance.

The mechanism of using lookahead distances was successfully implemented both for error computation and for velocity regulation. In the first however, it was seen that it was kept at low values, almost coincid-

ing with a fixed value also applied to it in the different simulations. In order to avoid this, a distinct way of relating the velocity with the curvature could have been used in order to attain higher velocities, and thus having more variation in the values of the lookahead distances. Still, comparing different velocity profiles, when combined with all the other varying parameters, would result in an even larger number of simulations, and could result in the lost of focus in the scope of this thesis, which is the path planning and lateral control of a FST vehicle. The velocity control should also be the subject of some analysis: firstly, the model for testing did not account for longitudinal slip in the tyres nor drivetrain dynamics, and therefore the results could be misleading. Nevertheless, it still allowed to include longitudinal dynamics in the model and respective control, which contributes to a better and more realistic scenario for the lateral control analysis.

## 7.2 Future Work

From the notes regarding the achievements of this work and the limitations encountered, remarks should be made as to what may follow this dissertation. Firstly, it is recommended to evaluate the performance of the best solutions with a mechanism that simulates the perception algorithms, computing the way-points from the positions of the cones in both sides of the tracks, as well as a model of the vehicle that accounts for more detailed longitudinal dynamics. This would allow for the establishing of an initial combined solution for a driverless FST vehicle, as well as explore further the controllers implemented in this work as well as their limitations.

After this, some additional control strategies could be studied. From the overview provided in section 1.2, many of the cited works use Model Predictive Control or a variation of it to steer a vehicle, or a complex dynamic model of it in simulation, and thus these methods and their application to the current problem could be studied. Similarly, the adaptive capabilities of including machine learning, either in control or path-planning, would certainly prove themselves useful in a racing scenario where multiple laps are to be taken.

Some other several aspects regarding more realistic simulations should also be accounted for in future work, namely including some realistic models for the sensors as to evaluate how the performance of the path-following is influenced. Lastly, a reduced-scale model could also be developed, with the objective of assessing if these controllers and path-planning strategies are applicable in a real-world situation beyond simulation, including the referred perception as well, if possible.

# Bibliography


[1] G. Klančar, A. Zdešar, S. Blažič, and I. Škrjanc. *Wheeled Mobile Robotics: From Fundamentals Towards Autonomous Systems*. Elsevier, 2017.

[2] M. Dikmen and C. Burns. Trust in autonomous vehicles: The case of tesla autopilot and summon. In *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1093–1098, 2017.

[3] W. Schwarting, J. Alonso-Mora, and D. Rus. Planning and decision-making for autonomous vehicles. *Annual Review of Control, Robotics, and Autonomous Systems*, 1(1):187–210, 2018.

[4] S. M. Rathod and S. K. Apte. Obstacle detection using sensor based system for an four wheeled autonomous electric robot. In *2019 International Conference on Communication and Electronics Systems (ICCES)*, pages 493–497, 2019.

[5] H. Tian, J. Ni, and J. Hu. Autonomous driving system design for formula student driverless racecar. In *2018 IEEE Intelligent Vehicles Symposium (IV)*, pages 1–6, 2018.

[6] J. Kabzany, M. I. Valls, V. J. Reijgwart, H. F. Hendrikx, C. Ehmke, M. Prajapat, A. Buhler, N. Gosala, M. Gupta, R. Sivanesan, A. Dhall, E. Chisari, N. Karnchanachari, S. Brits, M. Dangel, I. Sa, R. Dubé, A. Gawel, M. Pfeiffer, A. Liniger, J. Lygeros, and R. Siegwart. Amz driverless: The full autonomous racing system. *Journal of Field Robotics*, 37(7):1267–1294, 2020.

[7] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics*, 37(3):363–386, 2020.

[8] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger. Learning-based model predictive control for autonomous racing. *IEEE Robotics and Automation Letters*, 4(4):3363–3370, 2019.

[9] A. Antunes, C. Cardeira, and P. Oliveira. Sideslip estimation of formula student prototype through gps/ins fusion. In *2017 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pages 184–191, 2017.

[10] O. Törő, T. Bécsi, and S. Aradi. Design of lane keeping algorithm of autonomous vehicle. *Periodica Polytechnica Transportation Engineering*, 44(1):60–68, 2016.

[11] R. Pepy, A. Lambert, and H. Mounier. Path planning using a dynamic vehicle model. In *2006 2nd International Conference on Information Communication Technologies*, volume 1, pages 781–786, 2006.

[12] M. Laurenza, G. Pepe, D. Antonelli, and A. Carcaterra. Car collision avoidance with velocity obstacle approach: Evaluation of the reliability and performace of the collision avoidance maneuver. In *2019 IEEE 5th International forum on Research and Technology for Society and Industry (RTSI)*, pages 465–470, 2019.

[13] J. Connors and G. Elkaim. Trajectory generation and control methodology for an autonomous ground vehicle. 2008.

[14] R. Şolea and U. Nunes. Trajectory planning and sliding-mode control based trajectory-tracking for cybercars. *Integrated Computer-Aided Engineering*, 14(1):33–47, 2007.

[15] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing. In *2007 American Control Conference*, pages 2296–2301, 2007.

[16] J. Snider. Automatic steering methods for autonomous automobile path tracking. Technical report, Carnegie Mellon University, 2009.

[17] P. Polack, F. Altché, B. d'Andréa-Novel, and A. de La Fortelle. The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles? In *2017 IEEE Intelligent Vehicles Symposium (IV)*, pages 812–818, 2017.

[18] R. Rajamani. *Vehicle Dynamics and Control*. Springer, 2012.

[19] R. A. Cordeiro, J. R. Azinheira, E. C. de Paiva, and S. S. Bueno. Dynamic modeling and bio-inspired lqr approach for off-road robotic vehicle path tracking. In *2013 16th International Conference on Advanced Robotics (ICAR)*, pages 1–6, 2013.

[20] O. Amidi, C. E. Thorpe, W. H. Chun, and W. J. Wolfe. Integrated mobile robot control. In *Mobile Robots V*, volume 1388, pages 504 – 523, 1991.

[21] M. M. Quan, Y. Zhai, Y. Jiang, Y. J. Sun, D. L. Xu, and J. W. Gong. An improved pure pursuit algorithm for four-wheel-steering autonomous driving vehicle. In *Sensors, Mechatronics and Automation*, volume 511 of *Applied Mechanics and Materials*, pages 958–962, 2014.

[22] E. Maalouf, M. Saada, and H. Saliah. A higher level path tracking controller for a four-wheel differentially steered mobile robot. *Robotics and Autonomous Systems*, 54(1):23 – 33, 2006.

[23] P. Zhao, J. Chen, Y. Song, X. Tao, T. Xu, and T. Mei. Design of a control system for an autonomous vehicle based on adaptive-pid. *International Journal of Advanced Robotic Systems*, 9(2):44, 2012.

[24] K. Ogata. *Modern Control Engineering*. Prentice Hall, 2009.

[25] G. F. Franklin, J. D. Powell, and A. Emami-Naeini. *Feedback Control of Dynamic Systems*. Addison-Wesley Publishing Company, 1995.

[26] B. Friedland. *Control System Design: An Introduction to State-Space Methods*. Dover, 1986.

[27] J. M. Lemos. *Controlo no Espaço de Estados*. IST Press, 2019.

[28] J. P. Hespanha. *Linear Systems Theory*. Princeton University Press, 2009.

[29] A. B. Moutinho. *Modeling and Nonlinear Control for Airship Autonomous Flight*. PhD thesis, Instituto Superior Técnico - Universidade Técnica de Lisboa, 2007.

[30] S. Thrun, M. Montemerlo, H. Dahlkamp, and et al. Stanley: The robot that won the darpa grand challenge. *Journal of Field Robotics*, 23(9):661–692, 2006.

[31] S. Park, J. Deyst, and J. How. A new nonlinear guidance logic for trajectory tracking. 2004.

[32] M. Montemerlo, J. Becker, S. Bhat, and et al. Junior: The stanford entry in the urban challenge. *Journal of Field Robotics*, 25(9):569–597, 2008.

[33] H. Pacejka and E. Bakker. The magic formula tyre model. *Vehicle System Dynamics*, 21:1–18, 1993.

[34] H. Dugoff, P. S. Fancher, and L. Segel. Tire performance characteristics affecting vehicle response to steering and braking control inputs: final report. Technical report, Highway Safety Research Institute - University of Michigan, 1969.

[35] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009.

[36] S. C. Chapra and R. P. Canale. *Numerical Methods for Engineers*. McGraw-Hill, 2006.

# Appendix A

# Validation of the controllers

The validation of the linear controllers is presented in this appendix, as explained on section 4.3. As it was done with the previous validation processes, four images will be presented for each controller: two of them will correspond to the validation using a line segment as reference; and the remaining two will be similar, but with a circumference as the path to be followed. In both cases, for all controllers, the path to be tracked will be presented in blue, while the path the simulated vehicle describes is represented in red. Lastly, the cross-track error $e_y$ and the orientation error $e_\psi$ are also presented on the figures at the right for each validation scenario.

- Validation of the LQR controller with 2 states:

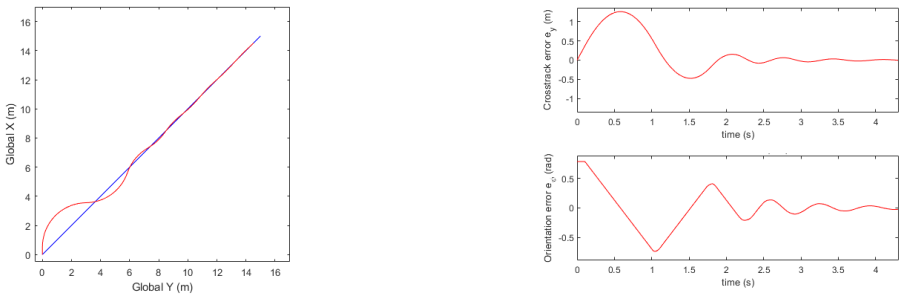


Figure A.1: 2-state LQR controller tracking a line segment (left), and resulting errors (right).

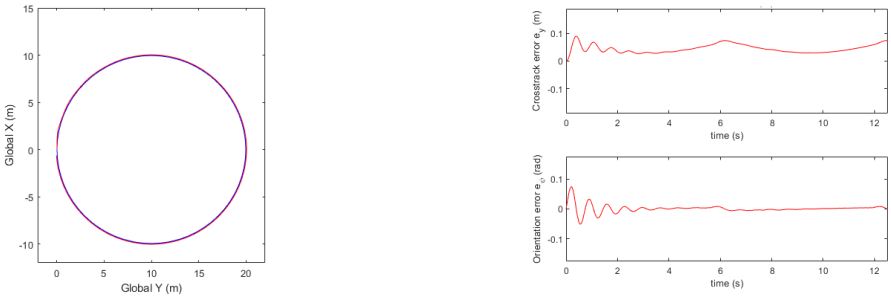


Figure A.2: 2-state LQR controller tracking a circumference (left), and resulting errors (right).

- Validation of the LQG controller with 3 states:

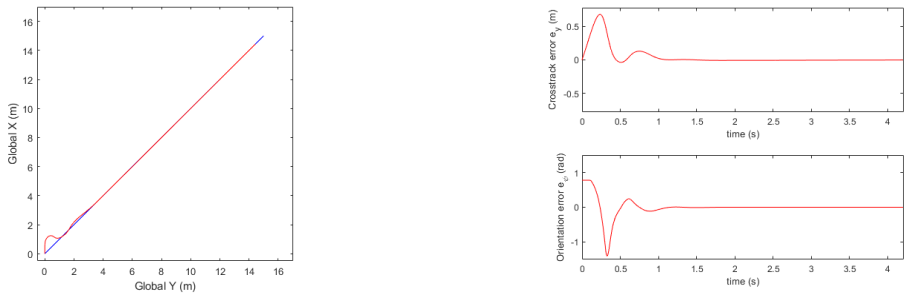


Figure A.3: 3-state LQG controller tracking a line segment (left), and resulting errors (right).

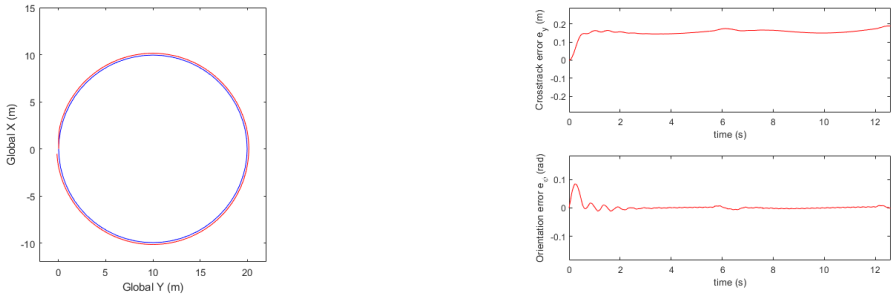


Figure A.4: 3-state LQG controller tracking a circumference (left), and resulting errors (right).

- Validation of the LQG controller with 4 states:

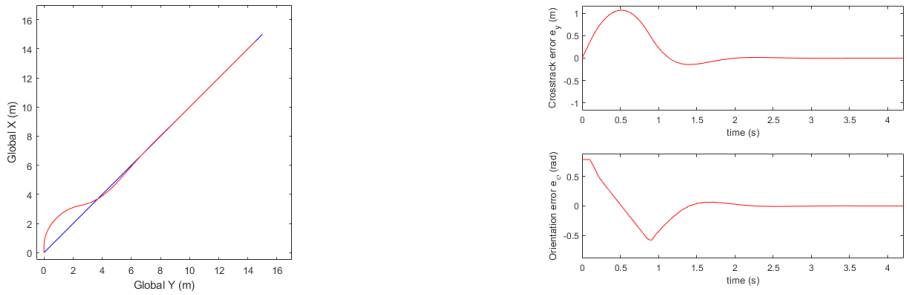


Figure A.5: 4-state LQG controller tracking a line segment (left), and resulting errors (right).

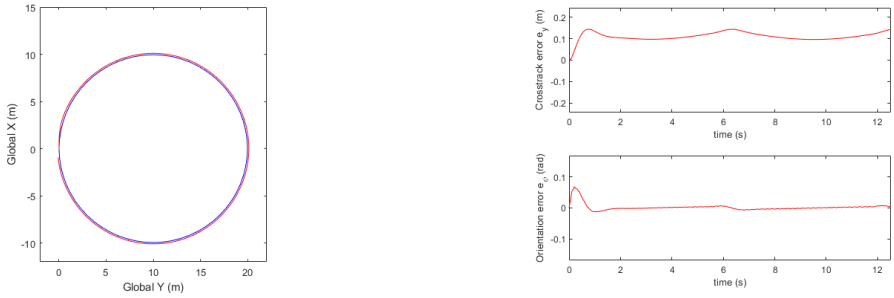


Figure A.6: 4-state LQG controller tracking a circumference (left), and resulting errors (right).

# Appendix B

# Results at varying velocity

In this last appendix, the results for the simulations are shown, in order to complement tables 6.3, 6.4 and 6.4. Each section corresponds to the results for the simulations in a given track, and for each controller. The figures on the left will represent the track, with its right and left boundaries depicted in red and blue, respectively, whilst the black sold line stands for the path described by the vehicle. On the other hand, the figures on the right will show the steering angle $\delta$ provided to the system in the top figure, and the cross-track errors for evaluation, $e^1_{eval}$ and $e^2_{eval}$, represented in black and green, respectively, together with the distance to the limit to the right of the track, in red, and to the left, in blue.

## B.1 Results for Track 1

- Stanley controller:


Figure B.1: Results for the Stanley controller without a lookahead distance for Track 1.


Figure B.2: Results for the Stanley controller with a 5 metre lookahead distance for Track 1.


Figure B.3: Results for the Stanley controller with a 10 metre lookahead distance for Track 1.


Figure B.4: Results for the Stanley controller with $L_e$ automatically regulated for Track 1.

- <u>L1 controller:</u>


Figure B.5: Results for the L1 controller with a 5 metre lookahead distance for Track 1.


Figure B.6: Results for the L1 controller with a 10 metre lookahead distance for Track 1.


Figure B.7: Results for the L1 controller with $L_e$ automatically regulated for Track 1.

- LQR controller with 2 states:


Figure B.8: Results for the 2-state LQR controller without a lookahead distance for Track 1.


Figure B.9: Results for the 2-state LQR controller with a 5 metre lookahead distance for Track 1.


Figure B.10: Results for the 2-state LQR controller with a 10 metre lookahead distance for Track 1.


Figure B.11: Results for the 2-state LQR controller with $L_e$ automatically regulated for Track 1.

- <u>LQG controller with 3 states:</u>


Figure B.12: Results for the 3-state LQG controller without a lookahead distance for Track 1.


Figure B.13: Results for the 3-state LQG controller with a 5 metre lookahead distance for Track 1.


Figure B.14: Results for the 3-state LQG controller with a 10 metre lookahead distance for Track 1.


Figure B.15: Results for the 3-state LQG controller with $L_e$ automatically regulated for Track 1.

- LQG controller with 4 states:


Figure B.16: Results for the 4-state LQG controller without a lookahead distance for Track 1.


Figure B.17: Results for the 4-state LQG controller with a 5 metre lookahead distance for Track 1.


Figure B.18: Results for the 4-state LQG controller with a 10 metre lookahead distance for Track 1.


Figure B.19: Results for the 4-state LQG controller with $L_e$ automatically regulated for Track 1.

## B.2 Results for Track 2

- Stanley controller:

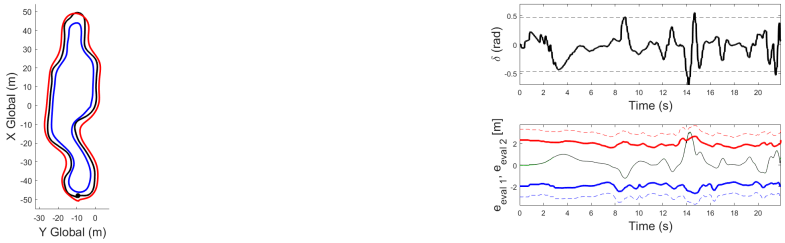


Figure B.20: Results for the Stanley controller without a lookahead distance for Track 2.

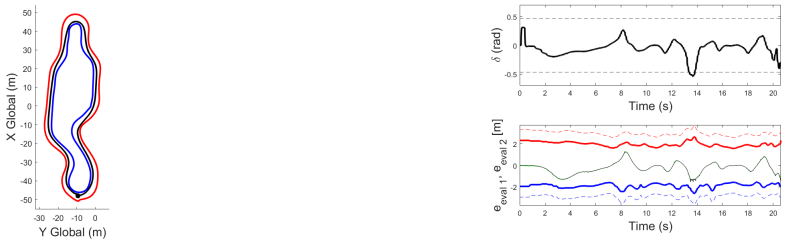


Figure B.21: Results for the Stanley controller with a 5 metre lookahead distance for Track 2.

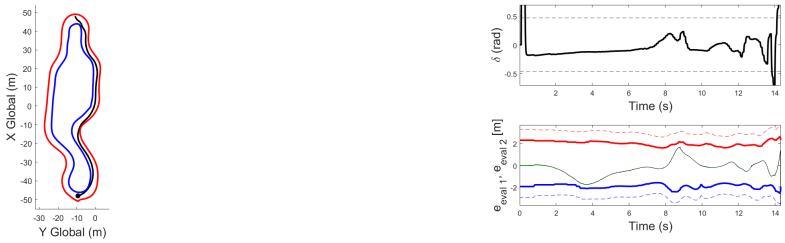


Figure B.22: Results for the Stanley controller with a 10 metre lookahead distance for Track 2.

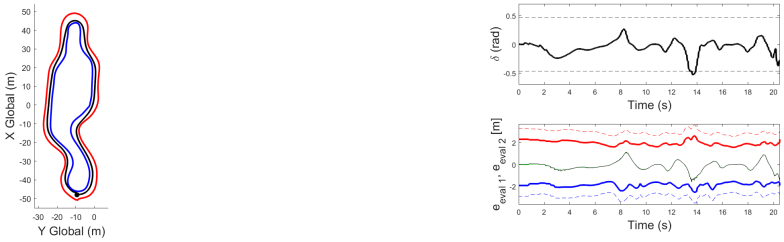


Figure B.23: Results for the Stanley controller with $L_e$ automatically regulated for Track 2.

- L1 controller:

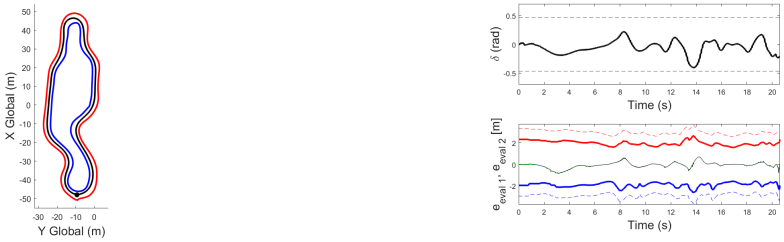


Figure B.24: Results for the L1 controller with a 5 metre lookahead distance for Track 2.

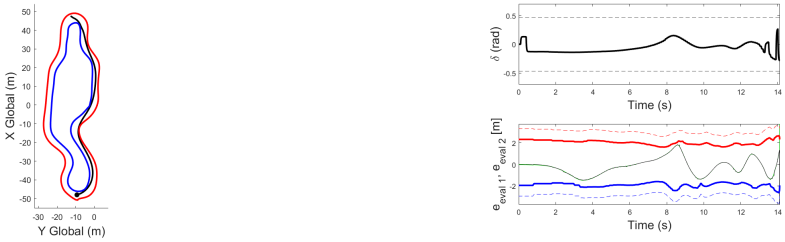


Figure B.25: Results for the L1 controller with a 10 metre lookahead distance for Track 2.

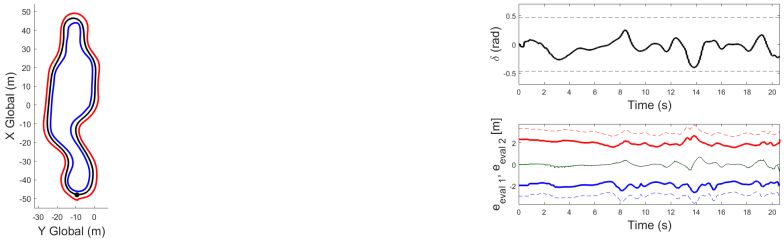


Figure B.26: Results for the L1 controller with $L_e$ automatically regulated for Track 2.

- LQR controller with 2 states:

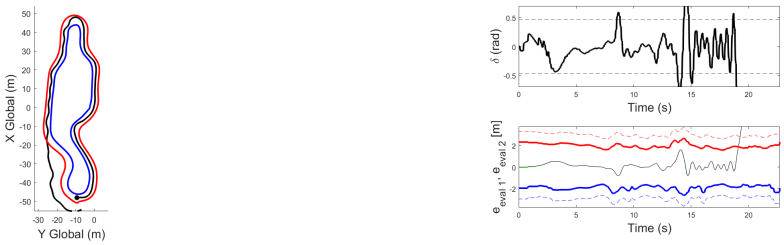


Figure B.27: Results for the 2-state LQR controller without a lookahead distance for Track 2.

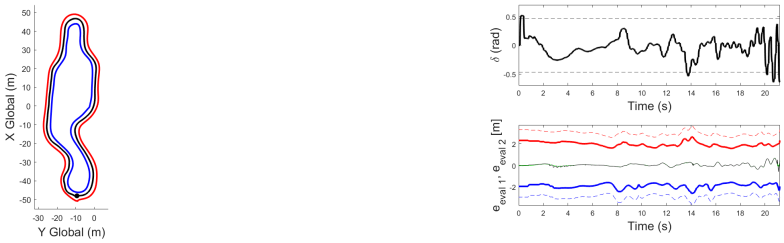


Figure B.28: Results for the 2-state LQR controller with a 5 metre lookahead distance for Track 2.

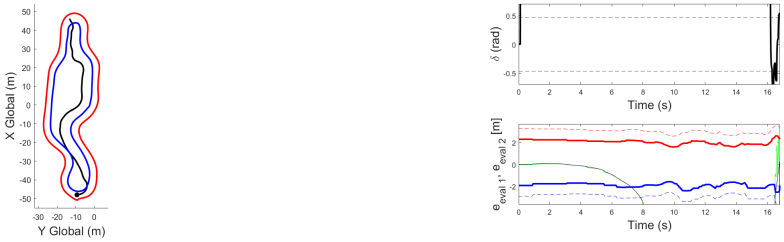


Figure B.29: Results for the 2-state LQR controller with a 10 metre lookahead distance for Track 2.

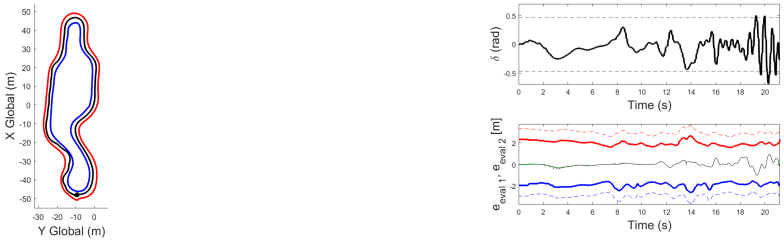


Figure B.30: Results for the 2-state LQR controller with $L_e$ automatically regulated for Track 2.

- LQG controller with 3 states:

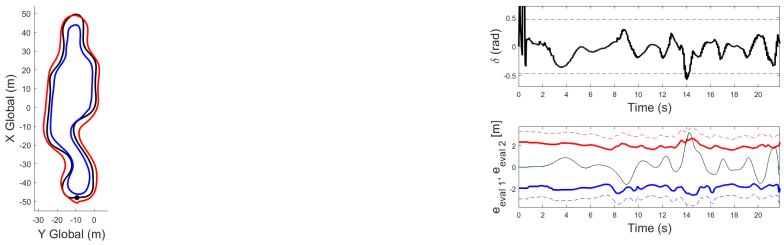


Figure B.31: Results for the 3-state LQG controller without a lookahead distance for Track 2.

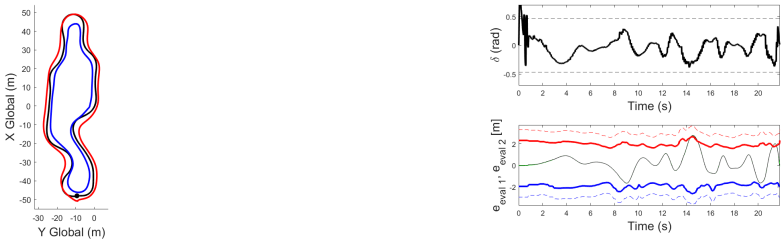


Figure B.32: Results for the 3-state LQG controller with a 5 metre lookahead distance for Track 2.

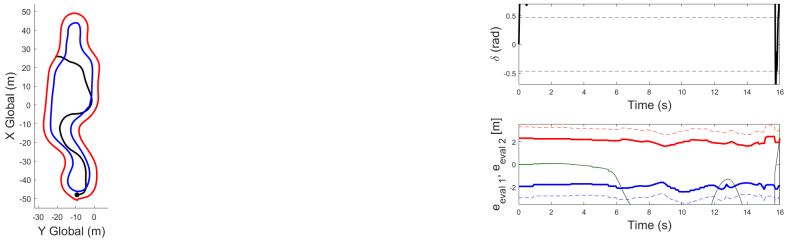


Figure B.33: Results for the 3-state LQG controller with a 10 metre lookahead distance for Track 2.

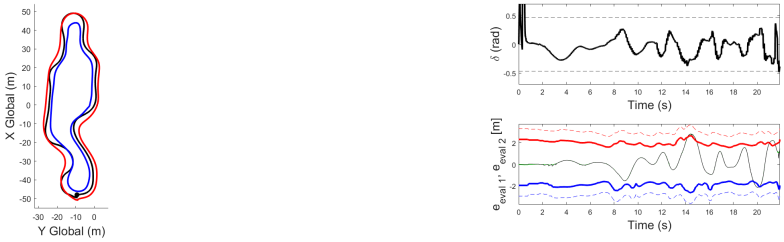


Figure B.34: Results for the 3-state LQG controller with $L_e$ automatically regulated for Track 2.

- LQG controller with 4 states:

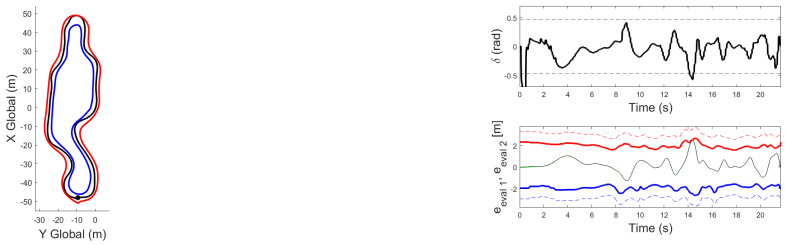


Figure B.35: Results for the 4-state LQG controller without a lookahead distance for Track 2.

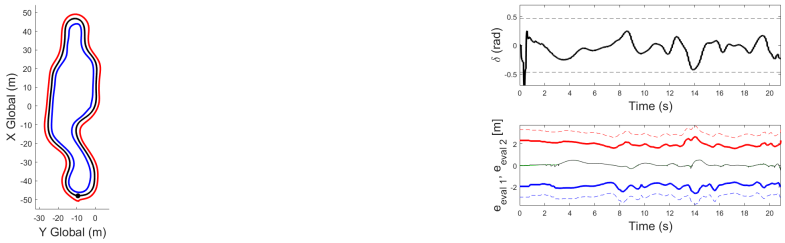


Figure B.36: Results for the 4-state LQG controller with a 5 metre lookahead distance for Track 2.

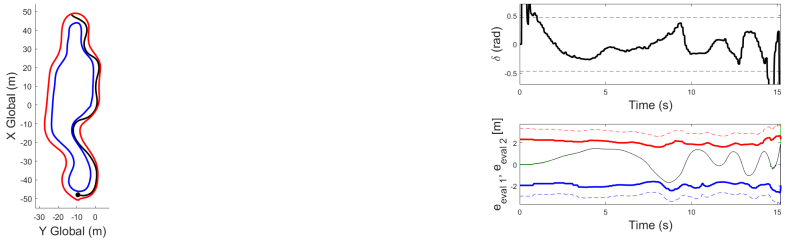


Figure B.37: Results for the 4-state LQG controller with a 10 metre lookahead distance for Track 2.

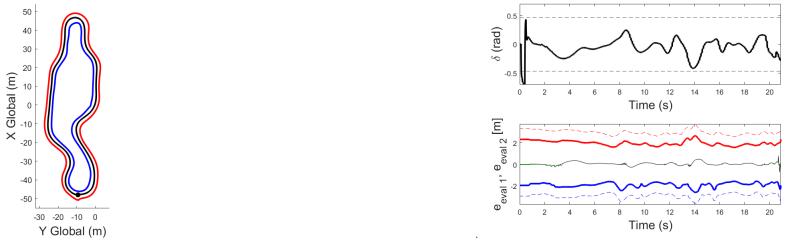


Figure B.38: Results for the 4-state LQG controller with $L_e$ automatically regulated for Track 2.

## B.3 Results for Track 3

- Stanley controller:


Figure B.39: Results for the Stanley controller without a lookahead distance for Track 3.

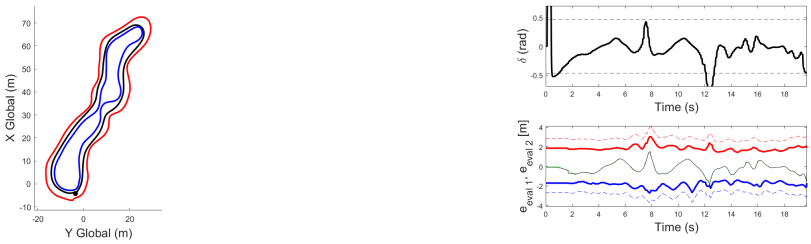


Figure B.40: Results for the Stanley controller with a 5 metre lookahead distance for Track 3.

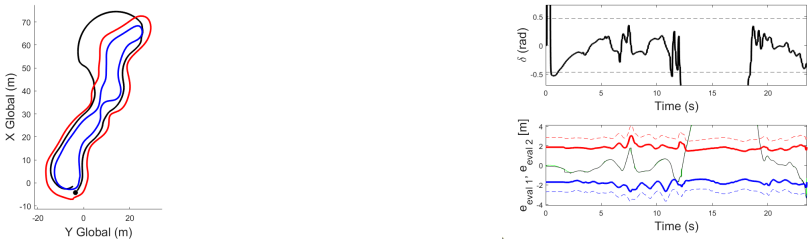


Figure B.41: Results for the Stanley controller with a 10 metre lookahead distance for Track 3.

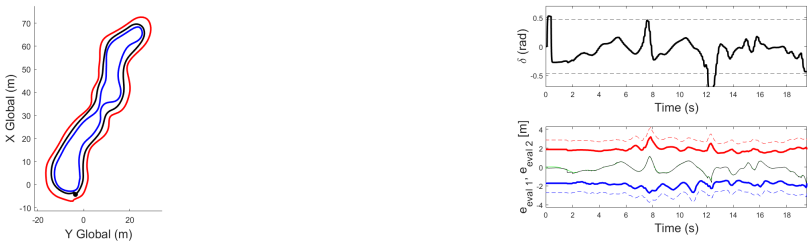


Figure B.42: Results for the Stanley controller with $L_e$ automatically regulated for Track 3.

- L1 controller:

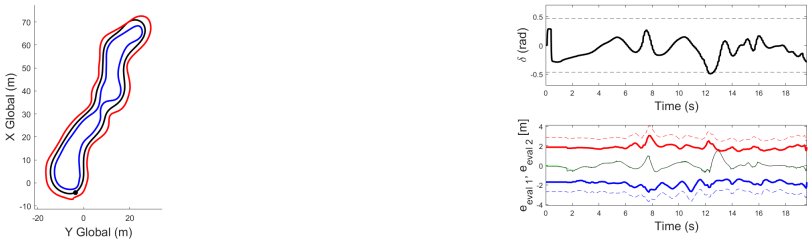


Figure B.43: Results for the L1 controller with a 5 metre lookahead distance for Track 3.

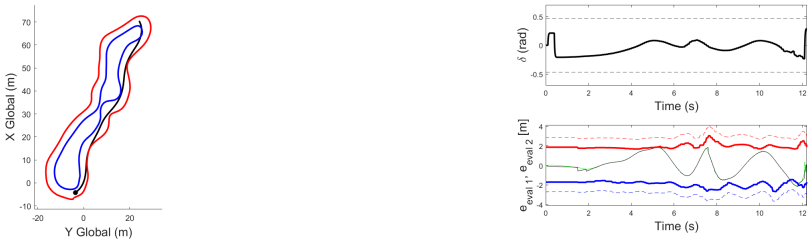


Figure B.44: Results for the L1 controller with a 10 metre lookahead distance for Track 3.

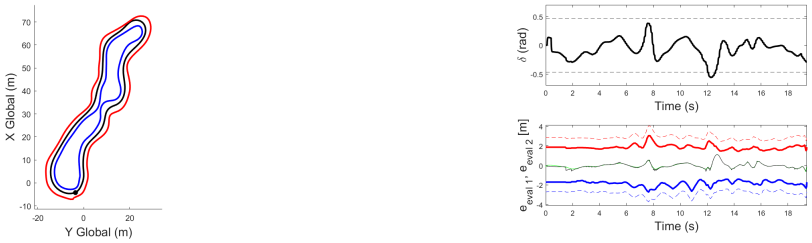


Figure B.45: Results for the L1 controller with $L_e$ automatically regulated for Track 3.

- LQR controller with 2 states:

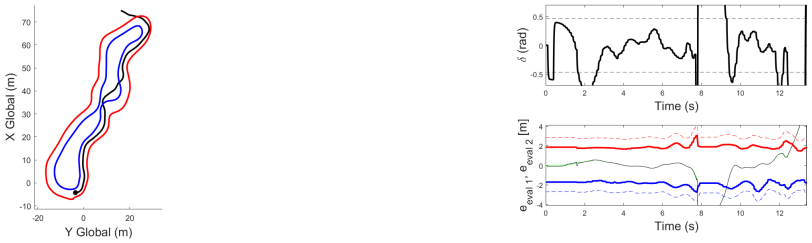


Figure B.46: Results for the 2-state LQR controller without a lookahead distance for Track 3.

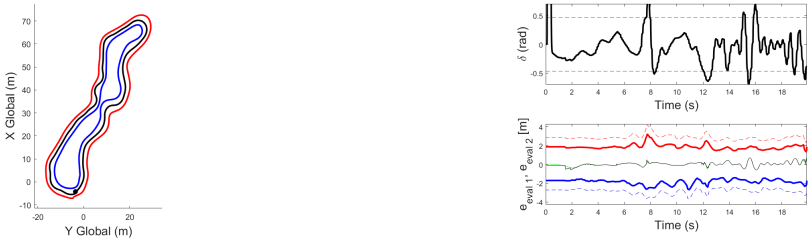


Figure B.47: Results for the 2-state LQR controller with a 5 metre lookahead distance for Track 3.

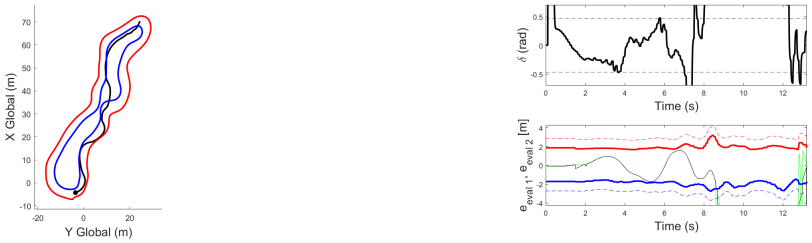


Figure B.48: Results for the 2-state LQR controller with a 10 metre lookahead distance for Track 3.

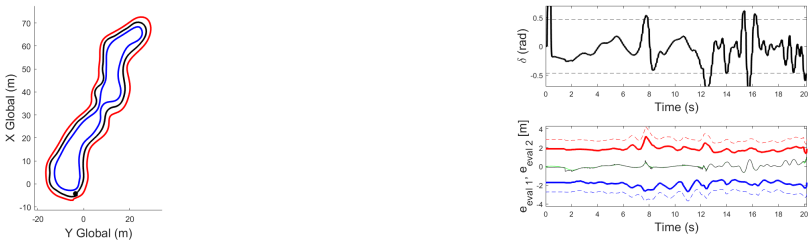


Figure B.49: Results for the 2-state LQR controller with $L_e$ automatically regulated for Track 3.

- LQG controller with 3 states:

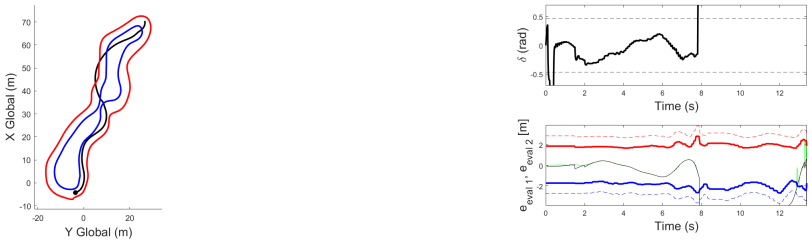


Figure B.50: Results for the 3-state LQG controller without a lookahead distance for Track 3.

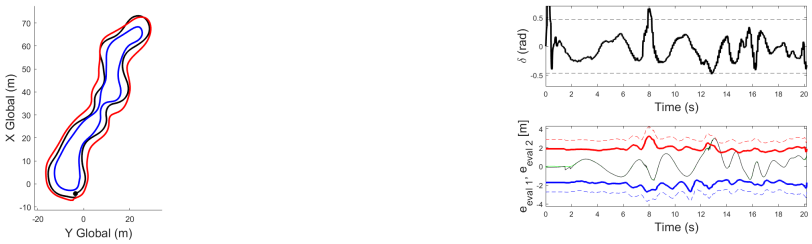


Figure B.51: Results for the 3-state LQG controller with a 5 metre lookahead distance for Track 3.

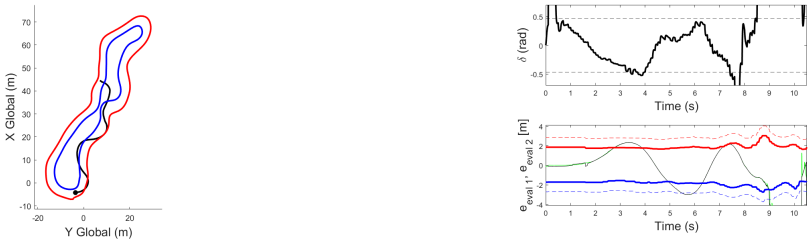


Figure B.52: Results for the 3-state LQG controller with a 10 metre lookahead distance for Track 3.

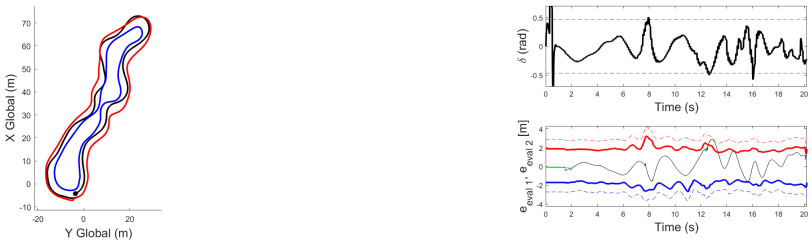


Figure B.53: Results for the 3-state LQG controller with $L_e$ automatically regulated for Track 3.

- LQG controller with 4 states:

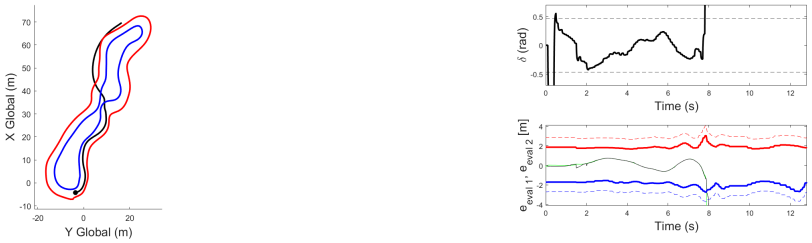


Figure B.54: Results for the 4-state LQG controller without a lookahead distance for Track 3.


Figure B.55: Results for the 4-state LQG controller with a 5 metre lookahead distance for Track 3.

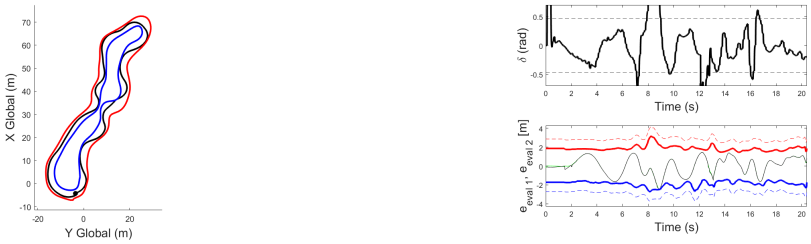


Figure B.56: Results for the 4-state LQG controller with a 10 metre lookahead distance for Track 3.

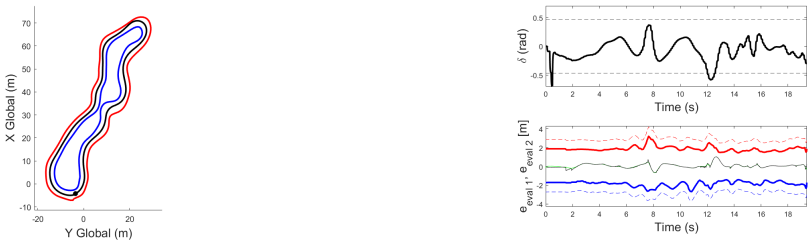


Figure B.57: Results for the 4-state LQG controller with $L_e$ automatically regulated for Track 3.